SpOdy

User manual — v0.4.1-beta

Desktop frontend for the SpOdy orbital propagator.
Patran-style file-based workflow: fill a TOML, run the binary, inspect the output.

Table of contents

1. Introduction

SpOdy is a desktop application for propagating spacecraft and debris orbits around the Moon or Earth. You give it a mission scenario described in a plain-text file — central body, initial state, force model, integrator settings — and it produces a trajectory you can plot, compare against a reference, or sweep across thousands of variant cases in a batch run.

The application is shipped as a single self-contained folder. There is **no installer**, **no Python environment to configure**, and **no network registration** beyond the one-time download of the public ephemeris and gravity-model data the wizard handles for you on first launch. Extract the archive, run `spody-gui.exe`, and you are minutes away from your first propagation.

1.1 Who this manual is for

This manual assumes you have received a SpOdy bundle — a folder containing `spody-gui.exe` plus its supporting files — and want to understand how to use it. No prior orbital-mechanics expertise is required to follow the chapters in order, although a working knowledge of state vectors, Keplerian elements, and inertial frames will help you make the most of the analysis features.

If you are looking for the development guide (building from source, extending the integrator, contributing to the C core) you want a different document — that one ships separately.

1.2 What SpOdy does

At its core, SpOdy is a numerical orbit propagator with a desktop front-end. The split is deliberate: the integration physics lives in a fast C engine (`spody.exe`) that runs on its own as a command-line tool, and the graphical interface (`spody-gui.exe`) is a thin PySide6 application that orchestrates the inputs, dispatches the runs, and visualises the outputs. The two halves talk only through files — **the GUI never links C code directly**, which makes the same C engine usable from scripts, notebooks, or the upcoming batch automation features without any GUI surface area.

In one window you can:

- **Build** the simulation input through a structured form with live TOML preview, per-field range checks, and inline validation against the C parser, eliminating the need to remember any TOML syntax.
- **Run** a single propagation or a parameter sweep across thousands of cases, streaming the engine's terminal output into an embedded pane.
- **Analyse** the binary outputs through a catalogue of plots (state vectors, orbit shape, classical orbital elements, per-force acceleration breakdown, eclipse fractions, event timelines) in both 2D matplotlib and embedded 3D VTK canvases.
- **Compare** two runs side-by-side through dedicated difference plots, with automatic cubic-Hermite interpolation when the two output grids do not align sample-by-sample.

1.3 What is in the bundle

When you extract the distribution archive you get a folder structured like this:

```
spody-gui/
├── spody-gui.exe          ← the desktop application (this manual's subject)
├── spody.exe              ← the C engine spody-gui drives
├── data/                  ← populated by the setup wizard on first launch
├── examples/              ← starter TOML scenarios you can open
└── _internal/             ← bundled Python interpreter + libraries
```

The `_internal/` folder is an implementation detail of the PyInstaller bundling and you should treat it as opaque: never edit, delete, or rename files inside it. Everything you interact with is either the two executables, the example scenarios, or the wizard-populated `data/` folder.

1.4 How to read this manual

The first three chapters are sequential: chapter 2 walks you through the first launch and the setup wizard, chapter 3 introduces the main window and the run-time workflow. From chapter 4 onward the organisation switches to reference-style: schema details for every TOML field, a catalogue of every plot, frame conventions, and CLI flags for the `spody.exe` engine.

A consistent set of typographical conventions runs through the chapters:

- `monospaced text` denotes literal file paths, code snippets, and things you type or that the application displays verbatim;
- Ctrl + R shows a keyboard shortcut;
- **Bold** highlights UI controls (menu items, button labels, panel headings) as they appear in the application;
- *italics* introduce a new term the chapter will then use.

Throughout the manual you will also encounter call-outs in the margin:

> *Quotation blocks are used for verbatim excerpts of file content, log messages, or longer passages from the application's interface.*

Note, Tip, and Warning admonitions flag information that is, in order, useful background, a practical shortcut, and something you should not skip without thinking.

2. Installation and first launch

SpOdy is distributed as a single archive containing everything it needs to run. This chapter takes you from receiving that archive to seeing your first propagation results on screen, including the mandatory one-time setup wizard.

2.1 Prerequisites

The minimum requirements are unsurprising for a desktop scientific application:

- **Windows 10 (64-bit) or Windows 11**. The bundle is built for `x86_64`; ARM-based devices are not supported in this release.
- About **600 MB of free disk space** for the application itself, plus **roughly 30 MB to 350 MB** for the external data files the setup wizard downloads (the exact figure depends on which ephemeris coverage profile you choose — see chapter 3).
- A working **internet connection** the first time you run the application, so the setup wizard can fetch the planetary ephemeris and lunar gravity-model data from NASA's public servers. After the first run, SpOdy works fully offline.
- A graphics adapter that supports **OpenGL 3.2 or higher**. Any Windows-supported integrated GPU from the last decade is fine; the VTK-based 3D viewer falls back to software rendering when no hardware acceleration is available, with a small frame-rate cost.

You do **not** need Python installed: the bundle ships with its own Python interpreter plus every library it depends on. You also do not need administrator privileges to install or run SpOdy.

2.2 Installing

Installation in the conventional sense does not exist: there is no installer wizard, no Start menu entry created automatically, and no registry keys written. SpOdy is fully portable.

1. Locate the archive you received. It is named in the form `spody-gui-<version>-win64.zip` (or `.7z` for the smaller compressed variant).
2. Right-click the archive and pick **Extract All…** — or use any archiver such as 7-Zip if you prefer.
3. Choose a destination folder. SpOdy stores all its data inside the extracted folder, so pick somewhere you have write access:
   - **Your user folder**, for example `C:\Users\<you>\Apps\spody-gui\`, is the safest default.
   - **An external drive** (USB stick, network share) works too if you want a portable installation you can carry between machines.
   - `C:\Program Files\` is *not* recommended: the wizard needs to write the downloaded data files into the same folder, and Windows restricts write access there to administrators.
4. Open the extracted folder and confirm you can see at least these four items:
   - `spody-gui.exe`
   - `spody.exe`

- `data/` (empty at this stage)
- `_internal/` (do not touch)

The bundle is now ready to launch.

2.2.1 Verifying the bundle (optional)

If you want to make sure the archive transferred without corruption before running anything, the distribution comes with a `SHA256SUMS` text file listing the expected SHA-256 hash of each top-level file. From a PowerShell prompt opened in the extraction folder:

```
Get-FileHash spody-gui.exe -Algorithm SHA256
Get-FileHash spody.exe     -Algorithm SHA256
```

Compare the hexadecimal values against the matching lines in `SHA256SUMS`. Mismatches mean the download was truncated or tampered with — redownload from the original source.

2.3 First launch

Double-click `spody-gui.exe`. After a brief warm-up the main window appears, with two tabs at the top (**Run** and **Analysis**) and an empty menu bar.

Because no data files are present yet, you will immediately see two modal pop-ups:

1. An informational dialog announcing **"Setup needed"** with the path to the data folder it expects to populate (always `data/` relative to `spody-gui.exe`).
2. Following that, the **Setup wizard** itself.

The wizard is the topic of the next chapter. For now it is enough to know that until you complete it, the application will refuse to launch any propagation: a *hard run guard* checks that all required data files are present before every run and will pop the wizard back up if any are missing.

> *The first launch can take 5 to 10 seconds before the window becomes responsive while the bundled Python interpreter and the Qt framework warm up. Subsequent launches are typically under a second — the operating system caches the bundle in memory.*

2.4 Settings file location

SpOdy persists a handful of user preferences (the path to `spody.exe`, the data-folder override, the recent-files list, the 3D Moon texture choice, the last-selected ephemeris coverage profile) through the standard Windows registry under `HKEY_CURRENT_USER\Software\SpOdy\SpOdy`.

Nothing inside the bundle folder is modified by writing settings, so the bundle remains fully portable: you can copy or move the folder to a different machine, and the application will start fresh with the wizard on first launch there, ignoring whatever settings remain in the old machine's registry.

To **reset SpOdy to a clean state** without uninstalling, delete the registry key above (via `regedit`) and the `data/` folder next to the executable. The next launch will behave exactly as if you had just extracted the bundle.

2.5 Uninstalling

Drag the extracted folder to the Recycle Bin. That is the entire uninstallation procedure. There is nothing else to clean up except the registry key mentioned above, which is harmless if left behind.

3. The setup wizard

A fresh SpOdy bundle cannot run a propagation. Several large data files — a planetary ephemeris, a gravity-model coefficient set, and (for Earth-centred runs) the IERS Earth-orientation tables — are needed before the first integration step, and none is shipped inside the archive (they are externally maintained, publicly distributed, and together comprise hundreds of megabytes that change rarely). The setup wizard is the dialog that downloads those files into the bundle's `data/` folder, then converts the raw ephemeris and gravity coefficients into the internal binary formats the engine expects. You run it once at first launch and never think about it again, unless you choose later to widen the ephemeris coverage, upgrade to a different gravity model, or refresh the Earth-orientation tables with a newer IERS bulletin.

3.1 When the wizard appears

The wizard pops in three situations:

1. **At first launch**, when no data files are present in the bundle's `data/` folder.
2. **At any later launch**, when one or more required files are missing or corrupt (this can happen if you delete the `data/` folder by hand, or if the previous download was interrupted and left a half-written file).
3. **On demand**, when you choose **Settings › Setup wizard…** from the menu bar. You typically reach for this when you want to switch ephemeris coverage profiles, add an extra gravity model, or simply re-verify that everything is in order.

While the wizard is open, the rest of the application keeps running in the background. You can close the wizard at any time with the **Close** button at the bottom-right; nothing you have downloaded so far is lost.

3.2 What the wizard manages

The wizard is structured as a list of *assets*, one card per file it needs to put on disk. Each card shows:

- a **status icon** indicating whether the file is present (✓ green), absent (✗ red), or present but too small to be trustworthy (⚠ amber);
- the asset's **human name** and, if present, its on-disk size;
- the **download URL** in an editable field (more on this in section 3.4);
- a **progress bar** (mostly idle until you press Download);
- a **Download** button on the right that toggles to **Cancel** while a transfer is in flight.

The assets fall into three categories:

Raw required assets are files SpOdy fetches verbatim from public servers and absolutely needs to run a propagation:

- the JPL `DE440` planetary ephemeris (header + one or more ASCII chunks); the wizard offers two coverage profiles, see section 3.2;
- the GRGM1200B lunar harmonic-gravity coefficients (the `.tab` file with the spherical-harmonic coefficients and a small `.lbl` metadata sidecar from the PDS Geosciences Node);

- the EIGEN-6C4 Earth gravity-model `.gfc` file (ICGEM format, ~178 MB) used when `central_body = "Earth"`;
- the IERS Earth-orientation files (`finals2000A.all` from the IERS Rapid Service plus the three IAU 2006 X/Y/s+XY/2 series tables `tab5.2{a,b,d}.txt`) needed by the engine's `R_ICRF↔ITRS` rotation;
- the CelesTrak combined space-weather table (`SW-All.csv`, daily F10.7 + 3-hour Ap) feeding the NRLMSISE-00 drag density model, needed only for runs with `force_model.drag = true`. Its card shows the date of the last *observed* row after download. All of these are Earth-specific.

Derived assets are files SpOdy produces from raw ones on your machine:

- `de440.spody`, the internal binary format the engine expects. Built once from the downloaded DE440 ASCII chunks by invoking the `spody.exe convert ephemeris` subcommand under the hood;
- `eigen-6c4.tab`, the GRGM-style coefficient table the engine reads for Earth harmonics, converted from `EIGEN-6C4.gfc` by the `spody.exe convert harmonics_icgem` subcommand. The resulting `.tab` is ~252 MB on disk (2.4 M coefficient rows up to degree 2190).

Both derived assets are produced **automatically** as soon as the raw inputs are complete; you never click a button for them. The wizard streams the converter's progress (chunk id for DE440, running `n = D / D_max` for the harmonics converter) into the **Conversion (auto)** status line at the bottom of the dialog.

Optional raw assets are files SpOdy can use to enrich the experience but does not require to run:

- the NASA SVS LROC color Moon texture (2K equirectangular TIFF, ~3 MB). When present, the Analysis tab renders the 3D Moon and the impact lat/lon backgrounds with the actual lunar surface; when absent, the views fall back to a flat-grey sphere;
- the NASA Visible Earth "Blue Marble" December 2004 texture (equirectangular JPEG, ~3 MB). Same role for Earth-centred scenes: when present, the 3D viewer renders Earth with the actual continents and oceans; when absent, the central-body sphere stays flat-grey. When the Moon appears as a *third body* in an Earth-centred scene, this is also the texture that paints its body-fixed marker so the Moon is still recognisable at its true ~384,000 km distance;
- the Solar System Scope Milky Way 8K star map (equirectangular JPEG, ~6 MB, CC BY 4.0). When present and the *Show starfield* toggle in the Scene options dialog is enabled, the 3D Analysis scene replaces the dark background with a real star map. The texture is re-projected on first use to align with the ICRF axes (catalogue stored in galactic coords; SpOdy chains a J2000 ICRF→galactic rotation before the lookup so the bulge ends up at ICRF -Y), with the rotated copy cached on disk for instant subsequent loads.

The three textures are asked for on demand by clicking **Download** on their respective cards — the *Download all missing* button intentionally leaves them alone so a fresh install does not pay the download cost unless the user wants the textures.

Every asset carries internal metadata identifying its **central body** (`Moon`, `Earth`, or body-agnostic for the DE440 ephemeris) and its **category** (`harmonics`, `ephemeris`, `texture`, `eop`, `iau2006`, …). The TOML form's dropdowns in `[force_model].harmonics_file`, `[force_model] .eop_file`, `[force_model].iau2006_dir` and `[ephemeris].file` use that metadata to show only the assets that apply to the currently-selected `central_body` — the user never has to type a path by hand. The Earth-only rows (`eop_file`, `iau2006_dir`) appear in the form only when `central_body = "Earth"`.

3.3 Choosing an ephemeris coverage profile

The DE440 planetary ephemeris is split into ~100-year chunks. SpOdy gives you two pre-set profiles, picked through a radio at the top of the wizard:

Profile	Chunks	Coverage window	Download size
Modern era *(default)*	1 chunk (`ascp01950.440`)	1950 – 2050	~30 MB
Full pack	11 chunks (`ascp01550..ascp02550`)	1550 – 2650	~340 MB

The default is *Modern era*. It is the right pick for anyone running near-present scenarios, which means almost everyone reading this manual: any simulation epoch from 1950 to 2050 is covered exactly, with the same accuracy as the full pack. The full pack only matters for historical reconstructions (lunar landings of the late 1960s, the Apollo program, deep-time analyses of orbital secular drift) or far-future scenarios.

Switching coverage at any time is non-destructive: changing the radio rebuilds the asset list to reflect the new requirement, but nothing is deleted from disk. So if you start with *Modern era*, later upgrade to *Full pack*, then change your mind, the extra chunks remain on disk and the wizard simply stops requiring them. The derived `de440.spody` always includes every chunk it finds in the `DE440/` folder, regardless of profile, so you never lose coverage by toggling.

> *Storage for the Full pack profile is dominated by the eleven ASCII chunks (~30 MB each); the derived `de440.spody` from the full set is about 100 MB on disk.*

3.4 Downloading the data

Press **Download** on a single card to fetch that file. For a fresh install the fast path is the **Download all missing** button in the wizard's footer: it walks every card whose state is `✗` or `⚠` and starts the download. Cards in flight show their progress in the bar; you can cancel one without affecting the others by clicking the **Cancel** button that replaces **Download** during a transfer.

Failed downloads do not leave corrupted files behind: SpOdy writes each transfer to a `.part` sidecar first and renames it to the final name only after the HTTP response completes successfully. If the connection drops mid-transfer, the next download attempt starts over rather than appending to a stale partial.

3.4.1 Why the URL field is editable

The URLs SpOdy ships with point at the canonical sources we have verified work at the time of the release:

- JPL's FTP-over-HTTPS server for the DE440 ASCII chunks;
- the PDS Geosciences Node at Washington University for the GRGM1200B `.tab` and `.lbl` files;
- GFZ Potsdam's ICGEM service for the EIGEN-6C4 `.gfc` Earth gravity coefficients;
- the IERS Rapid Service / Prediction Centre for `finals2000A.all` and the IAU 2006 X/Y/s+XY/2 conventions tables.

Both servers are stable, public, and well-maintained, but URLs do move over time. The **editable URL field** on each card lets you paste a different location if a download fails — for instance, if NASA mirrors the file under a different path after a server reorganisation, or if your organisation hosts an internal mirror behind a firewall. The wizard does not persist URL overrides: they apply only to the current dialog session. Once you confirm a new URL works, please share it with the SpOdy maintainers so the next release ships with the corrected default.

3.5 Conversion to the internal format

Two derived assets need a conversion step. The wizard runs both automatically, sharing the **Conversion (auto)** status line at the bottom of the dialog.

3.5.1 `de440.spody` (planetary ephemeris)

Once the wizard sees that every required *raw* DE440 file is present and the derived `de440.spody` is either missing or older than the most recent raw chunk, it triggers the conversion automatically. The status line cycles through three states:

1. *waiting on raw DE440 (…)* — while downloads are still in flight;
2. *converting… 01950* — the engine is reading the ASCII chunks (the chunk IDs are listed live);
3. *de440.spody ready (98.4 MB)* — success.

The conversion itself takes a few seconds for the *Modern era* profile and a few tens of seconds for the *Full pack*. It uses the `spody.exe convert ephemeris` subcommand under the hood; see chapter 12 if you want to run the same conversion manually from a shell.

3.5.2 `eigen-6c4.tab` (Earth harmonics)

When the EIGEN-6C4 `.gfc` is downloaded, the wizard auto-runs the ICGEM→tab converter the same way:

1. *waiting on raw EIGEN-6C4 (…)* — while the download is in flight;
2. *converting harmonics… n = 215 / 2190* — progress ticks per 100 degrees of the spherical-harmonic series;
3. *eigen-6c4.tab ready (252 MB)* — success.

Because the raw `.gfc` is 178 MB and the converter walks the full 2190-degree series row by row, this conversion takes a couple of minutes on a typical laptop. It uses `spody.exe convert harmonics_icgem` under the hood (chapter 12).

The two conversions run sequentially when both DE440 and EIGEN-6C4 are downloaded at the same time; the status line carries the currently-active conversion's progress.

3.6 Daily-table freshness checks (EOP + space weather)

Two of the wizard's assets are living tables that their upstreams keep regenerating:

- `finals2000A.all` is rebuilt by IERS every Thursday. Old copies still work for past dates, but the *predicted* portion (covering the next ~365 days from the file's vintage) drifts; for runs whose epoch is

more than a few weeks past your local copy's vintage, the predicted UT1-UTC and polar-motion values are no longer the best estimates IERS can offer.

- `SW-All.csv` (the CelesTrak space-weather table feeding the drag density model, *Space weather* card) is regenerated **daily**: yesterday's F10.7 / Ap observations only appear in a copy downloaded today, and the ~45-day prediction tail ages just as fast. A stale copy makes the engine either refuse a near-present drag run (window past the predicted horizon) or silently use older predictions where observations now exist.

On every launch SpOdy issues one lightweight HTTP HEAD request per table (only for the ones already on disk) and compares the server's `Last-Modified` header against the local file's mtime (and the `Content-Length` against the local file's size). If the server's copy is newer, a non-blocking pop-up appears:

> *A newer finals2000A.all / SW-All.csv is available on the server. Download the latest now?*

Clicking **Yes** opens the wizard and triggers that card's download. The freshness check itself is silent on success and on transient network failure (no connectivity at launch is not an error). The GUI does not refresh the in-memory rotation provider mid-session either — it stat()s the file on every rotation call, so a fresh download from the open wizard takes effect on the next 3D plot redraw without restarting the app. The engine reads the space-weather file at run start, so a refreshed download is picked up by the next run automatically.

3.7 When everything is green

A complete bundle has every card showing a green `✓` and a status line reading *de440.spody ready* with a size in megabytes. At that point you can close the wizard with the **Close** button and start using the application: opening one of the example TOML files, or building a new scenario from scratch in the Run tab.

3.8 Re-opening the wizard later

The wizard remains available at any time through **Settings › Setup wizard…** in the menu bar. The two most common reasons to reopen it later are:

- **Widening coverage**: you started with *Modern era* but now want to run an Apollo-era scenario, so you flip to *Full pack* and click **Download all missing** to fetch the additional historical chunks.
- **Re-verifying after a copy**: you moved the bundle to a new machine and want a quick visual confirmation that nothing got lost along the way. The wizard's status icons answer that instantly.

You can also open the wizard purely to **change the data folder** through the **Change…** button at the top. SpOdy persists the override in the registry and uses it from then on; the bundle's `data/` folder is no longer touched. This is useful if you keep multiple SpOdy bundles around but want them to share a single copy of the (potentially hundreds of megabytes of) downloaded data.

4. The main window

Once the wizard has done its job you can take stock of the application proper. This chapter is a tour of the main window: the two top-level modes (**Run** and **Analysis**), the menus, the status bar, and how the pieces work together over the course of a typical session. Detailed coverage of the form, the schema, the plot catalogue, and the diff workflow waits for the chapters that follow.

4.1 Layout overview

The window divides into five regions:

1. The **menu bar** along the top, with four menus: **File**, **Run**, **Settings**, **Help**.
2. The **global top bar** immediately below the menus, hosting the shared **Working dir** field and a **Browse…** button. Both the Run tab's TOML picker and the Analysis tab's bin tree read from this one path, so picking a folder here drives every downstream view.
3. The **mode tab strip** below the top bar, with three tabs: **Run**, **Analysis**, **Re-run**. The window switches between completely different layouts depending on which tab is active; the menus and the top bar stay shared.
4. The **active workspace**, which fills the bulk of the window and changes shape with the mode (see sections 4.2 and 4.3).
5. The **status bar** along the bottom, with the path of the currently loaded TOML on the left and the run state (`idle`, `running 12.3s`, `OK (47.1s)`, `exit 1 (3.2s)`) on the right.

The full working-dir path may exceed the field's width; hover the field to see it in the tooltip.

The window remembers its last size and position across launches through Qt's standard mechanism, persisted alongside the other settings in the registry.

4.2 The Run tab

The Run tab is where you describe a scenario, validate it, and launch the engine. Above the workspace sits a **TOML picker row**; the workspace itself is a horizontal splitter you can drag to redistribute width:

Top row (full Run-tab width)
TOML combo + **Load TOML…** + **Save** + **Save As…**

Pane (left)	Pane (right)
TOML form (chapter 5)	Terminal view streaming spody's output
Live TOML preview below	Engine status (idle / running)

The **TOML combo** lists every `.toml` file under the shared working dir, scanned fully recursively. Snapshots inside `output/<ts>/` are included as load targets, identifiable by the `<ts>_` filename prefix; draft files produced by Save (see chapter 5) carry a trailing `(draft)` tag. Entries display compactly as

`<parent>/<file>`; the full relative path lives in the item tooltip. Picking an entry loads it into the form (the form's standard unsaved-edits prompt runs first if needed).

- **Load TOML…** opens a file dialog — useful when the file lives outside the current working dir (loading it auto- adopts its scenario folder as the new working dir if so).
- **Save** writes the current form state to the TOML the form is pointing at. If that file lives next to output bins (a snapshot, or a source with associated runs), Save diverts to a `<stem>.wip.toml` sidecar instead; see chapter 5.
- **Save As…** always pops the file dialog; the WIP divert does NOT fire here (you've explicitly chosen the path).

On the left of the splitter, an automatically-generated form lets you fill in every field the TOML schema accepts — one widget per scalar key, collapsible sections for optional blocks (`[spacecraft.srp]`, `[events]`, `[batch]`, `[cr3bp]`), and a live TOML preview pane underneath showing exactly what will be written to disk on Save.

On the right, a read-only terminal view displays the merged stdout and stderr streams of `spody.exe` while it runs. Output appears line by line as the engine produces it, exactly as you would see it in a console; no ANSI colour codes are interpreted, because the engine does not produce any.

Three action buttons sit at the top of the form column itself (below the TOML picker row, above the section groups):

- **Validate**: write a temporary TOML next to the current one and invoke `spody.exe validate` synchronously, displaying the result as a coloured badge under the row (`✓ valid` in green, or `✗ <reason>` in red with the full error in the tooltip).
- **RUN** (green): launch `spody.exe propagate` or `spody.exe batch` (the form auto-detects which subcommand applies from the presence of a `[batch]` section). When the loaded TOML has unsaved edits, Save runs first — including the WIP divert when applicable. While a run is in flight the button is disabled.
- **Stop** (red): kill the engine process currently running — enabled only while one is. Same action as **Run > Stop** (Ctrl + .). A killed run terminates immediately; the run folder keeps whatever output records were already written (a truncated binary), and no notes stamping or WIP cleanup happens — those are reserved for runs that finish with exit 0.

Detailed coverage of the form and the schema appears in chapters 5 and 6.

4.3 The Analysis tab

The Analysis tab is where you inspect simulation outputs. Its workspace is a horizontal splitter with a richer left column:

Pane (left, vertical splitter)	Pane (right)
File tree (auto-scanned)	Animation bar (only when 3D plot)
Add external + **Refresh**	Stacked 2D / 3D canvas
Overlay selected button	Info label
(splitter)	
Plot tree (grouped by topic)	
Tile selected button	

The left column is split vertically: file selection on top, plot selection at the bottom, with a draggable bar between them. The right column is dedicated to the canvas, with the optional animation bar appearing only when a 3D plot is active.

The file tree scans the **shared working dir** (set in the global top bar, see section 4.1) for `.bin` outputs — fully recursive, so every bin under the root surfaces no matter how deep the run-folder layout. Build / VCS / venv folders (`__pycache__`, `.git`, `.venv`, `venv`, `build`, `dist`, `node_modules`) are pruned; `output/` is intentionally included. A small **Refresh** button next to **Add external** re-scans manually when you drop new bins in by hand.

Click a single file in the file tree to load it. The application detects its type from the 8-byte magic in the header (`SPDYOUT_` trajectory, `SPDYACC_` accelerations, `SPDYEVT_` / `SPDYEVTB` events) and rebuilds the plot tree with the catalogue applicable to that type. Click a leaf in the plot tree to render it immediately into the canvas. Re-click the same leaf to re-plot.

Detailed coverage of the analysis layout, including overlay and diff workflows, appears in chapters 8 through 11.

4.4 The menus

The menu bar is identical in both modes. Entries that do not apply to the current mode are still listed but are no-ops until you switch.

4.4.1 File

- **New** — reset the Run-tab form to a blank scenario.
- **Open…** — open a `.toml` file (also achievable via the Run tab's **Load TOML…** button or by picking an entry from the TOML combo).
- **Open Recent ›** — jump to one of the last eight files you have opened. **Clear list** at the bottom of the submenu empties the history.
- **Save** / **Save As…** — write the current form state to the current path, or to a chosen path.
- **Quit**.

4.4.2 Run

- **Validate**, **Propagate**, **Batch** — the same three engine subcommands available through the buttons on the form, exposed through keyboard shortcuts (Ctrl + T, Ctrl + R, Ctrl + B respectively).
- **Stop** (Ctrl + .) — kill the current engine run; same action as the red **Stop** button on the form. On Windows the process is killed outright (a console engine cannot receive the graceful-close request); on Linux/macOS it gets a SIGTERM with a two-second grace window first.

4.4.3 Settings

- **Paths…** — the persistent paths dialog: where to find `spody.exe`, the data dir, the Moon texture used by the 3D viewer.
- **Setup wizard…** — reopen the wizard (chapter 3) at any time, even when everything is already green.
- **About** — version and build info for both halves of the application.

4.4.4 Help

- **User manual** — opens this document.
- **Visit project page** — opens the SpOdy project landing page in your default browser, if a network is available.

4.5 The status bar

The status bar at the very bottom of the window shows two permanently visible items:

- on the **left**, the absolute path of the TOML the Run tab is currently editing (or the literal string `(unsaved)` if you have never saved). An asterisk after the path (`<path>*`) indicates unsaved edits in the form;
- on the **right**, the engine's current state:
- `idle` — no run in progress;
- `running 14s` — a run is in flight; the elapsed-time counter updates every second;
- `OK (47.1s)` (green) or `exit 1 (3.2s)` (red) — the result of the last completed run.

The status bar is purely informational; nothing in it is clickable.

4.6 A typical session

To put the layout in motion: a routine session of SpOdy looks like this.

1. Launch `spody-gui.exe`. The window opens with an empty form. Browse to the project folder you want to work in via the global top bar's **Browse…**, or use **File › Open…** (the working dir adopts the file's scenario folder automatically).
2. Pick a TOML from the Run-tab combo (it now lists every scenario under your working dir), or build a scenario from scratch by typing into the form.
3. Click **Validate** to check the schema. The badge under the button turns green; if it does not, fix the indicated field and try again.

4. Click **RUN**. The Run tab's right pane streams the engine's output for the duration of the propagation; the status bar ticks `running 12s`, `running 13s`, …
5. When the run finishes, the status bar shows `OK` in green. The Analysis tab's file tree (sharing the same working dir) has already been refreshed with the freshly produced `output/<ts>/<ts>_*.bin` files; the Run-tab combo has also picked up the new `<ts>_input.toml` snapshot.
6. Click a `.bin` file to load it; click a plot leaf to render it; use **Ctrl/Shift-click** for multi-selection if you want to overlay or tile several plots.

That is the loop. Everything else — batches, diffs, 3D scenes, frame conventions — is variations on it.

5. Building a simulation: the TOML form

The Run tab's left pane is a *form* over the TOML schema: one widget per scalar key, collapsible groups for optional sections, range checks at every keystroke, and a live preview of the canonical TOML the form will write to disk. This chapter walks through how to fill it in. The schema details — types, ranges, defaults, what each field means physically — live in chapter 6.

5.1 Anatomy of the form

The form is a vertical scroll of section groups, one per top-level TOML section. From top to bottom in the order spody expects them:

1. `[simulation]`
2. **Object** — either `[spacecraft]` *or* `[debris]` (mutually exclusive; the choice is made through a pair of radio buttons at the top of the group)
3. `[initial_state]`
4. `[force_model]`
5. `[ephemeris]`
6. `[integrator]`
7. `[output]`
8. `[events]` *(optional, enabled by checkbox)*
9. `[batch]` *(optional, enabled by checkbox)*

Each section header is the literal TOML name, so the mapping between the form and the file you produce is one to one. The form never invents fields and never silently drops fields it does not understand: anything it loads from a TOML file that it does not have a widget for is preserved verbatim through a *passthrough* mechanism and re-emitted on the next save.

5.2 Filling a field

Every field type follows the same pattern: a label on the left, a widget on the right.

- **Strings** (`name`, `frame`, `central_body`, …): a plain line edit, except for known enumerated values that use a combo box (e.g. `frame` accepts only `central_inertial` today).
- **Floats** (`mass_kg`, `et_start_s`, `rel_tol`, …): a line edit, no validator attached (so the text you type stays verbatim — no surprise normalisation of `1.0e-5` into `1e-05`). Range checks happen at every keystroke against the field's registered validator (see section 5.3).
- **Integers** (`harmonics_degree`, `thread_number`): a line edit with a `QIntValidator` enforcing the min/max range as you type.
- **Booleans** (`srp`): a checkbox.
- **Vector-of-three** (`position_km`, `velocity_kms`): three line edits side by side.
- **Lists of strings** (`third_bodies`): one checkbox per known value (`Sun`, `Mercury`, `Venus`, `Earth`, …).

- **Paths to user-chosen files** (`output.output_dir`, `batch.cases_source_file`, …): a line edit alongside a **Browse…** button that pops a file dialog and writes the chosen path back into the edit.
- **Paths to wizard-managed assets** (`force_model.harmonics_file`, `force_model.eop_file`, `force_model.iau2006_dir`, `force_model.space_weather_file`, `ephemeris.file`): a **dropdown** of files the Setup wizard has downloaded into the data dir, filtered by category and (for harmonics, EOP, IAU 2006) by `central_body`. A **Browse…** next to the dropdown adds an out-of-data-dir file as a one-off `(custom)` entry, so a TOML pointing at e.g. `external/spody-core/raw_data/GRGM1200B/...` still round-trips. The dropdown refreshes automatically when the wizard finishes a new download or when `central_body` changes. The Earth-only rows (`eop_file`, `iau2006_dir`, `space_weather_file`, `density_scale`, `density_scale_file`, `drag`) appear only when `central_body = "Earth"`; switching back to Moon hides them and drops them from the emitted TOML.
- **Density calibration (`force_model.density_scale` / `density_scale_file`)** — the drag-only pair from chapter 6: a plain float row for the constant factor (leave empty for the uncalibrated 1.0) and a path row for the fitted `mjd,k` node table. Both are dropped from the emitted TOML when `drag` is off or the central body is not Earth; the engine's XOR between the two keys is checked by **Validate**, not by the form. The file row carries a third button, **Calibrate…**: it asks for a full-state reference binary (`spody convert gps / glonass / oem`) and a fit window in hours, then runs `spody calibrate` (chapter 12) through the same runner as RUN — the button flips to a disabled *Calibrating…*, the per-window report streams into the Run-tab console (the red **Stop** button or **Run > Stop** aborts it), and on success the row is auto-filled with the emitted `k_nodes.csv` path (clearing the constant, honouring the XOR) and the form is marked modified so you decide whether to save. The bundled `examples/iss_drag_calibration/` scenario is the ready-made playground for the whole loop.
- **Epoch (`simulation.et_start_s`)** — a dual-cell row: the ET value on the left, a UTC ISO 8601 cell on the right, two arrow buttons between them. → converts ET to UTC, ← converts UTC to ET. Conversion is bit-identical to SPICE `str2et` / `et2utc` (see chapter 14 for the underlying algorithm). Only `et_start_s` is written to the TOML; the UTC cell is purely a typing aid.
- **Duration (`simulation.duration_s`)** — a line edit plus a unit combo (`s | min | h | days`). The combo selects only the *display* unit; the TOML always carries `simulation.duration_s` in seconds. Typing `3600` with the combo on `s` is equivalent to typing `1.0` with the combo on `h` — switching the combo reconverts the visible number so the underlying seconds-value stays invariant. On load the form auto-picks the largest unit whose factor does not exceed the loaded magnitude (a `86400.0` TOML value comes back as `1.0 days`, a `0.5` value stays as `0.5 s`). The `[output]` sampling interval (`output.interval_s`, shown only when `output.mode = "fixed"`) carries the identical combo, so a daily or weekly grid cadence can be typed in days rather than seconds.

Hovering the cursor over a label or its widget shows a **tooltip** with the field's one-line description; range-validated fields also include the allowed range in the tooltip.

5.3 Validation as you type

A field whose value falls outside the registered range is flagged **immediately** with a red border and a small warning glyph appended to its tooltip (`⚠ must be > 0`, `⚠ must be in [2, 1200]`). The form does not block invalid input — you can continue editing — but the visual cue makes the bad field obvious.

The range checks the form runs are local: they catch values that are physically impossible (negative masses, zero rel-tol) or violate a documented bound (harmonics degree above 2200, the absolute schema ceiling that accommodates the EIGEN-6C4 / EGM2008 Earth coefficient sets). Cross-field consistency (e.g.

`h_init_s` between `h_min_s` and `h_max_s`) is *not* checked by the form — that is the engine's job, and **Validate** is the right button to press once the per-field indicators look clean.

5.4 The XOR object switch

Two TOML sections describe the propagated object, and a propagation must use exactly one of them. The form models this with a pair of radio buttons at the top of the **Object** group:

- **Spacecraft** — the conventional case: a body with a known mass and (when SRP is enabled) a cross-sectional area. This is the `[spacecraft]` section in the emitted TOML.
- **Debris** — an inferred body where only the area-to-mass ratio matters, because the gravity-driven accelerations are mass- independent and SRP acceleration depends only on `A/m`. This is the `[debris]` section.

Switching the radio shows the matching widgets and hides the others. The form never emits both sections; whichever radio is up at **Generate** time is the one that ends up in the file.

The choice also affects which override targets are available in the `[batch.columns]` mapping table (chapter 7). For instance, `spacecraft.srp.area_m2` only appears as a target when the radio is on Spacecraft, and `debris.am_srp` only when it is on Debris. The drag parameters follow the same rule (`spacecraft.drag.area_m2` / `spacecraft.drag.Cd` vs `debris.am_drag` / `debris.Cd`).

5.5 Optional sub-blocks

Four sections are not always required, and the form gates them behind a checkbox:

- **`[spacecraft.srp]`** — the SRP cannonball sub-block of `[spacecraft]`. Enable the *Enable [spacecraft.srp]* checkbox to expose the sub-form; inside it a second pair of radios chooses between `area_m2` (with `A/m = area / mass`) and `am_srp` (the ratio specified directly).
- **`[spacecraft.drag]`** — the atmospheric-drag sub-block, identical in shape to SRP: an *Enable [spacecraft.drag]* checkbox, an `area_m2` / `am_drag` radio pair and the `Cd` coefficient. In Debris mode the equivalent is the *Enable drag (am_drag + Cd)* checkbox inside the debris box. Remember to also tick the `drag` toggle in `[force_model]` (visible for Earth) so the engine actually integrates the force.
- **`[events]`** — two independent opt-in sub-sections, each behind its own checkbox: *Enable eclipse detection* exposes the `eclipse_threshold` field, *Enable altitude crossings* exposes a body / altitude_km / action / refined table with Add / Remove buttons. The body combo per row tracks the model's valid bodies (central + checked third bodies in HF, the two primaries in CR3BP) and rebuilds automatically when any of those change. The two features are independent — CR3BP forbids eclipse but accepts altitude crossings.
- **`[batch]`** — the multi-case sweep block. Enable the *Enable [batch]* checkbox to expose the batch-specific form (name, output directory, thread count, cases CSV, column mapping table). Batch scenarios are covered in detail in chapter 7.

Disabling any of these checkboxes after filling in fields *hides* the widgets but does not erase the values: re-enable later and your typed numbers are still there. At emit time, however, a disabled block is omitted entirely — the form behaves as if you had never filled it. This is the safest behaviour for round-tripping files that contain sections you do not currently want to touch.

5.6 Conditional UI

A few smaller conditionals are wired into the form to keep the visible surface relevant:

- `output.interval_s` is only shown when `output.mode = "fixed"`. In step mode it is meaningless (the integrator decides when to emit a record), so the row hides itself and the field is stripped from the emitted TOML even if you typed a value previously.
- The `spacecraft.srp.area_m2` and `spacecraft.srp.am_srp` fields grey each other out depending on which SRP-parameter radio is selected. The parser rejects files that set both, so this prevents the most common XOR mistake before it happens.

5.7 Saving: the file gets written

There is one path for writing the TOML to disk: the **Save** / **Save As…** buttons in the Run tab's TOML picker row (or the matching **File › Save / Save As…** menu items and their Ctrl + S / Ctrl + Shift + S shortcuts). Both **Save** and the **RUN** button write through the same canonical emitter so the result diffs cleanly between runs.

The emitter is **schema-aware**: it knows the canonical order of sections and keys, formats floats with `repr()` precision, and emits inline tables for the entries inside `[batch.columns]` when they use delta mode. Two **Save** clicks on the same form state produce byte-identical files.

5.7.1 The WIP draft mechanism

When you click **Save** on a TOML whose folder already contains `.bin` output files — either a per-run snapshot inside `output/<ts>/` or a source TOML whose runs landed beside it — the form does NOT overwrite that file. Doing so would corrupt the on-disk record every existing run depends on (the snapshot documents what the engine actually ran; rewriting it makes the snapshot lie about its bins).

Instead, Save diverts to a `<stem>.wip.toml` sidecar next to the original file. A one-time popup announces the divert ("Saving as draft …"); subsequent Save clicks on the same draft overwrite it silently (the WIP IS the editing target). WIPs are tagged `(draft)` in the TOML combo so they're easy to spot in mixed folders.

If you genuinely want to overwrite the original (e.g. to update a template you've decided is no longer authoritative), use **Save As…**: that path always asks for the destination, never diverts, and writes wherever you point it.

A successful **RUN** launched from a WIP cleans up automatically:

1. The engine snapshots the WIP's content into the new run folder as `output/<new-ts>/<new-ts>_input.toml` (the canonical record of this run).
2. The GUI unlinks the WIP from disk (no longer needed).
3. The form is reloaded with that per-run snapshot — the only surviving on-disk copy of what actually ran (with the engine's final status line stamped into its notes). Runs launched from a regular (non-WIP) TOML leave the form on the source file instead.

Failed runs leave the WIP alone so you can fix and retry.

5.8 The live TOML preview

Below the form (the lower half of the splitter visible in the Run tab's left pane) lives a read-only TOML preview. It mirrors what **Generate** would write to disk, refreshed on every keystroke.

Practically, it gives you three things:

1. a sanity check that the widgets you have just touched produce the TOML you expect — especially useful when learning the schema or when you want to see how the form serialises an optional sub-block;
2. a copy-friendly view of the canonical output, so you can paste the result into another tool, a chat message, or a bug report without having to round-trip through disk;
3. a feedback loop for the **passthrough** mechanism: any section the form does not directly render but found in the loaded file appears at the bottom of the preview, confirming it is being carried through to the next save.

If you ever type something the emitter cannot serialise (very rare — only `ValueError` from an internal conversion would trigger it) the preview shows a single-line `# (form has invalid values: …)` placeholder until the next valid edit.

5.9 Switching the dynamics model

The `simulation.dynamics_model` combo picks between `high_fidelity` (default; the full force-model propagator the rest of this manual is built around) and `cr3bp` (the Circular Restricted 3-Body Problem). Switching the combo reflows the form:

- `high_fidelity` shows the **Object** (spacecraft / debris), **force_model** and **ephemeris** sections.
- `cr3bp` hides those three groups and reveals a single `[cr3bp]` group with `primary_1` and `primary_2` combos (today's only valid pair is Earth + Moon). HF-only output toggles (the `accelerations_file` checkbox and the eclipse-event entry) are greyed out with a tooltip explaining why.

The frame combo in `[initial_state]` also filters per model: `central_inertial` for HF, `synodic_rotating` for CR3BP. The underlying CR3BP schema details — barycenter offsets, omega derivation, impact-event auto-wiring — live in chapter 6.

5.10 Importing a CR3BP state (From CR3BP…)

Next to the frame combo, high-fidelity runs get a **From CR3BP…** button (hidden under the `cr3bp` model, where the state is already synodic). It opens a popup that turns a CR3BP synodic-rotating state — the form the JPL periodic-orbit catalog hands out — into the `central_inertial` Cartesian state the engine propagates, so you can seed an HF run from a halo / NRHO / Lyapunov catalog point without leaving the GUI. Typical use: picking several points along one CR3BP orbit and testing each in the full-force model.

The dialog fields, top to bottom:

- **state vector** — all six components in one line, separated by spaces and/or commas (paste a catalog row as-is).
- **units** — `dimensional [km, km/s]` or `nondimensional`.

- **L [km]** — the CR3BP characteristic length the state was built with (defaults to the curated pair separation, e.g. 384 400 km for Earth–Moon). Nondimensional input is scaled by it (velocities by L·ω with ω = √((μ1+μ2)/L³)); dimensional input is divided by it to recover the catalog state before the epoch's actual geometry is applied.
- **primaries** — one of the curated CR3BP pairs (chapter 6; today Earth–Moon).
- **coordinates centered on** — `barycenter` or either primary, matching however your source quotes the state.
- **epoch** — read-only: `simulation.et_start_s` from the form (with its UTC twin). Change the epoch in `[simulation]` and reopen to convert for a different date.
- **output** — read-only: the state lands in `force_model.central_body`-centered ICRF.

Convert shows the resulting `position_km` / `velocity_kms` (selectable text) plus a sanity line: distance from the central body and the *actual* primary–primary distance at the epoch versus the L you entered. **Insert into form** writes the vectors into the Cartesian block, snaps `frame = central_inertial` and `kind = cartesian`, and re-seeds the swap cache (below) so subsequent kind / frame toggles round-trip from the inserted values. The dialog remembers its fields within the session, so converting the next point along the orbit is paste → Convert → Insert.

The conversion uses the **instantaneous pulsating-frame transform** (chapter 10): the rotating frame is anchored to the *real* primary geometry read from the `[ephemeris]` file at `et_start_s` — actual separation, actual axes (x from primary 1 to primary 2, z along the orbital angular momentum), actual angular rate plus the radial pulsation — so both primaries land exactly on their DE440 positions. Keep in mind the physics: a CR3BP orbit is periodic only in the CR3BP, so the converted state is a *seed* — expect an NRHO-class orbit to stay on family for a few revolutions in the full model before departing.

Three guard rails: an empty `et_start_s` pops a warning before the dialog opens; a missing / unreadable `[ephemeris]` file refuses the conversion (the real geometry comes from it); and a `central_body` that is not one of the selected primaries refuses too, because the output would be centered on a body the engine is not propagating around.

5.11 Switching the initial-state flavour

The `[initial_state]` section carries a `kind` combo above the field block with two choices:

- `cartesian` (the default) shows the legacy `position_km` + `velocity_kms` vec3 widgets.
- `keplerian` shows the six classical orbital elements (`semi_major_axis_km`, `eccentricity`, `inclination_deg`, `raan_deg`, `arg_periapsis_deg`, `anomaly_deg` with an `anomaly_type = "true" | "mean"` combo), plus a `reference_body` combo that picks **which** inertial frame the elements live in. Under HF the only option is `central` (= the central body); under CR3BP the choices become `primary_1` and `primary_2` (the dialog defaults to neither, so the user picks explicitly which primary anchors the orbit).

Flipping the combo runs a **best-effort live conversion in either direction** so the values you just typed are not thrown away. The converter pulls the right μ (central body's GM for HF, primary's GM for CR3BP) plus the synodic-frame geometry (omega + barycenter offsets) and chains them through the `spopy.kepler` / `spopy.cr3bp` helpers. Empty / unparseable fields make the conversion silently bail (the destination block stays at whatever it was holding); a conversion that succeeds fills every field of the new block in one shot.

5.11.1 Lossless swap cache

The six representations — `cartesian` / `keplerian` crossed with `central_inertial`, `central_body_fixed` and `orbit_plane` — are kept in a per-form cache that snapshots each finalised edit. (`orbit_plane` is offered only when the central body is the Moon, mirroring the engine validator; see chapter 6 for what the frame is.) Whenever you leave a cart / kep field (focus loss or Enter), the form treats the visible block as the ground truth and derives the other representations from it via spopy. Subsequent toggles of the `kind` or `frame` combos then write the cached values verbatim — no further conversion runs at toggle time, so back-and- forth flips of either combo round-trip the displayed numbers bit-for-bit (no compounding ULP drift the way per-toggle conversions would).

The cache is invalidated wholesale by changes to anything the underlying conversions depend on: `et_start_s`, `central_body`, `dynamics_model`, `anomaly_type`, `reference_body`, the ephemeris file. After such a change the very next toggle falls back to the old on-the-fly conversion (and seeds the cache from that result, so the toggle *after* it lands on the lossless path again).

The engine applies the same converter at parse time, so a hand-written TOML with `kind = "keplerian"` produces a bit- identical Cartesian state regardless of which side wrote it. The underlying schema lives in chapter 6, section *kind = "keplerian"*.

5.12 Validating with the engine

The **Validate** button at the top of the form runs the engine's own parser against the form's current state, *without writing your real file*. Internally the form:

1. produces the canonical TOML from `to_dict()` (the same path the live preview uses);
2. writes it to a temporary file next to your current saved file (or to the OS temp dir if you have not saved yet) so that relative paths inside the TOML (`harmonics_file`, `ephemeris .file`, `batch.cases_file`) resolve identically to what they would at run time;
3. calls `spody.exe validate <tempfile>` and waits up to 30 seconds for the result;
4. deletes the temp file and displays the verdict as a coloured badge to the right of the button:
    - `✓ valid` in green when the engine returns exit code 0;
    - `✗ <last line of stderr>` in red otherwise, with the full message in the tooltip.

Any edit you make after a green badge clears it back to neutral, so a stale green never misleads you into thinking an out-of-sync file passed validation.

6. TOML schema reference

This chapter is the field-by-field reference for the TOML input files SpOdy reads. Every section, every key, every accepted value is listed here in the order they appear in the form. For the high-level workflow consult chapter 5; for batch-specific keys also see chapter 7.

The conventions used in the tables below:

- **Type** is the TOML type the engine expects, with units where applicable.
- **Default** is what the engine assumes when the key is absent. An em dash (—) marks keys that are required.
- **Range** documents the allowed values; an unbounded numeric field is shown as `> 0` or similar.

6.1 `[simulation]`

Scenario name and time window. Required.

Key	Type	Default	Range	Description
`name`	string	—	–	Human-readable scenario name. Used as the prefix for batch case output names.
`dynamics_model`	string	`"high_fidelity"`	`"high_fidelity"`, `"cr3bp"`	Selects the propagator. `high_fidelity` (default) drives the full force-model integrator the bulk of this manual describes; `cr3bp` switches to the Circular Restricted 3-Body Problem in the synodic rotating frame (see *The `[cr3bp]` section* below).
`et_start_s`	float	— (HF only)	–	Start epoch as TDB seconds past the J2000 epoch (2000-01-01 12:00:00 TT). Negative values are valid for pre-J2000 epochs. Required in `high_fidelity` mode; ignored (and rejected if present) in `cr3bp` mode — the synodic frame is time-invariant.
`duration_s`	float	—	`> 0`	Propagation duration in seconds. Positive only (forward-time propagation).

The `et_start_s` value is the same scale the planetary ephemeris uses internally. The form provides a UTC ⇔ ET converter (an ISO 8601 UTC field next to `et_start_s` with two arrow buttons between them): typing a UTC instant and clicking ← fills the ET cell; clicking → does the inverse. The conversion is bit-identical to SPICE `str2et` — same `deltet` algorithm (`K`, `EB`, `M0`, `M1` from the NAIF LSK kernel) plus the hard-coded IERS Bulletin C leap-seconds table. The UTC cell itself is never written to the TOML; only

`et_start_s` is serialised, so the engine still sees a single canonical number. The DE440 wizard data covers 1950 – 2050 by default; choose the *Full pack* coverage profile in the wizard if you need to start outside that window.

`duration_s` and `output.interval_s` are likewise always SI seconds on disk, but the form ships a unit combo (`s | min | h | days`) next to each field so a multi-day debris run — or a weekly output cadence — does not need to be typed as `86400.0` or `604800.0`. The combo affects only the displayed number; emit and load round-trip the same float value. Auto-pick on load chooses the largest unit whose factor is ≤ the loaded magnitude.

6.1.1 Sections required by `dynamics_model`

The schema branches on `simulation.dynamics_model`:

`dynamics_model`	Required sections	Forbidden sections
`high_fidelity` (default)	`[simulation]`, exactly one of `[spacecraft]` / `[debris]`, `[initial_state]` with `frame = "central_inertial"`, `"central_body_fixed"` or `"orbit_plane"` (Moon only), `[force_model]`, `[ephemeris]`, `[integrator]`, `[output]`	`[cr3bp]`
`cr3bp`	`[simulation]`, `[cr3bp]`, `[initial_state]` with `frame = "synodic_rotating"`, `[integrator]`, `[output]`	`[spacecraft]`, `[debris]`, `[force_model]`, `[ephemeris]`, `[events]` with `eclipse_threshold`, `[output].accelerations_file`

The validator rejects mismatches up front (HF without `et_start_s`, CR3BP with a `[force_model]` block, …) so a misclassified TOML never silently runs the wrong dynamics.

6.2 The `[cr3bp]` section

Required when `dynamics_model = "cr3bp"`, forbidden otherwise.

Key	Type	Default	Range	Description
`primary_1`	string	—	`"Earth"`	Larger primary; sits at synodic x = `-(mu2 / mu_tot) * L`.
`primary_2`	string	—	`"Moon"`	Smaller primary; sits at synodic x = `+(mu1 / mu_tot) * L`.

The pair `(primary_1, primary_2)` selects a curated entry in the engine's `CR3BP_PAIRS` table that fixes `L` (the primary separation in km, from `spody_const.h` — today `EARTH_MOON_DISTANCE_KM = 384400`). The synodic angular velocity `omega = sqrt((mu1 + mu2) / L^3)` is derived at run start; both primaries' GM values come from the same central-body registry the HF propagator uses, so the constants stay consistent across dynamics models.

Because the pair names are all the file carries, the numbers a CR3BP run actually integrates would otherwise leave no trace in the scenario. They are recorded in two places:

- the form writes a **comment block** under `[cr3bp]` whenever it saves a TOML — `L`, the two GM values, the mass ratio `mu` and the synodic `omega` (with its period), re-derived from the pair on every save. It is a comment: the engine ignores it and re-resolves the values from its own tables, so editing it changes nothing and it can never go stale. The per-run snapshot the engine copies into `output/<timestamp>/` is a byte-for-byte copy of the input, so the block travels with the results;
- `spody propagate`, `spody batch` and `spody validate` print the same values in their opening block (manual ch. 12), which puts them in the run log of hand-written TOMLs too.

State is in **dimensional km / km/s** in the synodic rotating frame: x along the line from primary 1 to primary 2 (positive toward primary 2), z along the rotation axis, y completes the right-handed triad. The frame rotates at `omega` in the inertial frame; the barycenter is at the synodic origin. Impact events on both primaries are auto-wired with their standard radii (no `[events]` block required for that).

6.3 `[spacecraft]` *or* `[debris]`

Mutually exclusive object descriptions. Exactly one of the two sections must be present in a valid TOML.

6.3.1 `[spacecraft]`

The conventional case: a body with a known dry mass. Gravity is mass-independent, but SRP scales as `A/m`, so when SRP is enabled the engine derives `A/m` from the area and the mass.

Key	Type	Default	Range	Description
`mass_kg`	float	—	`> 0`	Dry mass in kilograms.

The optional `[spacecraft.srp]` and `[spacecraft.drag]` sub-tables are detailed below.

6.3.2 `[debris]`

The inferred-body case: only the area-to-mass ratio matters. Use this section when you do not have or do not care about a mass value, typically for parameter sweeps over a debris cloud.

Key	Type	Default	Range	Description
`am_srp`	float	—	`> 0`	Area-to-mass ratio in m²/kg, used by SRP.
`Cr`	float	`1.5`	`>= 0`	Reflectivity coefficient, only consulted when SRP is enabled. `1.0` = pure absorbing, `2.0` = pure mirror.
`am_drag`	float	— (optional)	`> 0`	Drag area-to-mass ratio in m²/kg. Optional pair with `Cd` (both or neither); may differ from `am_srp` — the drag cross-section is not the SRP cross-section. Required when `force_model.drag = true`.
`Cd`	float	— (optional)	`> 0`	Drag coefficient, only consulted when drag is enabled.

In Debris mode, every batch override target that mentions a mass or area (`spacecraft.mass_kg`, `spacecraft.srp.area_m2`, `spacecraft.drag.area_m2`) is unavailable; only the `debris.*` targets are accepted.

6.3.3 `[spacecraft.srp]` (optional)

The cannonball solar-radiation-pressure sub-block. Present only when `[spacecraft]` is the active object and the *Enable [spacecraft.srp]* checkbox is ticked.

Within this sub-block exactly one of `area_m2` and `am_srp` is allowed; setting both is a validation error.

Key	Type	Default	Range	Description
`area_m2`	float	—	`> 0`	Cross-sectional area in m². The engine derives `A/m = area_m2 / mass_kg`.
`am_srp`	float	—	`> 0`	Area-to-mass ratio in m²/kg, specified directly. Equivalent to `area_m2 / mass_kg`; use this form when you want sweep over `A/m` independently of `mass_kg`.
`Cr`	float	`1.5`	`>= 0`	Reflectivity coefficient, same convention as in `[debris]`.

6.3.4 `[spacecraft.drag]` (optional)

The cannonball atmospheric-drag sub-block, same shape as SRP. Present only when `[spacecraft]` is the active object and the *Enable [spacecraft.drag]* checkbox is ticked; required whenever `force_model.drag = true`.

Within this sub-block exactly one of `area_m2` and `am_drag` is allowed; setting both is a validation error.

Key	Type	Default	Range	Description
`area_m2`	float	—	`> 0`	Drag cross-section in m². The engine derives `A/m = area_m2 / mass_kg`.
`am_drag`	float	—	`> 0`	Drag area-to-mass ratio in m²/kg, specified directly.
`Cd`	float	—	`> 0`	Drag coefficient. `~2.2` is the classic value for compact satellites in free-molecular flow; flat/panelled bodies run higher (the ISS ballistic value published by NASA/JSC is `2.40`).

The density model is NRLMSISE-00 (the engine's own validated port), evaluated at the geodetic (WGS-84) sub-satellite point with the observed daily F10.7 of the previous day, the 81-day centered average, and the 7-element 3-hour Ap history (storm-time mode) from `space_weather_file`. Air co-rotation with the Earth is included in the relative velocity. The per-step drag acceleration is written to the accelerations stream (`SPDYACC_`) like every other force.

6.4 `[initial_state]`

Initial position and velocity vector of the propagated object. Required. Two input flavours are supported via the optional `kind` key — **Cartesian** (default, the only choice before this release) and **Keplerian** (six classical elements + a reference body). The engine converts Keplerian input into the Cartesian state the integrator consumes; the rest of the pipeline (and the snapshot TOML on disk) is identical for both.

Key	Type	Default	Range	Description
`frame`	string	—	`central_inertial`, `central_body_fixed` or `orbit_plane` (HF), `synodic_rotating` (CR3BP)	Reference frame. Model-exclusive: only the listed values are valid under each `dynamics_model`. `central_inertial` (HF) leaves the parsed `(position, velocity)` in the integrator's working basis; `central_body_fixed` (HF) interprets the values in the central body's body-fixed basis at `et_start_s` (Earth ITRS, Moon PA) and the engine rotates them to ICRF via the same `bf_rotation` callback the force-model uses on every step, before the run begins — the downstream integrator still sees a `central_inertial` state. `orbit_plane` (HF, Moon only) is Ely's OP frame, described below. `synodic_rotating` (CR3BP) places the elements in the reference primary's local inertial frame; the engine then rotates / translates them into the synodic frame at `t = 0`. The value of `frame` still has to match the model.
`kind`	string	`"cartesian"`	`"cartesian"`, `"keplerian"`	Which set of keys below is consumed. Omit for the legacy Cartesian path.

6.4.1 `kind = "cartesian"` (default)

Key	Type	Default	Description
`position_km`	array of 3 floats	—	`[x, y, z]` position in km in the chosen frame.
`velocity_kms`	array of 3 floats	—	`[vx, vy, vz]` velocity in km/s, same frame as `position_km`.

The initial state must be self-consistent: an `|r|` smaller than the central body's mean radius will trigger an IMPACT event at the first step. A `|v|` greater than the local escape velocity turns the simulation into a hyperbolic flyby, which the engine handles but is rarely what the user intended; double-check the magnitudes against your scenario.

6.4.2 `kind = "keplerian"`

Six classical orbital elements + a reference body and an anomaly selector. The convention for the reference inertial frame matches the standard aerospace one: `inc = 0` means the orbit lies in the reference frame's *xy* plane, `raan = 0` puts the ascending node on the `+x` axis, `arg_periapsis = 0` puts periapsis at the ascending node.

Key	Type	Range	Description
`reference_body`	string	`"central"`, `"primary_1"`, `"primary_2"`	Which body the elements reference. HF: defaults to `"central"`; the explicit value is also accepted, others are rejected. CR3BP: **required**, must be one of the primaries (no implicit default since both are physical).
`semi_major_axis_km`	float	`> 0`	Semi-major axis in km.
`eccentricity`	float	`[0, 1)`	Eccentricity. Hyperbolic / parabolic orbits are not supported via Keplerian input (use the Cartesian path with the equivalent state).
`inclination_deg`	float	`[0, 180]`	Inclination, degrees.
`raan_deg`	float	any	Right ascension of the ascending node, degrees.
`arg_periapsis_deg`	float	any	Argument of periapsis, degrees.
`anomaly_deg`	float	any	Anomaly value at `t = 0`, degrees.
`anomaly_type`	string	`"true"`, `"mean"`	What `anomaly_deg` represents. Mean is converted to true via Kepler's equation before the state synthesis.

CR3BP caveat. Keplerian elements describe an instantaneously osculating Kepler orbit around the chosen primary. The CR3BP system is *not* a Kepler problem — the trajectory will not stay closed; the second primary's gravity perturbs it from the first step onward. This is exactly the same situation as a satellite inserted into a Lunar orbit feeling Earth's pull, and is normally the point of running a CR3BP scenario. The Keplerian input form is just a convenient way to specify the *initial* state; once the integration starts it is identical to a Cartesian IC carrying the same `(r, v)`.

6.4.3 `frame = "orbit_plane"` — Ely's OP frame

Available under `high_fidelity` with `central_body = "Moon"`, for both `kind` values. It is the frame the lunar frozen-orbit literature states its elements in, and getting it wrong is the single most common way to "reproduce" a published frozen orbit and watch it impact instead.

Let `I_p` be the Moon's north pole (the `+Z` of the lunar PA basis) and `(r_E, v_E)` the Earth's state relative to the Moon at `et_start_s`. The axes are

- `z_op = (r_E × v_E) / |r_E × v_E|` — the normal to the Earth's *apparent* orbit about the Moon;
- `x_op = (I_p × z_op) / |I_p × z_op|`;
- `y_op = z_op × x_op`.

The frame is built **once**, from the instantaneous state at `et_start_s`, and is then treated as inertial — it only ever labels the initial condition. As with `central_body_fixed`, the engine rotates the parsed values into the integrator's `central_inertial` basis before the run starts; the emitted TOML keeps the frame name, so the user's intent survives save / load.

Why this matters. The lunar equator and the Earth's apparent orbit plane are about 6.8° apart. An inclination measured against one is therefore *not* the same orbit as the same number measured against the other: with `a`, `e`, `arg_periapsis` fixed, a given lunar-equator inclination maps to an OP inclination

anywhere in a ±6.8° band depending on where the node sits. Frozen elliptical lunar orbits are stated in the OP frame; entering them as `central_body_fixed` with the same numbers puts the orbit up to 13° away from the frozen family, which shows up as a secular growth of `e` that ends in an impact within months.

Because the frame's `x` axis is undefined when the perturber orbits in the central body's equatorial plane, the engine rejects a degenerate configuration rather than producing a silently arbitrary node. The Moon/Earth pair is nowhere near that limit.

6.5 `[force_model]`

Forces the propagator integrates against. Required.

Key	Type	Default	Range	Description
`central_body`	string	—	`Moon`, `Earth`	Central body of the propagation. Two bodies are supported in this release. The choice drives the gravity-model coefficient set, the body-fixed rotation provider (lunar PA libration angles from DE440 for Moon, IAU 2006/2010 + IERS EOP for Earth), and the list of valid `third_bodies`.
`harmonics_file`	string (path)	— (required unless `harmonics_degree = 0`)	–	Path to a spherical-harmonic gravity coefficients file (`gggrx_1200b_sha.tab` for GRGM1200B / Moon; `eigen-6c4.tab` for EIGEN-6C4 / Earth, produced by the wizard from the upstream `.gfc`). In the form this row is a **dropdown of harmonics files the wizard has downloaded**, filtered by `central_body`. A **Browse…** button next to the combo adds an out-of-data-dir file as a one-off `(custom)` entry, so legacy TOMLs pointing at e.g. `external/spody-core/raw_data/...` keep round-tripping. Relative paths resolve against the TOML's directory. Optional when `harmonics_degree = 0`, since nothing reads it.
`harmonics_degree`	int	—	`0` or `[2, 2200]`	Truncation degree of the harmonic gravity expansion. Higher = more accurate but more expensive. The effective upper bound is whatever the chosen `harmonics_file` declares (1200 for GRGM1200B, 2190 for EIGEN-6C4 / EGM2008); the `2200` cap is the absolute schema ceiling. `0` **switches the gravity field off entirely**: the central body stays a point mass, so together with `third_bodies` the run becomes an ephemeris-driven restricted N-body problem (see *Turning the gravity field off* below). Degree `1` is rejected — it would only move the origin to the centre of mass, which the central-body convention already assumes. See *Choosing a harmonics degree* below for guidance.
`harmonics_adaptive`	bool	`false`	–	Let the engine lower the degree per integrator step based on the satellite's distance, using `harmonics_degree` as the ceiling. Off by default; see *Letting*

Key	Type	Default	Range	Description
				the degree follow the orbit below. Requires `harmonics_degree >= 2` — at degree `0` there is no expansion to truncate.
`eop_file`	string (path)	— (Earth only)	–	Path to the IERS Earth-orientation file (`finals2000A.all` from the IERS Rapid Service). Required when `central_body = "Earth"`, omitted otherwise. The form exposes this row as a wizard-populated dropdown that only appears when Earth is selected.
`iau2006_dir`	string (path)	— (Earth only)	–	Path to the directory containing the IAU 2006 X / Y / s+XY/2 conventions tables (`tab5.2a.txt`, `tab5.2b.txt`, `tab5.2d.txt`). Required when `central_body = "Earth"`. Wizard-managed; same conditional form row as `eop_file`.
`third_bodies`	array of strings	`[]`	one of `Sun`, `Mercury`, `Venus`, `Earth`, `Moon`, `Mars`, `Jupiter`, `Saturn`, `Uranus`, `Neptune` (excluding the central body)	Perturbing bodies whose point-mass gravity is added at every step.
`srp`	bool	`false`	–	Enable cannonball SRP. When `true` a `[spacecraft.srp]` block must be present (in Spacecraft mode) or `am_srp` must be set in `[debris]` (in Debris mode). The bodies that can eclipse the Sun are the central body plus every entry of `third_bodies` (see *Which bodies cast a shadow* below); there is no separate key.
`drag`	bool	`false`	–	Enable atmospheric drag (cannonball, NRLMSISE-00 density in the storm-time 3-hour-Ap mode). Requires a central body with a registered atmosphere model (`Earth` in this

Key	Type	Default	Range	Description
				release), a `[spacecraft.drag]` block (or the `am_drag` / `Cd` pair in `[debris]`) and `space_weather_file`. The form shows this row only when the central body has an atmosphere.
`space_weather_file`	string (path)	— (drag only)	–	Path to the CelesTrak combined space-weather CSV (`SW-All.csv`: daily F10.7 + 3-hour Ap, 1957 to a ~45-day prediction tail plus monthly long-range rows). Required when `drag = true`. The run window must start at least 3 days after the table's first row (the Ap history looks back 57 h) and end inside the predicted horizon; the engine refuses the run otherwise and points at the update URL, `https://celestrak.org/SpaceData/SW-All.csv`. Wizard-managed (*Space weather* card) with the same daily startup freshness probe as `eop_file`.
`density_scale`	float	`1.0` (drag only)	`> 0`	Constant density calibration factor: the drag force uses `k × rho(NRLMSISE-00)`. Empirical thermosphere models carry a bias of 20–40% at 400–500 km around solar maximum (chapter 11, *Drag validation and ballistic calibration*); this key applies the calibrated correction without misdeclaring the physical `Cd`. Mutually exclusive with `density_scale_file`; requires `drag = true`. Batch-targetable as `force_model.density_scale`.
`density_scale_file`	string (path)	— (drag only)	–	Path to a time-varying calibration table: plain text, one `mjd,k` pair per line (UTC MJD, strictly ascending; `k > 0`; `#` starts a comment). The factor is linearly interpolated between nodes and **held at the end values outside the node span** (the engine prints a warning when the run window extends past the nodes). A single-node file is equivalent to the constant key. Mutually exclusive with `density_scale`; requires `drag = true`. `spody calibrate` (chapter 12) fits and writes this file automatically from a reference trajectory.

6.5.1 Which bodies cast a shadow

When `srp = true`, the Sun is dimmed by **the central body plus every body listed in `third_bodies`** (the Sun itself is skipped: it cannot occult itself). The rule is deliberately "whatever you chose to model also casts a shadow", so there is nothing extra to configure — and, symmetrically, a body whose gravity you left out will not shade the satellite either.

Two consequences worth knowing:

- **In cislunar space this matters a lot.** With `central_body = "Moon"` and `"Earth"` among the third bodies, the satellite now goes dark when the Earth passes in front of the Sun — which, seen from lunar orbit, is exactly a lunar eclipse. On a 12 h arc through the total lunar eclipse of 2025-03-14, an object with A/m = 0.02 m²/kg picks up about 17 m of position change from this term alone.
- **Listing a planet has a (tiny) physical price.** Venus among the third bodies means that during a transit of Venus the SRP dips by about 0.1%. That is real physics, not an artefact; the outer planets can never pass in front of the Sun as seen from an inner orbit, so they cost nothing but a handful of arithmetic per step.

If two bodies cover the Sun at the same time, their shadows on the solar disc are combined as a **union**, not a sum: the overlap is subtracted so it is not counted twice. The engine reports the combined value in the `eclipse_fraction` column of the accelerations file (chapter 8). The eclipse *event* (`events.eclipse_threshold`) is a separate mechanism that watches **one** occulting body at a time, so during a double eclipse the logged event fraction and the fraction used by the force are different numbers on purpose.

6.5.2 Choosing a harmonics degree

The right degree depends on the central body, the altitude, and the duration you want to integrate. Two starter tables follow, both based on empirical scaling against external references; the cost of the harmonic evaluation itself scales as O(N²).

Moon (GRGM1200B):

N	Use case
30 – 50	quick sanity propagation, low-fidelity orbit averaging
80	reasonable default for LRO-class missions; sub-km vs SPICE LRO POD over 6 days
150	sweet spot for low-lunar orbits; recovers ~95% of N=200's residual reduction at half the cost
200	high-fidelity floor; beyond ~200 the GRGM1200B coefficients become weakly observed and adding terms can slightly *increase* mean drift

Higher N values are accepted (up to the model's nominal 1200) but do not visibly improve accuracy for the example scenarios shipped with SpOdy.

Earth (EIGEN-6C4):

N	Use case
30 – 50	quick sanity propagation, sub-percent of N=70 cost
70	standard for GNSS-altitude propagation (GLONASS, GPS at ~20-25,000 km); matches IGS reprocessing conventions
120 – 200	LEO-altitude work; the EIGEN-6C4 high-degree terms become observable below ~1000 km
2190	full EIGEN-6C4 expansion; only relevant for surface gravity or very-low-LEO long-arc work

At GNSS altitudes the harmonics contribution is already tiny compared to the central two-body term, so degree 70 is comfortably above the noise floor of the rest of the force model (luni-solar third-body gravity, SRP) for most use cases.

6.5.3 Letting the degree follow the orbit

The tables above ask you to pick one degree for a whole run. That is the right question for a near-circular orbit, where the satellite always sits at roughly the same distance. On an eccentric one it is the wrong question, because the degree you need at closest approach and the degree you need far out are not the same number, and a single setting has to be the larger of the two.

How much larger is easy to see. The degree-n term of the potential carries a factor `(R_ref / r)^n`, so it dies off geometrically with distance. For a lunar orbit with `a = 6541 km` and `e = 0.6`, `spody maxhgdegree` puts the useful degree at 82 near periapsis and at 19 near apoapsis. Since the cost of one evaluation goes as `(N+1)²`, running the whole orbit at 82 does about eight times the work it needs to in the outer part of the revolution.

`harmonics_adaptive = true` removes that waste. Before every integrator step the engine picks the degree from

```
N(r) = ln(1/eps) / ln(r / R_ref)
```

capped at your `harmonics_degree`, where `r` is a conservative bound on the closest the step can get to the body. Only the ratio `r / R_ref` appears, so there is nothing to calibrate per body or per gravity model — the same rule serves the Moon, the Earth and any body added later.

It does not trade accuracy for speed. The threshold is set below the resolution of double precision, so the terms it drops cannot change the accumulated acceleration at all. Measured end to end:

run	fixed degree	adaptive	speed-up	difference
Lunar ELFO, 365 days, N = 80	32.1 s	14.0 s	2.3×	none, bit for bit
GPS, 7 days, N = 70	0.686 s	0.184 s	3.7×	none, bit for bit

“None” is literal: 453,447 and 672 output records, identical to the last bit, with the same number of accepted and rejected integrator steps. The step controller takes the very same sequence of steps, which is what tells you the per-step degree change is not perturbing it — the degree is held fixed across all stages of a step precisely so that it cannot.

The gain is largest on eccentric orbits and on runs whose degree is high relative to their altitude. A near-circular orbit whose degree is already close to what the altitude needs will see little or nothing, which is the correct outcome: there was no waste to remove.

Leaving the key out (or setting it to `false`) reproduces earlier runs bit for bit, so it is safe to add to an existing TOML without invalidating anything you have already computed.

6.5.4 Turning the gravity field off

`harmonics_degree = 0` removes the harmonic term altogether: the central body becomes a point mass and `harmonics_file` may be omitted. This is not a degenerate setting, it is a deliberate baseline, useful in three situations.

Isolating what the gravity field contributes. Run the same scenario at degree 0 and at your working degree, and the difference between the two trajectories *is* the harmonics contribution — no algebra, no separate acceleration breakdown.

A restricted N-body problem on real ephemerides. With `third_bodies` populated, degree 0 gives point-mass central body plus point-mass perturbers pulled from DE440. That is the classical restricted problem, except the perturbers move on their true ephemeris rather than the idealised circular orbits assumed by `dynamics_model = "cr3bp"` (chapter 6, `[simulation]`). It is the natural bridge between the two dynamics models: same restricted setup, real geometry.

A cheap first look. Degree 0 costs nothing per evaluation, so it is a fast way to check that an epoch, an initial state and a duration are sane before paying for a high-degree run.

On the bundled GPS G11 example (7 days, IGS precise orbits as reference), degree 0 leaves a 115 km residual against a 581 m one at degree 70 with third bodies — which is simply the size of the Earth's oblateness effect over a week at that altitude.

Degree `1` is rejected rather than accepted-and-ignored: it would only shift the origin to the centre of mass, which the central-body convention already assumes, so a TOML asking for it has almost certainly confused degree with something else.

6.6 `[ephemeris]`

Path to the planetary ephemeris binary. Required.

Key	Type	Default	Range	Description
`file`	string (path)	—	–	Path to a `.spody` ephemeris file. Use `de440.spody` produced by the setup wizard (chapter 3). In the form this row is a body-agnostic **dropdown of ephemerides the wizard has produced** (DE-series files cover every planet at once, so the list does not depend on `central_body`). A **Browse…** button next to the combo adds an out-of-data-dir file as a `(custom)` entry. Relative paths resolve against the TOML's directory.

A future release may accept `.bsp` SPICE kernels directly; today only the internal `.spody` format is supported.

6.7 `[integrator]`

Integration algorithm and tolerances. Required.

Key	Type	Default	Range	Description
`type`	string	—	`rkdp45`	Integration scheme. Only Dormand-Prince 5(4) is supported in this release.
`rel_tol`	float	—	`> 0`	Relative error tolerance per accepted step. `1e-11` is the recommended default for orbital regression work.
`h_init_s`	float	—	`> 0`	Initial step size in seconds. Normally somewhere between `h_min_s` and `h_max_s`.
`h_min_s`	float	—	`> 0`	Minimum allowed step size. The integrator gives up and reports failure if it would need to step smaller than this.
`h_max_s`	float	—	`> h_min_s`	Maximum allowed step size. Useful as a guard against the integrator picking very large steps in low-perturbation regions and missing events.

A typical low-lunar-orbit setup uses `rel_tol = 1e-11`, `h_init_s = 60`, `h_min_s = 1e-5`, `h_max_s = 2700`. The relatively large `h_max_s` (45 minutes) is harmless because the adaptive controller picks smaller steps where the dynamics need them.

6.8 `[output]`

Output stream configuration: which files to write, and at what cadence. Required.

Key	Type	Default	Range	Description
`mode`	string	—	`fixed` or `step`	Output cadence. `fixed` writes records on a uniform grid (`interval_s`); `step` writes one record per accepted RKDP step.
`interval_s`	float	—	`> 0`	Sampling interval in seconds when `mode = "fixed"`. Ignored otherwise.
`csv_file`	string (path)	none	–	CSV trajectory output. Empty/absent = no CSV produced.
`bin_file`	string (path)	none	–	Binary (`SPDYOUT_`) trajectory output. Recommended for analysis: the GUI's reader path uses this format.
`log_file`	string (path)	none	–	Path that the engine tees its stdout/stderr into.
`accelerations_file`	string (path)	none	–	Per-force acceleration breakdown (`SPDYACC_` format). Empty = no breakdown produced.
`events_log`	string (path)	none	–	Event records (`SPDYEVT_` format) for impacts and (if `[events]` is enabled) eclipses.

Output paths in the form are not edited directly; the `[output]` block exposes five **on/off checkboxes** (csv, bin, accelerations, events, log) plus a single `output_dir` picker. spody auto-derives every enabled stream's filename as `<output_dir>/<sim_name>_<subject>_<frame>.<ext>` (e.g. the state-vector binary is `<sim_name>_state_icrf.bin`, the accelerations binary is `<sim_name>_acc_icrf.bin`); the under-the-hood TOML still carries the five `<stream>_file` strings so the engine sees no schema change.

On every invocation the engine creates a **per-run folder** named `<output_dir>/<UTC-ISO8601>/` (compact format, e.g. `2026-06-09T120000Z`) and rewrites every enabled output path so it lives inside that folder. The TOML used to start the run is also copied into the run folder as `input.toml`, so a run is fully self-contained: zip the folder and you have the inputs + outputs together. The Analysis tab (chapter 8) groups files by these folders.

6.9 `[events]` (optional)

Opt-in event detection. Two independent sub-sections, each gated by its own form checkbox:

6.9.1 `eclipse_threshold` — eclipse detection (HF only)

Key	Type	Default	Range	Description
`eclipse_threshold`	float	—	`[0, 1]`	Sunlight-fraction crossing that fires an eclipse event. `0` = enter umbra (start of total eclipse); `1` = full sunlight (end of any eclipse); `0.5` = penumbra midpoint.

Rejected under `dynamics_model = "cr3bp"` (no Sun in the model).

6.9.2 `[[events.altitude_crossing]]` — altitude triggers

Array of tables; one entry per altitude band the user wants logged. Fires on *every* sign change of `|r_sat - r_body| - body_radius - altitude_km`, so the same band logs both the ascending and the descending crossing of one orbit. Direction is recoverable from the radial velocity at trigger (`v_trigger · r̂_trigger`).

Key	Type	Default	Range	Description
`body`	string	—	–	Body to measure altitude from. HF: the central body or any entry in `force_model.third_bodies`. CR3BP: one of `cr3bp.primary_1` / `cr3bp.primary_2`.
`altitude_km`	float	—	`> 0`	Target altitude above the body's mean radius (km). Use the always-on IMPACT detector for surface contact (`altitude_km = 0` is rejected).
`action`	string	`"log"`	`"log"`, `"stop"`, `"log_and_stop"`	Behaviour on trigger. `log` keeps the propagation going (the natural choice for monitoring several bands); `stop` ends the run silently; `log_and_stop` does both.
`refined`	bool	`true`	—	When `true` (default), Brent + dense-output localises the trigger sub-microsecond inside the accepted step. When `false`, the trigger lands at the end of the accepted step (step-size precision). Refinement is essentially free in steady state — Brent only runs at the actual sign-change step — but the toggle is exposed for catalog-style runs with many bands.

Example:

```toml
[[events.altitude_crossing]]
body        = "Earth"
altitude_km = 500
action      = "log"

[[events.altitude_crossing]]
body        = "Earth"
altitude_km = 1000

[[events.altitude_crossing]]
body        = "Moon"
altitude_km = 100
action      = "log_and_stop"
```

6.9.3 Always-on IMPACT detection

Any trajectory that crosses a central-body or third-body surface (mean radius) produces an IMPACT record regardless of the `[events]` section. The section only controls the opt-in eclipse and altitude detection above.

6.9.4 Life markers (`INITIAL_STATE` / `FINAL_STATE`)

Also unconditional, and also not configurable: whenever an events log is written, every propagated object contributes one `INITIAL_STATE` record at `t = 0` and one `FINAL_STATE` record at whatever ended its run — the planned duration, an impact, or a stop-class altitude crossing. They are not triggers: no predicate is evaluated and nothing can suppress them.

They exist because a log built only from triggers is not a complete record of what was propagated. An object that stays between two altitude thresholds for the whole run crosses nothing, so it wrote nothing, and was indistinguishable from an object that never ran — including in its own denominator, which silently inflated every per-object percentage computed from the file. With the markers, the `case_idx` values present in a batch log are the complete list of propagated cases, so post-processing no longer needs `cases.csv` to know the batch size.

They also carry the object's starting altitude and the true end of its window, which is what lets the altitude-band analysis measure those two facts instead of inferring them from the direction of the first crossing and from the planned duration.

Field layout follows the altitude-crossing convention — `radius_km` is the body's radius, `distance_km` the body-centric distance, so `distance_km - radius_km` is the altitude. One marker is written per body whose distance needs no ephemeris query: the central body in high fidelity, both primaries in CR3BP. A `FINAL_STATE` that never arrives after an `INITIAL_STATE` means that case died mid-run (integrator or I/O failure); the Info tab reports those as *Objects never closed*.

Reading older files. Logs written before August 2026 carry no markers and are read exactly as before. New logs are two records per object per body longer, so an events `.bin` from before that date is not byte-comparable with a fresh one — trajectory and acceleration binaries are unaffected.

6.10 `[batch]` (optional)

Multi-case sweep section. Present only when the *Enable [batch]* checkbox is ticked. Covered in detail in chapter 7.

Key	Type	Default	Range	Description
`name`	string	—	–	Batch run name. Used as the prefix for per-case output names.
`output_dir`	string (path)	—	–	Existing directory under which the engine auto-creates a per-run folder named with a compact ISO 8601 UTC timestamp (e.g. `2026-06-09T120000Z/`). All per-case binaries, the aggregated events file, and a copy of the input TOML land in that folder, so each batch invocation is self-contained and discoverable. Replaces the older `batch/` subfolder convention.
`thread_number`	int	`1`	`[1, N_cpu]`	Worker thread count. `1` = sequential. Capped at the host's logical-CPU count by the form. Parallel execution requires the OpenMP-enabled engine build.
`cases_file`	string (path)	—	–	CSV file describing the parameter sweep. First non-comment line is the header.
`columns`	inline table	empty	–	Mapping from CSV columns to target paths (see chapter 7). Programmatic; the form's column-mapping table is the friendly editor.

7. Batch propagations

A *batch* is a parameter sweep: the same scenario propagated many times with one or more fields varied case by case. SpOdy describes the sweep through a CSV file (one row per case, columns matching the fields to vary) and a mapping table that tells the engine which CSV column overrides which TOML field. This chapter walks through how to set one up.

7.1 Why batches

The typical reasons to run a batch instead of repeated single-case runs are:

- **Sensitivity analyses** — sweep over `harmonics_degree` to see how much accuracy you actually need; sweep `Cr` to understand the SRP coefficient's contribution to long-term drift; sweep `mass_kg` to characterise the response of a chosen propellant tank load.
- **Monte-Carlo simulation** — randomly perturb the initial state across thousands of cases and compute statistics on the final position.
- **Debris-cloud propagation** — one case per fragment, each with its own `A/m` ratio drawn from a debris-population distribution.

The engine threads cases across CPU cores when the OpenMP-enabled build is used and `batch.thread_number > 1` (see section 7.5), giving near-linear speed-up for the typical case sizes.

7.2 The CSV file

A SpOdy batch cases file is a plain CSV. The conventions are:

- Lines starting with `#` are comments and are skipped.
- The **first non-comment, non-blank line is the header**: a comma-separated list of column names. Whitespace around each name is trimmed.
- One special header value is reserved: `id`. When present, the column's value is used as the per-case name (for the output file basenames). When absent, the engine auto-generates names like `case_0001`, `case_0002`, …
- Every other column is a **parameter override** whose name is arbitrary; the mapping to a TOML field is decided by the `[batch.columns]` section, not by the column name itself.
- All non-`id` cells are parsed as floats.

An example three-case CSV for a mass + Cr sweep:

```
id, mass_kg, Cr
A,  1916.0,  1.3
B,  1916.0,  1.5
C,  2500.0,  1.3
```

Three cases (`A`, `B`, `C`) with two parameters varied.

7.3 The column mapping table

The form's `[batch]` section embeds a table that maps every non-`id` CSV column to a *target*: a dotted path inside the TOML schema. When the engine runs case *i*, it reads row *i* of the CSV, applies each cell to the target the column is mapped to, and propagates the resulting scenario.

The table updates live whenever you change the **cases_file** field above it:

- you can browse to a CSV with the **Browse…** button, which also re-reads the column list immediately;
- or type the path manually and press **Re-read columns**;
- or load a TOML that already has a `[batch.columns]` block: the table populates from the file.

For each row of the table you see three cells:

CSV column	Target dropdown	Mode dropdown

The **Target** dropdown contains all valid override paths for the currently-selected object mode (Spacecraft or Debris), pre-filled with a heuristic when the column name matches the last segment of a known target (`mass_kg` ⇒ `spacecraft.mass_kg`, `am_srp` ⇒ `debris.am_srp`). The **Mode** dropdown picks between the two override semantics described in section 7.4 below. A column you leave on `(unassigned)` is silently dropped at **Generate** time — the engine validates that the table covers every non-`id` column at run time, so unassigned columns trigger a clean error.

7.3.1 Available targets

The list of override paths is the same as the path-style API the engine accepts for `[batch.columns]`:

- `simulation.et_start_s`, `simulation.duration_s`
- `spacecraft.mass_kg`, `spacecraft.srp.area_m2`, `spacecraft.srp.Cr` (only in Spacecraft mode)
- `debris.am_srp`, `debris.Cr` (only in Debris mode)
- `initial_state.position_km[0]`, `[1]`, `[2]`
- `initial_state.velocity_kms[0]`, `[1]`, `[2]`
- `force_model.srp`
- `integrator.rel_tol`, `integrator.h_init_s`, `integrator.h_min_s`, `integrator.h_max_s`
- `output.interval_s`

Component access into the vector-of-three fields (`position_km[0]`) lets you sweep along a single axis without disturbing the others.

7.4 Override vs delta

The **Mode** dropdown picks between two semantics for the CSV value:

- `override` (default) — the value in the CSV cell *replaces* the field's TOML value. If your TOML has `mass_kg = 1916.0` and a row's `mass_kg` column reads `2500.0`, the case runs with `mass_kg = 2500.0`.
- `delta` — the cell value is *added* to the field's TOML value. The same row with `mass_kg` in delta mode would run with `mass_kg = 1916.0 + 2500.0 = 4416.0`. Useful for Monte-Carlo perturbations around a nominal case, where the CSV column is literally a delta drawn from a distribution.

In delta mode the emitted TOML uses an *inline table* for the column descriptor:

```
[batch.columns]
mass_kg = "spacecraft.mass_kg"                          # override
mass_dm = { target = "spacecraft.mass_kg", mode = "delta" }
```

Mixing both modes across columns is supported. Mixing across *cases* (i.e. the same column behaving as delta in one row and override in the next) is not.

7.5 Rotating-frame batch input (RIC / LVLH)

The propagation engine (`spody.exe`) **only accepts ICRF central- inertial state**: every cell that ends up overriding `initial_state.position_km[i]` or `velocity_kms[i]` must be in that frame. There is no `frame =` knob at the cases-CSV level on the C side.

The GUI bridges this gap so a user with state measurements in a **rotating frame attached to a reference satellite** does not have to rotate them by hand. Two rotating frames are accepted today (both anchored to the reference orbit declared in `[initial_state]`):

- **RIC** — the typical *debris cloud seen from the chaser* use case: `x = r_hat` (radial), `y = (h x r)_hat` (in-plane, forward of the orbit normal), `z = h_hat` (cross-track). Pairs naturally with sensor-frame snapshots from a chaser tracking a target.
- **LVLH** — the NASA / Goddard convention used by, e.g., the NASA breakup model: `z = -r_hat` (nadir), `y = -h_hat` (anti orbit normal), `x = y x z` (horizontal, ≈ `+v` for a circular orbit). Picking this is the right move when your source CSV is the binary dump from a debris-evolution tool using the LVLH convention.

The form has a single cases-CSV path field (always showing the user-picked source) plus a `cases_frame` combo (`icrf` | `ric` | `lvlh`), and emits three coordinated keys inside `[batch]`:

TOML key	Meaning
`cases_source_file`	The path the user picked — the file the form shows.
`cases_frame`	`"ric"` or `"lvlh"` for a rotating source; otherwise the propagator's own frame — `"icrf"` under `high_fidelity`, `"synodic"` under `cr3bp` — meaning the components are used as they are. The combo relabels that first entry when you switch model.
`cases_file`	The file `spody.exe` actually reads. See the rule below.

The contract between the three keys is:

- `cases_frame` = **the propagator's own frame** (`"icrf"` / `"synodic"`): `cases_file` equals `cases_source_file` byte for byte. The picked CSV goes straight to `spody.exe`. A TOML written by an older SpOdy that says `"icrf"` on a CR3BP run still loads: both spellings mean "use the components as they are".
- `cases_frame = "ric"` **or** `"lvlh"`: at **Generate-TOML** the GUI
- reads the column-mapping table to find which CSV columns target `initial_state.position_km[i]` / `initial_state.velocity_kms[i]`;

- computes the rotation `R = ric_basis(r_ref, v_ref)` or `R = lvlh_basis(r_ref, v_ref)` according to the combo — the reference orbit `(r_ref, v_ref)` comes straight from `[initial_state]`, so no extra input is required;
- rotates each row's position triplet and velocity triplet by `R @ vec` (**pure change of basis** — the reference vector is *not* added to the result);
- writes the rotated copy to `<stem>_wrt_<target>.csv` next to the source, where `<target>` is the frame the propagator integrates in (see *Which frame the offsets land in* below);
- emits `cases_file = "<stem>_wrt_<target>.csv"` so `spody.exe` reads the rotated copy. `cases_source_file` and `cases_frame` round out the triple so the form restores its rotating-frame state on the next Load without the user re-picking the source.

The rotated copy is a **derived artifact**: it is rebuilt at every Generate *and* again before every Run, so editing the source CSV outside SpOdy and pressing Run cannot launch against the previous rotation. If the source has gone missing the launch is refused rather than run with stale rows. The only time the rebuild is skipped is when you answer *Discard* at the unsaved-changes prompt — the run then uses the TOML already on disk, and the rotated copy that goes with it.

Treat the `_wrt_` copy as regenerable output: keep the **source** CSV under version control, not the rotated one, and copying a scenario elsewhere only needs the source.

`spody.exe` ignores `cases_frame` and `cases_source_file` today — its TOML parser only reads the keys it knows about. The two extras are also a placeholder for a future engine-side rotating-frame handler that would take the source CSV directly.

7.5.1 Which frame the offsets land in

The engine adds batch offsets to `initial_state.position_km` / `velocity_kms` **after** the initial state has been collapsed to the representation the propagator integrates in. That representation is not the same for the two dynamics models, so neither is the rotation target:

`dynamics_model`	reference orbit used for the basis	file written
`high_fidelity`	central-body inertial (ICRF)	`<stem>_wrt_icrf.csv`
`cr3bp`	synodic rotating, relative to the primary	`<stem>_wrt_synodic.csv`

Rotating into the other one would add components of one frame to a vector expressed in another — the offsets would still *look* plausible, they would simply point somewhere else.

Two details specific to CR3BP:

The basis is built about the primary, not the barycentre. The `[initial_state]` block under CR3BP is barycentre-centred, and a local-vertical axis taken there points along the primary-primary line. For a close orbit that is nearly perpendicular to the real local vertical — on a lunar ELFO the two differ by almost 90°. The GUI therefore removes the primary's offset first, picking the **nearer** primary and naming it in the status line under the combo (*"Local axes are built about Moon."*), so the choice is visible and checkable. Its velocity needs no correction: in the rotating frame both primaries are stationary, so the object's synodic velocity already is its velocity relative to them.

Keplerian input works too. It goes through the same chain the engine uses — Kepler elements to Cartesian on the reference primary's `mu`, then the synodic transform — so the basis is built on the state the engine will actually start from rather than on a parallel derivation of it.

A consequence worth stating for ΔV work: a velocity offset expressed in synodic components is the *same physical* ΔV it would be in inertial components, because the rotating and inertial velocities differ by `omega x r`, which depends only on position and is unchanged by an instantaneous burn. Only the direction of the along-track axis differs slightly (a fraction of a degree for a close orbit). The magnitude is preserved exactly.

7.5.2 Live rotated preview

When the combo is set to a rotating frame, a second preview table appears under the standard cases-CSV preview, showing the first 10 rows *after* rotation. Its header reads **Rotated preview (post RIC -> ICRF)**, **(post LVLH -> SYNODIC)** and so on — naming both the source convention and the target — so the displayed convention is never ambiguous. It refreshes automatically when you change the source path, the frame combo, or re-read the columns — that automatic refresh only redraws the table, it never touches the filesystem.

A **Rotate + refresh** button on top of it does both halves: it writes the rotated copy to disk *and* recomputes the table, so you can open the rotated file and check it before committing to a run. Use it after editing `[initial_state]` or the column-mapping table. It is never a prerequisite — running rotates again on its own — it just lets you look first.

7.5.3 Pairing with delta mode

Because the GUI rotates *components without translation*, the typical RIC workflow pairs each rotated state column with `mode = "delta"` (see "Override vs delta" above). With the rotated cell playing the role of an offset and `[initial_state]` playing the role of the base, the engine computes per case

```
final[i] = initial_state.<vec>[i] + rotated_cell
```

which is the absolute ICRF state to integrate. Override mode is also accepted (in which case the rotated cell *replaces* the base value — useful only if the CSV's row magnitudes are already on the order of the reference state, which is unusual for sensor- frame measurements).

7.5.4 Offsets are always in the propagator's own frame

`[initial_state]` may be written in any frame the schema allows and as either Cartesian or Keplerian. Before the first case is applied, `spody batch` collapses it to the one representation the propagator consumes, and **every batch offset is interpreted in that same representation**, regardless of how the base state was expressed:

- under `high_fidelity`, a central-inertial (ICRF) Cartesian `(position, velocity)` pair — whether the block was written as `central_inertial`, `central_body_fixed` or `orbit_plane`, Cartesian or Keplerian;
- under `cr3bp`, the synodic rotating pair.

That is what makes the RIC / LVLH pipeline above correct: the GUI rotates the cases CSV into the *same* frame the base is resolved to, so the addition happens between two quantities in one frame. It is also why the rotation target follows `dynamics_model` (see *Which frame the offsets land in*).

Nothing is required of you, and the TOML keeps the frame you wrote — it is worth knowing only if you hand-write a batch file, or if you are comparing against results produced before this rule existed. Older SpOdy applied each case *before* rotating the state into the working frame, so a batch whose initial state was `central_body_fixed` interpreted ICRF offsets as body-fixed components and quietly displaced every case.

7.5.5 Sensor-frame snapshot convention

The GUI uses a **sensor-frame snapshot** convention for the velocity: no `omega x r` term is added. The `dv` components are treated as plain vector projections onto the instantaneous rotating-frame axes (RIC or LVLH) — exactly what an onboard sensor (radar / lidar / optical relnav) or a debris-evolution tool would report when expressing fragments in a chaser-fixed frame. This is **not** the Hill / Clohessy-Wiltshire rotating-frame convention used in rendezvous literature; if you have CW residuals from another tool and want SpOdy to interpret them, transform them before passing the CSV in.

See `examples/debris_ric_demo/` for a runnable end-to-end RIC example. The LVLH path follows the same column-mapping rules; the only difference is the basis matrix `R` the GUI applies.

7.5.6 Metadata columns: `target = ""`

Real-world cases CSVs often carry bookkeeping columns that the propagator does not consume — e.g. the characteristic length of each fragment from a NASA breakup-model run, a debris ID for post-processing, or a classification tag. These columns must still appear in `[batch.columns]` (the validator errors if a CSV header has no matching entry, which is the typo guard you want) but they should not override any propagator field.

The schema reserves the **empty target string** as the metadata sentinel:

```
[batch.columns]
dv_x_kms  = { target = "initial_state.velocity_kms[0]", mode = "delta" }
dv_y_kms  = { target = "initial_state.velocity_kms[1]", mode = "delta" }
dv_z_kms  = { target = "initial_state.velocity_kms[2]", mode = "delta" }
L_char_m  = ""                                                  # metadata
```

A column declared with `target = ""` is type-checked by the parser (it must still be a valid float in every data row) but its values are never applied to any field. In the form's column-mapping table the same semantics surface as the **"(unassigned)"** target choice — pick it for any column you want to keep in the CSV without threading it into a propagator override. The inverse is also supported: loading a TOML with `col = ""` restores the **(unassigned)** selection in the form instead of silently reassigning by the name-based heuristic.

7.6 The data preview

Below the mapping table, a small read-only table previews the first ten data rows of the cases CSV verbatim, with the original column names (including `id`) as headers. It is purely informational, but two things make it worth glancing at:

- you can sanity-check that the column-to-target mapping is right by comparing each header against the value you see — `mass_kg` values around 1916 should belong to a mass column;

- on long CSVs the row counter `(preview: first 10 of N rows)` next to the preview tells you how many total cases the engine will see, useful for catching truncated files.

7.7 Threading

The `batch.thread_number` field caps automatically at the host's logical-CPU count (the form's integer validator enforces this as you type, with the cap shown next to the field). Setting more threads than cores never helps and often hurts (oversubscription), so the form refuses values above the cap rather than letting them through silently.

A few additional points:

- **Sequential execution** (`thread_number = 1`) works with any engine build. If you do not know whether your `spody.exe` was built with OpenMP, leave the field at `1` and the batch will still run correctly, just one case at a time.
- **Parallel execution** (`thread_number > 1`) requires an OpenMP- enabled engine. The standard SpOdy bundle ships with one such build. The engine rejects the run with a clean message if the build does not support parallel execution.
- **Determinism**: the engine processes cases in a fixed order regardless of thread count, so the per-case output is bit- identical across thread counts.

7.8 Output layout

When the batch runs, the engine creates a per-run subfolder named after the UTC instant the run started (ISO 8601 compact, e.g. `2026-06-05T195819Z`) inside `batch.output_dir`, snapshots the source TOML inside as `input.toml`, and writes one set of files per case alongside it:

```
<batch.output_dir>/<UTC-ISO8601>/
  input.toml                                  (snapshot of the source)
  <name>_<case_id>_state_icrf.csv             (if csv_file is set)
  <name>_<case_id>_state_icrf.bin             (if bin_file is set)
  <name>_<case_id>_acc_icrf.bin               (if accelerations_file is set)
  <name>_events.bin                           (if events_log is set; aggregated)
```

The `<name>` part comes from `batch.name`; the `<case_id>` part is the value of the `id` column or the auto-generated `case_NNNN` name. Each invocation goes in its own timestamp folder, so the results of any prior run are never overwritten silently — the listing under `<batch.output_dir>/` is the chronological history of every batch run executed from that scenario.

> *The events file is **aggregated**: a single binary covers every trigger across the whole batch. Each row carries an extra `case_idx` (int32) field at the front so post-processing can join with the cases CSV. The file's 8-byte magic is `SPDYEVTB` (versus `SPDYEVT_` for the per-run propagate output). The Python readers in `python/spody_io/events.py` auto-detect the format from the magic and return the matching numpy dtype.*

7.9 A complete worked example

The `examples/batch_demo/` scenario shipped with SpOdy is a realistic, runnable batch. Open `examples/batch_demo/input.toml` in the Run tab to see the form populated:

- a single `[simulation]` block;
- `[spacecraft]` with a nominal `mass_kg = 1916.0`;
- `[spacecraft.srp]` enabled with a nominal `Cr = 1.3` and `area_m2 = 15.0`;
- `[batch]` enabled with `cases_file = "cases.csv"`, `name = "mass_srp_sweep"`, and a column mapping that overrides `spacecraft.mass_kg` and `spacecraft.srp.Cr`.

The CSV at `examples/batch_demo/cases.csv` lists three cases (`A`, `B`, `C`) varying mass and reflectivity, exactly as in section 7.1. Press **RUN** to dispatch all three; the Run tab's terminal pane streams the case-by-case progress, and the resulting files land in `examples/batch_demo/output/<UTC-ISO8601>/`. Switch to the Analysis tab afterwards: the working directory has auto- pointed at `examples/batch_demo/output/`, the per-run folder appears as a group in the file tree, all three trajectories are inside it, and you can `Ctrl`-click them all and press **Overlay selected** on a single-line plot (such as **Radial distance |r|**) to compare the cases visually.

8. The analysis tab

The Analysis tab is where you inspect the binary outputs the engine produces. Its workspace is organised around two trees on the left (files and plots) and a **three-tab right pane** — one tab for plots, one for the raw record table, one for a per-kind summary — that lets you switch between graphical, tabular and high-level views of the same loaded file without re-reading from disk. This chapter walks through the layout and the interaction patterns; the plot catalogue itself lives in chapter 9.

8.1 Layout

The Analysis tab consists of three regions (the working dir lives in the application-wide top bar above the tabs — section 4.1 of the main-window chapter — not in the tab itself):

1. The **left column**, a vertical splitter with two halves:
   - **upper half**: the file tree, with `+ Add external file...`
   - ⟳ **Refresh** buttons at the bottom (Refresh re-scans the working dir when you've dropped bins in by hand) and `→ Overlay selected` below them;
   - **lower half**: the plot tree, with the `▦ Tile selected (N)` button at the bottom. The splitter bar between the two halves is draggable; pull it up to give the plot tree more room when many plots are visible, pull it down when you want more file rows.
2. The **right column**, a vertical stack:
   - a **tab bar** at the top with three tabs: **Plot** (the canvas), **Table** (a spreadsheet view of the loaded file's records), and **Info** (a per-kind key/value summary populated from the file + its run snapshot);
   - inside the **Plot** tab: an **animation bar** that appears only when the active plot is 3D (chapter 9 covers it), followed by the **canvas** (matplotlib for 2D plots, VTK for 3D). The matplotlib toolbar carries a **Plot options...** button on its right edge for the 2D plot — today's actions are listed under *Plot options* below;
   - inside the **Table** tab: a `QTableView` over the loaded record array, populated whenever you click a file (see section *The Table tab* below);
   - inside the **Info** tab: a two-column key/value table summarising the loaded binary (see section *The Info tab* below);
   - an **info label** at the bottom (shared between the three tabs), with the current file or operation summary.
3. The **horizontal splitter** between left and right columns is draggable too.

Switching tabs on an already-loaded file is **free**: the loaded record array is held in memory once, and both tabs render off the same data. The Plot tab does not re-render on every tab switch — it only fires when the active plot is empty (e.g. you loaded a new file while looking at Table and then switched to Plot for the first time on that file).

The window-size and splitter positions are not yet persisted across launches in this release; default sizes apply on every launch.

8.2 The shared working directory

The Analysis tab does not own its own working dir — it consumes the **shared one** from the application top bar (see chapter 4, section 4.1). The file tree's **In folder** section auto-populates from a fully recursive scan of that path; build / VCS / venv folders (`__pycache__`, `.git`, `.venv`, `venv`, `build`, `dist`, `node_modules`) are pruned, but `output/` is intentionally included so the per-run snapshots and bins surface.

The working directory updates **automatically** in two situations:

- when you load a TOML (the top bar's auto-adopt logic walks up the loaded path's ancestors looking for a scenario root — the closest folder that contains both an `output/` subdir and a `.toml` — and the Analysis tab follows along);
- when a run completes (the file tree rescans the same dir so new files appear immediately).

You can also set it manually via the global top bar's **Browse…**, or refresh the current scan with the ⟳ **Refresh** button next to **+ Add external file…** — useful when you produce files outside of SpOdy.

8.3 The file tree

The file tree has three top-level sections:

1. **In folder (`<working_dir>`)** — everything the scan found, grouped by **per-run folder**. The engine creates one `<UTC-ISO8601>/` directory per invocation under `output_dir` (see chapter 6), and the file tree mirrors that grouping: every run becomes its own collapsible header (`run: 2026-06-09T120000Z`), most-recent first. Files that do not sit inside any run folder land in a final *loose files* group, collapsed by default. File names inside a run folder carry the run's timestamp as a prefix (e.g. `2026-06-09T120000Z_lro_state.bin`) so they're unambiguous when copied or moved out of context.
2. **External (N)** — explicit files you added via **+ Add external file…**. These remain in the list across working-directory changes, so you can keep a reference file visible regardless of which run-output folder you are looking at.

Each leaf shows the basename of a `.bin` file. Hovering a leaf shows the absolute path. Adding an *external* file via the button also loads it immediately (single-click load), which is a small convenience — you typically add an external file because you want to look at it next.

8.3.1 Selection modes

The file tree supports two interaction modes:

- **Single click** on a leaf — loads that file. The application detects its type from the 8-byte magic in the header and rebuilds the plot tree with the catalogue applicable to that file's kind:
  - `SPDYOUT_` — trajectory (state vectors + time)
  - `SPDYACC_` — per-force accelerations breakdown
  - `SPDYEVT_` — events log from a single `propagate` run
  - `SPDYEVTB` — **aggregated batch events** (one file per batch invocation, with a `case_idx` column joining each row back to a CSV case). Sourced from `cmd_batch`'s `events_log` stream. The plot catalogue under this kind has a richer set of views (impact lat/lon map, density heatmap, 3D impact view, … see chapter 9).

- **Ctrl-click / Shift-click** — extends the selection (Qt's *ExtendedSelection* mode). The currently-loaded file (the one whose data feeds the active plot) does not change. Multi- selection is the input for the **Overlay** and **Diff** buttons.

8.4 The plot tree

The plot tree below the file splitter is the catalogue of plots applicable to the currently-loaded file's kind. It is grouped by topic (for trajectories: *State vectors*, *Orbit shape*, *Orbital elements*, *Diff (pick 2 files)*) with collapsible folders. Single- click a leaf to render it into the canvas; re-clicking the same leaf re-plots.

Three things to know about it:

1. **Click is dispatch** — there is no "Plot" button to press after picking a leaf. The plot fires on the click.
2. **Ctrl-click extends** — the *Tile selected* button below the tree uses the multi-selection (chapter 9, section 9.4).
3. **The animation bar appears only for 3D plots** — the row immediately above the canvas hides itself when the active plot is 2D, because there's nothing to animate there. When a 3D plot is selected the bar reappears with play / scrub / speed controls plus a **Scene…** button that opens the per- scene options dialog (per-body visibility, triads, trail, CR3BP primary selector for osculating elements).

The plot tree is rebuilt every time you load a different *kind* of file (e.g. switching from a trajectory to an accelerations breakdown). Within the same kind it persists across loads.

8.5 Overlaying multiple files

Selecting multiple files in the file tree (Ctrl/Shift-click) and pressing → **Overlay selected** plots them together on a single axes, with a turbo-coloured palette and a legend listing each file basename.

The overlay only applies to plots that draw **one line per file**. A plot that already draws three lines (the per-component `Position x, y, z`, `Velocity vx, vy, vz`, the per-force accelerations breakdown) is not overlay-safe; selecting it and pressing **Overlay** triggers an explanation dialog instead of an illegible 3N-line plot.

The button works for the **3D orbit + Moon** plot too. In that case the overlay paints each trajectory as its own polyline in the same VTK scene, with a viewport legend and **Ctrl+click picking** enabled on every polyline (more on picking in chapter 9, section 9.7).

A few rules:

- All selected files must be the **same kind** as the currently loaded one. Files of a different kind are quietly skipped and listed under *Skipped* in the info label.
- The overlay tolerates **different time grids** as long as the underlying plot uses each file's own t-axis (e.g. `|r|(t)` plots each file's own `t` independently). Time-aligned overlay is not available; for sample-aligned comparison use the diff plots.
- The **Overlay** button silently ignores **Diff (pick 2 files)** specs — clicking a diff plot is its own dispatch pathway and does not need an overlay button.

8.6 Tiling several plots

The ▦ **Tile selected (N)** button below the plot tree is the dashboard mode. Multi-select N plot leaves with Ctrl/Shift-click (the counter on the button updates live), then click. The canvas splits into `ceil(sqrt(N)) × ceil(N/cols)` subplots, one per selected plot, all rendered against the currently-loaded file.

Two modes are auto-detected from the selection:

- **All single-file plots** — the subplots draw against the loaded file. Useful for at-a-glance overview: select |r|, |v|, a, e, i, Ω (six leaves) and the result is a 2×3 dashboard of the most relevant orbit quantities.
- **All diff plots** — the subplots draw against the two files selected in the *file tree* (with the same selection rule as a single diff click). Read disks once, render four diff subplots showing |Δr|, |Δv|, components, and RIC decomposition.

Mixed sets (some single, some diff) are refused with a clear message. **3D plots** are filtered out and the count of skipped 3D plots is reported in the info label. The hard cap is **12 subplots** to keep the result legible; selecting more triggers a warning and aborts the tile.

8.7 Plot options (Export CSV, Plot frame, ...)

The **Plot options...** button on the matplotlib toolbar opens a small non-modal dialog that hosts per-plot actions on the currently-displayed 2D figure. Today it exposes:

8.7.1 Plot frame (ICRF / body-fixed)

A radio group that picks the basis the plot fns interpret state vectors and Keplerian angles in:

- **ICRF (inertial)** — the integrator's native frame; the default, leaves every plot exactly as before.
- **Body-fixed** — the central body's body-fixed basis at the corresponding ET (Earth ITRS via `spopy.icrf_to_itrs`, Moon PA via the DE-series libration angles). The radio's label tracks the body name (`Body-fixed (ITRS)` / `Body-fixed (PA)`). Disabled (greyed back to ICRF) for CR3BP runs and for central bodies without a registered orientation provider.

The selection re-renders the active plot immediately. Affects:

- **State vectors** — `|r|`, `|v|`, `x, y, z`, `vx, vy, vz` (magnitudes are basis-invariant and only the title suffix changes there).
- **Orbit shape** — the XY / XZ / YZ projections trace the ground track in BF (closing after one body rotation) instead of the inertial orbit.
- **Orbital elements** — the magnitude triplet (a, e, i) round-trips unchanged across the swap; **RAAN**, **AOP**, **true anomaly** and the e-vs-ω phase plot get the rotation baked into the angles (their evolution then reflects the BF basis spinning under the orbit).

Pure rotation only (no ω × r correction). The same convention is used by the `central_body_fixed` initial-state frame, so values round-trip cleanly through both the form input and the plot view.

8.7.2 Export CSV

The **Export CSV** box lists the CSV export types that apply to the current plot / file as a set of radio buttons, with one *Export* button that saves the selected one. Each type greys out on its own when it doesn't apply — so you see *which* exports exist and why one is unavailable, instead of a lone button that mysteriously greys. If the selected type becomes unavailable (you switch plots or files) the selection jumps to the first available one; the *Export* button is active whenever at least one type is available. The dialog shows a *Saving to…* status during the write and auto-closes on success; errors stay visible so you can adjust the destination.

Four export types today:

- **Plot lines (as drawn)** — every `Line2D` on the active figure (single, overlay and tile modes supported). One section per subplot, separated by blank lines and a comment header with the subplot title; lines sharing an x-array collapse to a single `x` column plus one column per series, others get paired `x_<label>, y_<label>` columns. Scatter / fill / heatmap layers (impact maps, density plots, altitude-band views) carry no `Line2D`, so this type is greyed out there. Also greyed while the active plot is 3D (that canvas is VTK, not matplotlib).
- **Altitude bands (per batch element)** — enabled when the file is an events log with central-body altitude crossings. **One row per batch element** (ascending case id; a single row for a `propagate` run) and, for each band in ascending-altitude order, a **paired** `t_<lo>-<hi>km_s` (total time in band) and `entries_<lo>-<hi>km` (crossings *into* the band, never exits) column set. A `#`-comment header records the body, the thresholds and whether they came from the run snapshot or were clustered out of the records. A `bands window: 0 to [N] days` field appears next to the export types while this one is selected: leave it blank to cover the whole run (the default, reconciling with the Info tab's pooled view), or type an upper bound to restrict the statistics to the first *N* days of sim time. A segment straddling the bound contributes only its part before it, and only crossings up to the bound count as entries; the window and its bound are recorded in the `#`-header (`window_s,0..<seconds>`) and the suggested filename gains a `_0-<N>d` suffix. The Info tab is unaffected — it always reflects the whole simulation.
- **Impact points (lat/lon + time of flight)** — enabled when the file has at least one IMPACT and the central body has a body-fixed frame. One row per impact: `case_id`, body-fixed `lat_deg` / `lon_deg` (the same projection as the impact lat/lon maps) and time of flight (`tof_s`, `tof_days`), rows ascending by case id.
- **Band snapshot (per object, at one instant)** — the instantaneous companion of the cumulative bands table, enabled once the file qualifies for that one *and* a snapshot time is set (see below). One row per object: `case_id`, the `band` index it occupies at that instant, and that band's `band_lo_km` / `band_hi_km`. Objects whose run had already ended are kept with `band = -1` and empty range cells, so the row set is the whole population and a reader can compute fractions of it without a second file. The `#`-header records the instant twice (`t_s` and `t_days`), the object count and how many had ended. There is deliberately **no altitude column**: between two crossings the reconstruction knows which band an object is in, not where inside it, and an interpolated altitude would be fabricated — read the trajectory `.bin` if you need the actual altitude at an instant.

The band, snapshot and impact exports read the events data, not the figure, so they work from any plot in the events tree (not only their matching view).

8.7.3 Altitude-band snapshot

Its own box, above *Export CSV*: a single field, `at [N] days from start`, with the loaded run's length shown underneath so the number has a scale. It sits outside the export box because it drives both the *Band snapshot at t* plot (chapter 9) and the snapshot export — tying a plot parameter to an export radio's enabled state would be wrong. Confirm with Enter (or by leaving the field) and the active plot redraws; clear it to unset. It is independent of the `bands window` field inside the export box, which means something different: that one is the *upper bound of a cumulative window*, this one is *an instant*.

8.8 3D UTC overlay

A small text overlay at the bottom-right of the 3D canvas shows the UTC corresponding to the current animation tick (the spacecraft marker's time on the slider, converted via `spopy.time.et_to_utc` of `et_start + t_anim_s`). Anchored to the viewport via a `vtkTextActor` 2D, so it tracks the canvas size; cleared when the canvas leaves 3D or when the loaded run has no `et_start_s` (CR3BP, bare `.bin` without snapshot). Format is `UTC 2026-06-25T12:34:56.789Z`, fractional seconds at millisecond resolution.

8.9 3D scene options — starfield background

The Scene options dialog (animation bar on the 3D canvas, see chapter 9) has a **Show starfield** checkbox that swaps the dark background for an equirectangular star map. The asset (Solar System Scope Milky Way 8K, CC BY 4.0) ships through the Setup wizard alongside the Moon and Earth textures; until it is downloaded the checkbox stays disabled with an explanatory tooltip.

On first activation, SpOdy re-projects the wizard image into the ICRF axes the scene uses (pole = +Z, vernal equinox = +X). The shipped texture is in galactic coordinates (image centre = Sgr A*), so the re-projection chains a standard ICRF→galactic rotation before the (l, b) lookup — without it the bulge would land in the wrong patch of sky and breach the "looks right when overlaid with the ICRF triad" sanity test. The rotated copy is cached on disk next to the source as `<stem>_icrf<ext>` so subsequent runs are instant. Toggle state is persisted in QSettings, so the next session opens to the same background.

The skybox lives on the FAR (background) renderer with `vtkSkybox. Sphere` projection. The shader forces depth = 1 (far plane), so spacecraft trajectories, central body, third-body markers and triads all render in front of it correctly regardless of zoom level.

8.10 3D scene options — sun illumination

The **Sun illumination (day/night terminator)** checkbox (on by default, HF scenes only) replaces the default camera headlight — which always lights whatever side you are looking at — with a directional light aimed from the Sun's true position, read from the ephemeris at each sample of the loaded trajectory. The effect:

- the central body and every textured third-body marker show a physical day/night terminator with hard contrast; the night side stays just barely visible (low ambient) so the body outline never quite disappears against the dark sky;

- a thin orange ring on the central body traces the terminator itself (the great circle perpendicular to the sun direction), so the shadow front is explicit even where the shading falloff is gradual;
- during playback the light re-aims every animation tick, so on multi-day runs you can watch the terminator sweep across the rotating surface;
- the Moon marker in an Earth-centred scene (and vice versa) shows its actual phase, since one parallel sun direction lights every body in the scene consistently;
- the Sun's own marker is exempt (it is the light source), and trajectories, arrows, triads and the spacecraft marker keep their flat colours — only body surfaces are shaded.

Unchecking the box restores the headlight look (every visible surface lit). The option is hidden for CR3BP scenes: the synodic frame has no epoch, hence no sun direction to point the light at.

8.11 Picking in the 3D viewer

When a 3D plot is active, **Ctrl+left-click** on any rendered trajectory polyline picks it: the line is highlighted, the matching file is selected (highlighted) in the file tree on the left, and the info label below the canvas reports the picked file's path. Pointing at the central body (the Moon sphere) does nothing. This is mostly useful in overlay scenes, where ten lines may cross each other and you want to know which file produced which.

The pick interaction does not change which file is loaded, only which one is *highlighted*. Click a file in the tree (or Ctrl- click anywhere on a polyline) to keep working with it.

8.12 The Table tab

The Table tab is the spreadsheet view of the loaded file's records. It uses the same underlying numpy structured array the Plot tab does — no re-read happens when you switch tabs.

Column layout follows the file's dtype:

- Scalar fields appear as a single column with the field's name (`t`, `kind`, `naif_id`, …).
- **Nested array fields** are flattened into N columns named `<field><i>`: e.g. an `EventRecord`'s `y[6]` shows up as `y0`, `y1`, …, `y5`.
- **Padding fields** (names starting with `_`, used to align C-side dtypes) are **hidden** so the view stays tidy.
- A handful of fields get **friendlier display names** where the on-disk name is misleading: the events files store the trigger metric in a `distance_km` slot, but the slot is a *jolly* (impact distance for IMPACT, eclipse fraction for ECLIPSE, satellite-to-body distance for ALT_CROSSING — subtract `radius_km` for the attained altitude), so the table header surfaces it as `trigger_value`. The TOML schema and the engine's on-disk format are unchanged — only the column label differs.

Floating-point cells are formatted with **12 significant digits**, enough to round-trip a typical km-scale state vector through the clipboard without surprise. Integer cells stay raw. The events `kind` column gets the symbolic label (`IMPACT`, `ECLIPSE`, `ALT_CROSSING`) instead of the enum integer.

8.12.1 Spreadsheet-style selection

The table accepts the conventional spreadsheet click patterns:

- **Click a cell** to select just that cell.
- **Click a column header** to select the whole column.
- **Click a row index** (left edge) to select the whole row.
- **Shift-click** extends the rectangular selection; **Ctrl-click** toggles individual cells in or out (non-rectangular sets are supported).

Ctrl+C copies the current selection to the clipboard as **TSV** (Tab-Separated Values, one row per line, one column per tab). Cells outside the selection in a non-rectangular set are emitted as empty fields so the row alignment is preserved. Paste straight into Excel / LibreOffice / a Jupyter cell and the columns line up; pasting into a chat or a code comment also works because the format is plain UTF-8 text.

This is the path to take whenever you want the *numbers* — the Plot tab is for the *picture*.

8.13 The Info tab

The Info tab is a **per-kind summary** of the loaded binary. It trades off the Plot tab's visual answer and the Table tab's exhaustive record list for a compact one-screen overview: which file you're on, what the run setup was, and a handful of key statistics computed from the data.

The tab refreshes on three triggers:

- when you click a new file in the file tree;
- when you click a plot in the plot tree (so the diff-aware rows described at the end of this section appear / disappear with the plot selection);
- when you switch into the tab on an already-loaded file.

What it costs. The summary is computed **once per loaded file**, the first time you actually look at it: the tab is not rebuilt while it sits behind the Plot or Table tab, and every later visit or plot click reuses the stored result. On ordinary runs that first pass is imperceptible. On a very large events log — a debris batch of a few thousand cases can reach ten million triggers — it is a few seconds, spent under a wait cursor with a `Working: computing info…` note; after that the tab is instant for as long as the file stays loaded, and the event plots reuse the same work rather than repeating it. If you want that pass out of the way before you start clicking around, just switch to Info once after loading the file.

8.13.1 Run summary (always shown)

Every kind starts with a *Run summary* section. The first four rows always come from the file itself:

- **File** — the binary's filename.
- **Folder** — the absolute parent directory.
- **Type** — one of `trajectory (SPDYOUT_)`, `accelerations (SPDYACC_)`, `events log (SPDYEVT_)`, `events log (SPDYEVTB, batch-aggregated)`.
- **Records** — the record count.

The remaining rows come from the **per-run snapshot TOML** the engine drops next to the bin (`<ts>_input.toml`); they are skipped when no snapshot sits alongside the loaded file:

- **Central body** — `Moon` / `Earth` / …
- **Dynamics model** — `high_fidelity` or `cr3bp`.
- **CR3BP primaries** + **CR3BP mass ratio μ** — only shown in CR3BP runs.
- **ET start [s]** — the simulation epoch.
- **Planned duration** — the scheduled `duration_s`, rendered in seconds plus a friendly unit (min / h / d) once it crosses the obvious thresholds.
- **Ephemeris** — the resolved DE-series file's basename.
- **Cases file** — the resolved batch CSV's basename (batch runs only).

8.13.2 Trajectory files (`SPDYOUT_`)

Two sections on top of *Run summary*:

- *Trajectory* — t range and span (s + d), Δt min/avg/max, |r| min/max [km], |v| min/max [km/s];
- *Initial state* and *Final state* — the (x, y, z) and (vx, vy, vz) triplets at t0 and tf.

For high-fidelity runs whose central body is registered (Moon, Earth) a final *Kepler elements (HF, central body)* section adds the six classical osculating elements (a, e, i, Ω, ω, ν) at t0 and tf, computed via `spopy.cartesian_to_ keplerian` with the body's GM. The section is silently omitted in CR3BP runs (the relevant frame is primary-relative; switch to the Plot tab's *Orbital elements* group with the Scene- options dialog's primary selector for a per-primary breakdown).

8.13.3 Acceleration files (`SPDYACC_`)

- *Accelerations* — t range, span, Δt stats, plus |a_total| min/max/mean/RMS [km/s²];
- *Per-force RMS [km/s²]* — one row each for the 2-body, harmonics, 3rd-body, SRP, and drag components;
- *Eclipse* — the minimum `eclipse_fraction` reached over the run plus the **time in shadow** integrated trapezoidally over the (1 − fraction) signal.

8.13.4 Events files (`SPDYEVT_` / `SPDYEVTB`)

The top *Events* section is shared between per-run and batch events: **Total records**, **IMPACT count**, **ECLIPSE count**. A file carrying life markers (chapter 6) adds **Objects propagated** — the authoritative object count, straight from the log rather than from `cases.csv` — and, only when the number is non-zero, **Objects never closed**, i.e. cases whose `FINAL_STATE` never arrived because the run died mid-flight. Batch logs add three extra rows up there (**Cases with events**, **Cases with impact**, and — when the cases CSV is resolved next to the snapshot — **Cases total (CSV)**, **Survivors (no impact)**, **Impact rate %**). *Cases with events* counts cases that fired a **predicate**: the markers are excluded from it, otherwise it would silently become a second, redundant batch-size row and stop agreeing with the same row on an older file.

Two derived sections follow when the underlying counts are non-zero:

- *Impact timing* (any IMPACT row present) — first / last / median / mean impact time, each formatted as raw seconds with the friendly (min / h / d) equivalent in parentheses.
- *Altitude bands* (any central-body `ALT_CROSSING` row, **or** a central-body life marker with thresholds in the run snapshot — a run whose objects all stayed inside one band has no crossings at all, and is precisely the one whose occupancy is worth reading) — the configured thresholds, sorted ascending,

split the altitude axis into bands `[surface, h_a)`, `[h_a, h_b)`, …, `[h_top, &infin;)`. For each band the summary reports **Entries** (crossings *into* the band), **Time in band** (with % of the analysis window in a single run, or the summed *object-time* in batch), **Visit duration** min / avg / max, and — in batch — **Population** min / avg / max (objects simultaneously in the band, time-averaged) plus **Objects visiting**. The timeline is reconstructed from the crossing records alone (no trajectory file): the crossed threshold comes from `distance_km - radius_km` snapped to the nearest configured value and the direction from the sign of **r·v** at the trigger state. Where each object *starts*, and when its window *ends*, are read from its life markers (chapter 6); only on a pre-marker file are they inferred instead — the start band from the direction of the first crossing, the window from the earliest of the planned duration, the object's impact, and its first stop-class crossing. Thresholds entered out of order (or duplicated) are sorted and de-duplicated, so the bands are always well-formed.

Three rows appear only with markers. **Objects analysed** says *propagated* rather than *with ≥ 1 crossing*, because the object set is now the whole batch; **Confined to one band** counts the objects that never crossed a threshold at all, which a pre-marker file lost silently; and **Start-band mismatches** counts objects whose measured start band disagrees with the one inferred from their first crossing. That last row is a data-quality warning rather than a statistic: the two can only disagree if a crossing went unrecorded, which points at an `[[events.altitude_crossing]]` entry running with `refined = false` and a step long enough to jump a whole band. Crossings measured from a third body get a counts-only line (their trigger state isn't body-centric, so occupancy isn't reconstructed). - *Eclipses* (any ECLIPSE row present) — **Trigger records** is the raw count of crossings (entry + exit). The engine emits one record per threshold crossing of the signed (fraction − threshold) predicate, so successive triggers for the same `{case_idx, naif_id}` group alternate entry ↔ exit; consecutive pairs are one **complete eclipse** with duration `t_exit - t_entry`. The summary reports the pair count and the min / avg / max duration. Odd tails (the run started or ended inside shadow) are silently dropped from the pairing — expect `2 &times; Complete eclipses + odd-tails = Trigger records` per group.

8.13.5 Diff overlay (plot-aware)

When the active plot in the plot tree is one of the *Diff (pick 2 files)* specs — and a successful diff dispatch has already cached the aligned pair — the Info tab appends a section named after the plot label with:

- the A and B filenames;
- the alignment note ("B interpolated onto A's grid") when the two grids didn't match;
- |Δr| **max / mean / RMS / final** [km];
- |Δv| **max / mean / RMS / final** [km/s];
- |Δr| **linear growth** as the least-squares slope in km/day;
- **RIC frame (A)** — |Δ| max and RMS broken down into radial / in-track / cross-track in A's local frame.

The overlay disappears as soon as you click a non-diff plot or load a different file.

9. Plot catalogue

This chapter is the reference for every plot SpOdy ships with. The entries are grouped first by file kind (trajectory, accelerations, events) and within each by the topic folder the plot tree shows them under. Each entry documents what the plot displays, the formulas used, whether the plot is *overlay-safe* (a single line per file, so an N-file overlay produces N lines rather than 3N), and whether it is single- or two-file (*diff*).

Notational conventions throughout the chapter:

- `t` — time elapsed from the start of the propagation, in seconds.
- `r, v` — position and velocity in km and km/s, in the central-body inertial frame (chapter 10).
- `|x|` — Euclidean norm of vector `x`.
- The horizontal axis of any *(t)* plot is the trajectory's own time column, in seconds. The toolbar at the top of the canvas lets you zoom, pan, and save the current view as PNG.

9.1 Trajectory plots (`SPDYOUT_`)

9.1.1 State vectors

Radial distance |r|

Distance of the spacecraft from the central body's centre, as a function of time.

Formula. `|r|(t) = sqrt(x^2 + y^2 + z^2)`.

When to use. Quickest sanity check that the orbit is in the right altitude band. For LRO, the average altitude is about 50 km above the Moon's mean radius of 1737.4 km, so `|r|` oscillates around 1788 km.

Overlay-safe. Yes.

Speed |v|

Magnitude of the inertial velocity vector.

Formula. `|v|(t) = sqrt(vx^2 + vy^2 + vz^2)`.

When to use. Companion to `|r|(t)` — for an elliptical orbit, `|v|` oscillates inversely to `|r|` per the vis-viva relation.

Overlay-safe. Yes.

Position x, y, z

The three Cartesian components of the position vector, on a shared axes.

When to use. When you want to see which component is the dominant driver of an altitude excursion, or to spot a slow secular drift on a single axis (typical of certain harmonics-driven perturbations).

Overlay-safe. No (draws three lines per file).

Velocity vx, vy, vz

Same as the position components but for the velocity vector.

Overlay-safe. No.

9.1.2 Orbit shape

XY projection (and XZ, YZ)

Projection of the orbit onto the named coordinate plane of the inertial frame, with a green dot at `t = 0` and a red dot at the final sample.

When to use. Visualises the orbit's geometric shape and the sense of motion (the trajectory line goes from green to red along the orbit's direction). For a near-polar lunar orbit the XY plot looks like a tight loop; the XZ and YZ projections show the inclination directly.

Overlay-safe. Yes.

3D orbit + central body

The trajectory rendered as a polyline in a 3D scene with the central body sphere at the origin. The sphere is textured with the equirectangular image configured in **Settings › Paths** for the active central body (Moon texture for `central_body = "Moon"`, Blue Marble texture for `central_body = "Earth"`); when the texture is absent the sphere falls back to flat grey.

The scene also draws **two reference-frame triads** from the central body's centre (introduced in v0.1.2-beta for the Moon and generalised in v0.1.3-beta to track the active central body):

- **Body-fixed triad** (bright RGB, ~2.1 body radii long) with billboard labels `X_pa / Y_pa / Z_pa` for the Moon (Principal Axes) or `X_itrf / Y_itrf / Z_itrf` for the Earth (ITRS). For Earth the triad **rotates in real time** along the animated trajectory: at every animation frame the engine's `R_ICRF→ITRS` rotation is recomputed (IAU 2006/2000A_R06 + the IERS EOP table) and applied to the basis vectors, so the triad and the textured continents stay locked to the same Earth orientation. For the Moon the same animation step applies the libration-driven `R_ICRF→PA` rotation from DE440.
- **ICRF triad** (muted RGB, ~1.8 body radii long, opacity 0.25) with labels `X_icrf / Y_icrf / Z_icrf`, fixed in scene coordinates. Useful as a static reference against the rotating body-fixed triad.

Interactions.

- Left-drag rotates the camera.
- Scroll zooms in and out.
- Middle-drag pans the camera.
- `r` resets the camera to the default oblique view.
- `Ctrl` +left-click on a polyline picks it (in overlay mode).

The Sun-arrow row above the canvas applies to this plot. Type an epoch (TDB seconds past J2000) into the field and click **+ Sun arrow** to add an arrow pointing from the central body toward the Sun at that epoch. The epoch field auto-fills with the currently-loaded TOML's `simulation.et_start_s` so it usually contains a sensible value already.

When the **Moon appears as a third body** in an Earth-centred scene (`third_bodies = ["Moon", ...]`), its marker is rendered as a textured sphere using the wizard-managed Moon texture (rather than a flat-grey dot), so the Moon stays recognisable at its true ~384,000 km distance.

Overlay-safe. Yes (the overlay variant adds N trajectories in turbo colours plus a legend).

9.1.3 Orbital elements

The six entries in this folder are the classical Keplerian elements computed from the state vectors at every sample. The shared implementation handles the two degenerate cases gracefully: in a circular orbit (`e ≈ 0`) the argument of periapsis and the true anomaly are set to 0; in an equatorial orbit (`i ≈ 0`) the RAAN is set to 0 and its rotation is folded into the argument of periapsis. The thresholds are tight enough (`1e-8`) that any realistic propagated orbit is unaffected.

The default central-body gravitational parameter is the Moon's (`mu = 4902.800066 km^3/s^2`).

Semi-major axis a

Formula. From the vis-viva relation: `1/a = 2/|r| - |v|^2 / mu`.

Units. km.

Overlay-safe. Yes.

Eccentricity e

Formula. Magnitude of the Laplace-Runge-Lenz vector: `e_vec = (v × h) / mu - r/|r|`, where `h = r × v` is the specific angular momentum.

Units. Dimensionless.

Overlay-safe. Yes.

Inclination i

Formula. `cos(i) = h_z / |h|`.

Units. Degrees.

Overlay-safe. Yes.

RAAN Ω

Formula. With `n = z_hat × h = (-h_y, h_x, 0)` the node line, `cos(Ω) = n_x / |n|`, quadrant resolved by the sign of `n_y`.

Units. Degrees, range `[0, 360)`.

Overlay-safe. Yes.

Argument of periapsis ω

Formula. `cos(ω) = (n · e_vec) / (|n| |e_vec|)`, quadrant resolved by the sign of `e_vec_z`.

Units. Degrees, range `[0, 360)`.

Overlay-safe. Yes.

True anomaly ν

Formula. `cos(v) = (e_vec · r) / (|e_vec| |r|)`, quadrant resolved by the sign of `r · v`.

Units. Degrees, range `[0, 360)`.

Note. For a multi-revolution propagation `v(t)` shows the saw-tooth wrap at every orbit. This is the correct shape of the wrapped angle; if you want a monotone "cumulative" angle, derive the mean anomaly externally.

Overlay-safe. Yes.

9.1.4 Diff (pick 2 files)

These plots subtract one trajectory from another. Selecting one of them in the plot tree requires **exactly two** trajectory files selected in the file tree (sorted top-down = A then B); the dispatch applies the operation and renders the result.

If the two files share the same time grid sample-by-sample, the subtraction is direct. If they do not, B is automatically interpolated onto A's time grid restricted to the overlap window: position uses cubic Hermite interpolation with B's `v` as the analytical derivative, velocity uses linear per-component. The plot title gets a `(B interpolated)` suffix and the info label states the interpolation parameters so you know the diff is not direct.

Time-window mismatches with no overlap are reported with a clean message rather than a numpy traceback.

|Δr| (log y)

Formula. `|r_A(t) - r_B(t)|` on a logarithmic y axis.

When to use. The default error-magnitude plot for orbital regression. The log scale spans the typical many-orders-of- magnitude growth from sub-millimetre level (when B is an interpolation of a finer-grid reference) up to kilometre level over multi-day propagations.

|Δr| (linear y)

Same data on a linear axis. Better for inspecting the *shape* of the error growth near the end of the propagation, where the log scale flattens out.

|Δv| (log y)

Formula. `|v_A(t) - v_B(t)|`.

The velocity error sits typically two to three orders of magnitude below the position error for well-tuned propagators (the velocity field is smoother than the position field, and integrator errors accumulate quadratically in position vs linearly in velocity).

|Δv| (linear y)

Same data on a linear axis.

Δx, Δy, Δz per component

Per-component position difference: `A.x - B.x`, `A.y - B.y`, `A.z - B.z` on a shared axes with a legend.

When to use. When you want to see which coordinate axis dominates the error and at what frequency. Modulations periodic with the orbital period are common; secular drift on one component points at a specific perturbation mismatch.

RIC frame (radial/in-track/cross-track)

The most informative diff plot for orbital regression. The position-error vector is projected onto the RIC frame of trajectory A:

- **Radial** axis: `r_hat_A` (positive outward).
- **Cross-track** axis: `c_hat_A = (r_A × v_A) / |r_A × v_A|` (orbit normal, positive along `h`).
- **In-track** axis: `i_hat_A = c_hat × r_hat_A` (completes the right-handed frame; for circular orbits this aligns with the velocity direction).

The three components are plotted on a shared linear y axis with a legend.

Interpretation.

- A growing **in-track** component is the signature of a small **energy / timing drift** — the two propagators have slightly different mean motion and one drifts ahead of the other along the orbit. Typical for harmonics-degree mismatches.
- A growing **radial** component points at an **altitude error** — one orbit sits slightly higher than the other on average.
- A growing **cross-track** component points at an **out-of-plane / nodal drift** — difference in i or RAAN.

For a tuned high-fidelity setup on a near-circular orbit, the in-track component is the largest by an order of magnitude or more.

> *The "in-track" axis here is the standard RIC convention (Vallado's "S" axis, also called the Hill frame). It is **not** exactly the LVLH velocity axis, although for circular orbits the two coincide. See chapter 10, section 10.4 for the explicit relation.*

|Δr| distribution

Histogram of the per-sample position-error magnitude across the shared sample grid. Bin count is `min(60, sqrt(N))` so a 6-day LRO regression (~9k samples at 1-minute cadence) gets ≈60 bins and a short smoke test still produces a readable histogram.

A small box in the bottom-right corner of the figure reports the **RMS**, **median**, **p95** and **max** of the distribution. The RMS is `sqrt(mean(|Δr|^2))`, the canonical single-number summary in orbit-determination and conjunction work; the percentiles are the standard summary when the underlying error

distribution is **not normal** (which is the usual case for SPICE-vs-SpOdy `|Δr|`).

|Δr| empirical CDF

The cumulative distribution function of the same `|Δr|` samples, drawn as a `steps-post` line from 0 to 1. The CDF is the natural tool when the question is **"what fraction of samples is below this error?"** — read the curve at the horizontal value of interest, scale up to the percentile on the y axis. Conversely, to find the value such that 95% / 99% / 99.9% of samples are below it, scan the y axis at 0.95 / 0.99 / 0.999.

A box in the bottom-right corner of the figure reports the **RMS**, **median**, **p95**, **p99**, **p99.9** and **max**. These (minus the RMS) are the **distribution-free percentile budgets** — the right answer regardless of whether the underlying distribution is normal, log-normal, multi-modal, or anything else. The standard "`mean &plusmn; 2σ`" 95% interval only works if the distribution is normal; the empirical p95 read off the CDF works in every case, which is the point of this plot for regression work. The RMS sits alongside the percentiles because OD and conjunction budgets traditionally quote it.

9.2 Accelerations plots (`SPDYACC_`)

The accelerations binary records the per-force accelerations at every record, alongside the total. These plots break the contributions down.

Third bodies are named individually rather than summed into one `3rd-body` series. The binary stores only how many third bodies were active, so the names come from the `force_model.third_bodies` list in the run folder's `input.toml` snapshot, which the engine fills the per-body slots from in the same order. If that snapshot is missing or lists a different number of bodies, the plots fall back to positional labels (`3rd-body #0`) rather than risk attaching the wrong name to a curve.

Every acceleration plot picks its time unit from the span it draws (seconds, minutes, hours or days), so a multi-day run does not print a six-digit second count. In an overlay the first file fixes the unit for the whole axis.

Total |a_total|

Formula. `|a_total| = sqrt(a_x^2 + a_y^2 + a_z^2)` on a logarithmic y axis.

When to use. Quickest sanity check on the order of magnitude of the total perturbing acceleration. For a low-lunar-orbit at N=80 harmonics, `|a_total|` is dominated by the central two-body term (a few `m/s^2` at the surface, dropping with altitude) and the harmonics contribution (smaller by an order of magnitude).

Overlay-safe. Yes.

Per-force breakdown (log y)

Each of the active force contributions (two-body, harmonics, each third body by name, SRP, drag) is plotted as its own line on a logarithmic y axis with a legend. A force that is switched off for the run is omitted rather than drawn flat at the axis, so an empty legend slot never gets mistaken for a negligible contribution.

When to use. The "who is doing what" plot — tells you which force dominates at every part of the orbit. Typical observations: SRP shows the eclipse cuts (zero during umbra, modulated by penumbra); harmonics oscillates with the spacecraft's orbital phase; third-body Earth dominates over third-body Sun for near-

Moon orbits.

Overlay-safe. No (draws multiple lines per file).

Perturbation budget (share, log y)

The same channels as the breakdown, minus the central two-body term, each divided by the sum of all of them and drawn as a filled curve on a logarithmic percentage axis. The legend carries each force's time-averaged share, which ranks them without reading the axis.

Formula. `share_k(t) = 100 * |a_k(t)| / Σ_j |a_j(t)|`, where *j* runs over harmonics, each third body, SRP and drag.

When to use. Attribution — *which* perturbation owns a drift, and whether that changes during the run. Because the shares are normalised, the overall growth of `|a|` divides out: what is left is composition alone, so a decaying orbit shows drag taking over rather than everything simply rising together.

Two properties worth knowing before reading numbers off it. First, the shares divide **magnitudes**, so they sum to the sum of the norms and not to `|Σa|`, which is smaller wherever two forces partly cancel; this is a "who is pushing, and how hard" chart, not a vector decomposition of the net perturbation. Second, the axis is logarithmic rather than a 100%-stack on purpose: a stack is only readable when the contributors sit within about a decade of each other, which holds at GNSS and GEO altitudes but fails in LEO, where the harmonics take 99.99% and every other band would fall below a single pixel.

The bottom of the axis is set from a low percentile of the shares, not their minimum, so an SRP collapse inside an eclipse clips at the frame instead of stretching the plot over empty decades. A force that reaches exactly zero (full umbra) leaves a gap, which is the logarithmic axis being honest about it.

Overlay-safe. No (draws multiple curves per file).

Eclipse fraction

If `[events]` was enabled and the engine recorded the eclipse sunlight fraction at every step, this plot displays it on a linear y axis in `[0, 1]`. `1.0` = full sunlight, `0.0` = total umbra, intermediate values = penumbra.

Overlay-safe. Yes.

9.3 Events plots (`SPDYEVT_`)

The single-run events binary records every detected event (IMPACT, ECLIPSE entry/exit) along **one** propagation.

Events timeline

A horizontal-axis-time scatter plot with one marker per detected event. The y axis is one **series** per row: IMPACT, ECLIPSE, and — for altitude crossings — **one row per crossed altitude**, labelled with the altitude itself (e.g. `alt 45 km`) rather than the raw event-kind code. Altitude rows split ascending vs descending crossings by marker (▲ up / ▼ down); the altitude values are clustered straight from the records, so the plot still works with no `input.toml` in sight. When more than one body is involved the altitude rows carry the body's NAIF id.

When to use. Quick visualisation of when, in the simulation window, each event happens. For impact-prediction work, the first IMPACT marker gives the predicted collision time at a glance; for altitude monitoring, each band's row shows the entry / exit cadence at a glance.

On very large logs. Past 4000 triggers in a single row the markers are **thinned to canvas resolution** — at most one per pixel column, which is all a screen can show anyway — and the title says `thinned to canvas resolution`. The picture is the same solid band it would otherwise be, but it draws in a fraction of a second instead of tens of them. Below that count nothing is dropped: every trigger keeps its exact time. If you need the counts rather than the pattern, use the density variant below; if you need individual event times, read them in the **Table** tab.

Overlay-safe. No (multi-series plot).

Events timeline (density)

The aggregated companion to the marker timeline, for files with so many events that the scatter smears into a solid band. Same y-rows (IMPACT, ECLIPSE, one per crossed altitude), but each row is a **time-binned count heatmap** (300 bins) coloured by the number of events per bin instead of individual markers — the temporal density pattern stays legible at millions of events, and it draws several times faster than the scatter there. The marker timeline is kept unchanged for smaller files where individual events matter.

When to use. Large batch / debris-cloud event logs, to read *where in time* the events concentrate rather than each one.

Overlay-safe. No.

The x axis of every events plot auto-scales its unit (`s` / `min` / `h` / `days`) to the plotted span, so a days-long batch reads in days rather than a six-digit second count.

9.4 Batch-events plots (`SPDYEVTB`)

When a batch run with `events_log` enabled finishes, the engine writes a **single aggregated events file** (`SPDYEVTB` magic, chapter 7) covering every trigger across every case. Each row carries an extra `case_idx` (int32) so each event can be joined back to a row of the cases CSV. The Analysis tab exposes five views over that file in addition to the generic *Events timeline* (which still works because it only consumes `t` and `kind`):

- Time-to-impact histogram
- Survival timeline per case
- Impact lat/lon (equirect)
- Impact lat/lon (Mollweide)
- Impact density heatmap
- Impact 3D on Moon

The four "Impact …" views project IMPACT rows onto the lunar surface in the **Moon Principal Axes (PA)** body-fixed frame (chapter 10): for each event the dispatcher reads `et = simulation.et_start_s + row.t`, queries the DE440 ephemeris for the lunar libration angles via the bundled `spopy` package, builds the ICRF→PA rotation matrix, and applies it to the `y[0:3]` ICRF state. The Moon's central body is required (today `spopy`'s libration model is lunar-only; running with a different `central_body` makes these views show a friendly "not applicable" message rather than crash).

The four impact views all rely on three pieces of context the binary itself does not carry: `simulation.et_start_s` (sim time to ET), `[ephemeris].file` (to query libration), and `simulation.duration_s` (for survivor counts). They read these from the `input.toml` **snapshot** the engine drops into the run folder at every invocation. Opening an events file from outside any run folder triggers a "snapshot not found" message in place of the plot.

Time-to-impact histogram

Distribution of `t_trigger` across IMPACT rows on a 1D histogram. Bin count is Sturges' rule capped at 40; the x axis is in **days**.

Overlay-safe. No (single-file aggregate).

Survival timeline per case

One horizontal bar per `case_idx`. Bars for cases that impacted are **red**, ending at the first `t_impact`; bars for survivors are **green**, extending to `simulation.duration_s`. Cases are sorted by `t_impact` ascending; survivors follow in `case_idx` order. The title states the total number of cases, impacted count, and survivor count.

Reads `duration_s` from the run-folder TOML snapshot; the total case count comes from the `cases_file` CSV (`#`-prefixed comment lines and the header row are skipped). When either is unreachable, only impacted cases are drawn and the title flags it.

Overlay-safe. No.

Impact lat/lon (equirect)

Scatter plot of impact positions on the lunar Principal Axes frame in an equirectangular projection (extent `[-180, 180]` × `[-90, 90]` degrees). Background is the Moon texture when present (NASA SVS LROC color, chapter 3); points fall back to a flat-grey background when the texture is missing.

Marker colour is `time of flight [days]` via the `turbo` colormap, with a colorbar on the right. The same colour encoding is reused on the 3D impact view so a fragment can be recognised across views.

Overlay-safe. No.

Impact lat/lon (Mollweide)

Same data as the equirect view but on matplotlib's `mollweide` (equal-area) projection — the elliptic map that does not exaggerate the polar areas. Backgrounded by a downsampled (720 × 360) **grayscale** copy of the Moon texture; the colour scatter on top reads cleanly against the muted background.

When to use. Whenever the impact field is spread over a wide longitude range — the equirect projection visually stretches the high-latitude bands, whereas Mollweide preserves area.

Overlay-safe. No.

Impact density heatmap

A 2D histogram of impact (lat, lon) in 2.5° cells (144 × 72 bins), rendered as a `pcolormesh` on the Mollweide projection over the same grayscale Moon background. Empty cells are transparent (mask) so the surface texture stays visible where no impacts happened.

The colour scale is the per-cell impact count; the cap is set to `max(hist)` so a debris cloud with a strong hot spot still shows graded intensity across the rest. The title declares the per-cell angular size (e.g. `2.5° x 2.5° bins`).

When to use. Bulk debris cloud impact analysis. With a batch of a few thousand fragments the heatmap reveals clusters and concentration zones at a glance; with a small batch (a few dozen impacts) the individual cells stay visible and the view degrades gracefully.

Overlay-safe. No.

Impact 3D on Moon

3D scene with the textured Moon at the origin and one **30 km solid sphere per impact** placed in PA coordinates. Marker colours follow the same `turbo`-on-time-of-flight encoding as the 2D maps; no in-scene colorbar (VTK does not offer a comfortable equivalent), so the 2D views are the colour legend.

Two **reference-frame triads** are drawn from the centre of the Moon:

- **PA triad** (bright RGB, ~2.1 Moon radii long) with billboard labels `X_pa`, `Y_pa`, `Z_pa`. Since the scene IS the body-fixed PA frame, these axes are identity in scene coordinates: `X_pa` exits the prime meridian (sub-Earth point), `Z_pa` exits the north pole.
- **ICRF triad** (muted RGB, ~1.8 Moon radii long) with labels `X_icrf`, `Y_icrf`, `Z_icrf`. These point where the ICRF basis vectors land in the scene at `et_start_s`, computed by applying `R_icrf_to_pa(et_start)` to `(1,0,0) / (0,1,0) / (0,0,1)`. Useful as a sanity check: rotate the scene until `X_pa` points at the camera and you have the same view as the equirect map centred on the prime meridian.

Interactions are the same as **3D orbit + Moon**: left- drag rotates, scroll zooms, middle-drag pans, `r` resets the camera.

Overlay-safe. No (the view is already an N-marker overlay from a single file).

9.5 Altitude-band plots (events)

When an events file carries `altitude_crossing` triggers on the central body, an **Altitude bands** folder appears in the plot tree. The thresholds, sorted ascending, split the altitude axis into bands (chapter 8, *Info tab → Altitude bands*); these views draw the occupancy reconstructed from the crossing records, plus the `INITIAL_STATE` / `FINAL_STATE` life markers that pin where each object started and when its run ended (chapter 6, *Life markers*). On a file written before those existed the reconstruction falls back to inferring both, and an object that never crossed a threshold is missing from these views entirely. All are high-fidelity only (the reconstruction measures altitude from the central body; the CR3BP synodic frame has no comparable altitude) and colour the bands with a sequential map (dark = low, bright = high) so the visual order matches the physical one. The time axis auto-scales (s / min / h / days) to the analysis window.

Time per band

Horizontal bar per band = time spent in it: total time (with the percentage of the window) for a `propagate` run, or the summed *object-time* over all cases for a batch. Lowest band at the bottom, so the axis reads like an altitude axis.

Overlay-safe. No.

Band occupancy timeline

A single object's occupancy Gantt: one row per band, a coloured bar for every interval the object spends in that band. Reads as "which altitude band, when" at a glance. Per-run events file (one object).

Overlay-safe. No.

Band population over time (batch)

Stacked step-area of how many objects sit in each band over time. The stack height at any instant is the number of objects still alive (not yet impacted or stopped); each coloured layer is that band's instantaneous population, whose time-integral is the Info tab's *population mean*.

On very large logs. The step function is exact as long as its transitions fit a 2000-node budget. Past that — a batch of a few thousand cases crossing five bands produces tens of millions of transitions — the curve is **sampled** on a uniform 2000-node grid and the title says `sampled on N nodes`. Each sampled level is still counted exactly; what a sampled curve cannot show is a transition shorter than one grid step, which at a 400-day window is a few hours. The per-band totals in the Info tab and in the CSV export are unaffected: those are computed from the exact segments, never from this grid.

Overlay-safe. No (single-file aggregate).

Per-case time in band (batch)

Heatmap with one row per case (ascending id) and one column per band, coloured by the time that case spent in the band. The visual companion of the *Altitude bands* CSV export — spot at a glance which cases live low vs high.

Overlay-safe. No (single-file aggregate).

Band snapshot at t

Every other view in this folder is cumulative; this one is a single instant. One horizontal bar per band holding the objects that are in it at time *t*, labelled with the count and its percentage of the population. The companion of *Time per band*: that answers "how much time was spent down there over the whole run", this answers "how many objects are down there right now". A debris cloud can accumulate an enormous time in a shell it has entirely left by day 300, and only the snapshot shows that.

Choosing the instant. *Plot options → Altitude-band snapshot*, field `at [N] days from start`. Confirm with Enter and the view redraws. The field shows the loaded run's length underneath, so the number has a scale. Until you set one, the view says so rather than picking a default: every instant tells a different story, and a silently chosen one would be read as if it had been asked for. An instant past the end of the run is reported as such instead of drawing a chart in which everything has ended.

The `ended` bar. Objects whose propagation had already finished — impact, stop-class trigger, or simply a shorter window — get their own grey bar rather than disappearing. This is what keeps the percentages honest: on a year-long debris run where half the fragments have hit the surface, dropping them would compute every fraction against a denominator that shrinks as the cloud dies. The bars plus `ended` always sum to the propagated population.

Overlay-safe. No (single-file aggregate).

10. Frame conventions

A great many sign and orientation mistakes in orbital work come down to differing frame conventions. SpOdy uses a single small set of frames, all spelt out here.

10.1 The central-body inertial frame

The propagation frame is the **central body's inertial frame**, which the schema spells `central_inertial`. For both supported central bodies (Moon and Earth):

- **Origin** at the centre of the central body.
- **Axes** aligned with the ICRF (International Celestial Reference Frame), which is the modern realisation of the J2000 inertial axes. In practice the X axis points toward the dynamical equinox of J2000, the Z axis aligns with the J2000 mean rotation pole of Earth, and Y completes the right-handed triad.
- **Right-handed.**
- **No rotation** with respect to the distant background of galaxies; the central body orbits, librates / rotates *inside* this frame.

All TOML inputs (`initial_state.position_km`, `velocity_kms`) and all binary outputs are in this frame. The engine does not perform any frame conversions internally beyond the conversions needed to evaluate body-fixed gravity harmonics at each step.

> *If you have a state vector in a different frame (Earth-centred J2000, Moon's body-fixed frame, an ecliptic frame), convert it to the lunar-centred ICRF before pasting it into the TOML. The SPICE toolkit's `spkez` plus `pxform` calls are the canonical way to do this; the SpOdy bundle does not include a SPICE wrapper.*
>
> *One exception is **batch input in a rotating frame**: if your cases CSV is a snapshot of a debris cloud (or any collection of objects whose deltas are measured against a reference satellite's local axes), the GUI rotates the state columns to ICRF automatically at Generate-TOML and emits `cases_file` pointing at the rotated copy. Two rotating frames are supported via the `cases_frame` combo: **RIC** (radial / in-track / cross-track, the typical chaser sensor frame) and **LVLH** (NASA / Goddard nadir-pointing convention, common in breakup models). The TOML carries `cases_frame` and `cases_source_file` as GUI-only round-trip metadata; `spody.exe` ignores them. See chapter 7 ("Rotating-frame batch input (RIC / LVLH)") for the workflow.*

10.2 The body-fixed frames

Internally the engine evaluates the spherical-harmonic gravity expansion in the **central body's body-fixed frame**, which is the frame the coefficient set is expressed in. The rotation from inertial to body-fixed is rebuilt at every step from the right source for the active central body.

10.2.1 Moon: Principal Axes (PA)

For Moon-centred runs the body-fixed frame is the **lunar Principal Axes (PA)** frame, which is the frame the GRGM1200B coefficient set is expressed in. The rotation is built from the lunar libration angles that DE440 carries in its slot 12:

```
C_ICRF→PA = Rz(psi) · Rx(theta) · Rz(phi)
```

with `(phi, theta, psi)` the 3-1-3 Euler angles of the lunar mantle at the requested ET. The convention is `r_PA = C · r_ICRF`.

The PA frame **does become visible** in the Analysis tab's *impact* views (chapter 9): the impact lat/lon map (both projections), the density heatmap, and the 3D impact scene all project IMPACT events from ICRF onto PA so the latitude and longitude axes refer to the actual lunar surface. The same rotation pipeline is reused there, this time exposed in pure Python through the bundled `spopy` package (`spopy.Ephemeris.lunar_libration_angles` + `spopy.icrf_to_moon_pa`), which is a numpy-only re-implementation of the spody-core C helpers (bit-identical, validated at the time of landing).

10.2.2 Earth: International Terrestrial Reference System (ITRS)

For Earth-centred runs the body-fixed frame is the **International Terrestrial Reference System (ITRS)**, the co-rotating Earth-fixed frame realised by the ITRF datums (for practical purposes, ITRF2014 / 2020). The rotation from inertial to body-fixed follows the **IAU 2006/2000A_R06** Earth-orientation conventions plus the IERS Earth-Orientation Parameters (EOP):

```
R_GCRS→ITRS = W(t) · R3(+ERA(t)) · Q(t)
```

where `Q(t)` is the celestial-to-intermediate frame matrix assembled from the IAU 2006 X, Y, s+XY/2 series (the `tab5.2{a,b,d}.txt` files the wizard downloads), `ERA(t)` is the Earth Rotation Angle driven by UT1 (which IERS publishes as `UT1 - UTC` in `finals2000A.all`), and `W(t)` is the polar-motion matrix built from the (`xp`, `yp`) angles in the same EOP file. The chain is exact at the SOFA / ERFA precision floor (sub-mas).

The wizard manages the two raw data sets (`finals2000A.all` plus the IAU 2006 series tables); chapter 3 covers the startup-freshness check that nudges you to refresh `finals2000A.all` after IERS publishes a new bulletin.

The same rotation pipeline is exposed in pure Python through the bundled `spopy` package (`spopy.MappedEOP` for the IERS table reader plus `spopy.icrf_to_itrs(et, eop)` for the rotation itself). The Python implementation wraps `pyerfa` and matches the C engine at machine epsilon.

10.2.3 Common to both bodies

For the **propagation itself** users do not see the body-fixed frame: the inputs and outputs stay in the inertial frame, and the body-fixed conversion happens transparently inside the harmonics evaluation. You do **not** need to worry about libration angles, principal-axis vs mean-Earth axes, sidereal time, or polar motion in your inputs.

The active body-fixed triad is also drawn in the **3D orbit** plot (chapter 9) and animated along with the trajectory: PA libration for the Moon, IAU 2006 + EOP rotation for the Earth, so the textured body and the triad stay locked to a consistent orientation at every animation frame.

10.3 The RIC frame (RIC = Radial / In-track / Cross-track)

This is the frame used by the **diff RIC** plot. It is built **on the fly at every sample** from the state vector of trajectory A:

Axis	Definition
Radial	`r_hat_A = r_A / |r_A|`, positive outward
Cross-track	`c_hat_A = (r_A × v_A) / |r_A × v_A|`, positive along the orbit normal
In-track	`i_hat_A = c_hat × r_hat_A`, completes the right-handed frame

The frame is right-handed; the axes are mutually orthogonal at every sample, but their inertial orientation rotates with the orbit. The frame is also called the **RSW frame** (Vallado's nomenclature) or the **Hill frame** (in the context of the Clohessy-Wiltshire linearised equations).

10.3.1 What goes where

When you decompose a position-error vector `Δr = r_A - r_B` onto this frame:

- A positive **radial** component means trajectory A is higher (farther from the central body) than B at that instant.
- A positive **in-track** component means A is *ahead* of B along the orbit. Persistent growth of `+` in-track is the canonical signal of a small energy / mean-motion drift between the two propagations.
- A positive **cross-track** component means A is offset out of B's instantaneous orbital plane along the orbit normal direction.

10.3.2 Comparison with LVLH

The Local Vertical / Local Horizontal frame (LVLH), used in spacecraft attitude work, is *related to but not identical with* the RIC frame. The signs differ:

	RIC (SpOdy)	LVLH
Z / W	`+r_hat`, outward	`-r_hat`, nadir (toward central body)
X / I	`c_hat × r_hat` (≈ `+v_hat` for circular orbits)	≈ `+v_hat`
Y / C	`+h_hat` (orbit normal)	`-h_hat`

If you need an LVLH decomposition of a **diff plot** you can compute it from the SpOdy diff RIC output by flipping the radial and cross-track signs. SpOdy does not provide an LVLH plot directly because RIC is the conventional choice in conjunction- assessment and orbit-regression work, and a sign-flipped duplicate plot would be just visual noise.

LVLH is **fully supported on the batch-input side**, however: the GUI's `cases_frame` combo accepts `lvlh` and applies the same rotate-to-ICRF-at-Generate pipeline as `ric`, with the NASA / Goddard sign convention (`z = -r_hat`, `y = -h_hat`, `x = y x z`). This is the convention used by the NASA breakup model, by CCSDS conjunction messages, and by the rest of the debris-evolution-tool ecosystem — so a binary export from those tools can be fed into SpOdy without a manual sign flip.

10.4 The synodic rotating frame (CR3BP)

Under `dynamics_model = "cr3bp"` the state lives in the classic **synodic rotating frame**: origin at the barycenter, x-axis along primary 1 → primary 2, z along the orbital angular momentum, rotating at the constant $\omega = \sqrt{(\mu_1+\mu_2)/L^3}$. The primaries sit frozen on the x-axis (primary 1 at $-\mu_2/(\mu_1+\mu_2)\cdot L$, primary 2 at $+\mu_1/(\mu_1+\mu_2)\cdot L$). By convention the synodic axes coincide with the run's inertial axes at t = 0 — the engine and `spopy.inertial_to_synodic` / `synodic_to_inertial` share this t = 0 identity.

10.4.1 The instantaneous pulsating variant (From CR3BP…)

The idealised frame above assumes circular primary motion at the fixed separation L. The **From CR3BP…** dialog (chapter 5), which seeds a *high-fidelity* run from a CR3BP catalog state, uses the **instantaneous pulsating** variant instead: at the requested epoch it reads the *real* primary geometry from the ephemeris — actual separation l^*, axes built from the actual relative position and velocity, angular rate $\omega = h/l^{*2}$ along the actual orbit normal, plus the radial rate $\dot{l}^*$ — and maps the nondimensionalised catalog state through it. Two properties make this the right bridge between the two worlds: both primaries land **exactly** on their DE440 states, and the transform degenerates to the t = 0 identity convention when the real orbit is replaced by the circular idealisation.

Because a CR3BP orbit is periodic only in the CR3BP, the mapped state is a **seed**, not a bound orbit: the full-force propagation shadows the catalog orbit for a while (a 9:2 NRHO holds a few revolutions) and then departs. That divergence is the physics of the ephemeris model, not a conversion error.

10.5 Classical orbital elements

The classical Keplerian elements are reported in the **central- body inertial frame**, with the active central body's gravitational parameter `mu` baked in:

Central body	`mu` (km^3/s^2)	Source
Moon	4902.800066	GRGM1200B
Earth	398600.4415	EIGEN-6C4

Both constants live alongside the central-body radii in `spody_const.h` (single source of truth, shared between the C engine and the Python GUI), and are the same values used by the two-body reference acceleration at every step.

10.5.1 Reference plane

Inclination, RAAN, and the argument of periapsis are reported relative to the **inertial XY plane** (the ICRF XY plane, projected at the central body's centre). This is the ICRF equator, *not* the central body's equator.

- For **Moon-centred** runs the lunar pole is tilted ~5.1° from the ecliptic normal, so the lunar equator is tilted ~24° from the ICRF XY plane. A near-polar lunar orbit (which is "polar" with respect to the Moon's body-fixed pole) appears at `i ≈ 85–115°` in SpOdy's elements, not at exactly 90°, because the reference plane is the ICRF and not the lunar equator.

- For **Earth-centred** runs the difference is much smaller: the J2000 mean equatorial plane *is* (by definition) the ICRF XY plane up to the ~arcsec-level precession / nutation that the rotation pipeline handles separately. A near-polar Earth orbit sits very close to `i = 90°` in SpOdy's elements, drifting only by tens of milliarcseconds per year from precession.

This is the conventional choice and matches the SPICE convention when querying the LRO or GLONASS ephemerides in J2000. If you specifically need elements relative to the central body's equator, post-process the inertial state vectors externally; SpOdy does not currently offer this view.

10.5.2 Quadrant resolution

The right quadrants are picked from the standard sign tests:

- **RAAN**: `Ω` is in `[0, π]` when the Y component of the node line is non-negative, in `(π, 2π)` otherwise.
- **Argument of periapsis**: `ω` is in `[0, π]` when the Z component of the eccentricity vector is non-negative, in `(π, 2π)` otherwise.
- **True anomaly**: `ν` is in `[0, π]` when `r · v ≥ 0` (i.e. moving away from periapsis), in `(π, 2π)` otherwise.

10.6 Sun direction (3D viewer)

The `+ Sun arrow` button in the 3D viewer computes the direction to the Sun **from the central body's centre** at the typed epoch, using a **low-precision analytic model** (Meeus-style series for the solar ecliptic longitude and the obliquity of the ecliptic, sufficient for visualisation accuracy of arc-minute level).

The arrow direction is rendered in the same `central_inertial` frame as the rest of the scene, so it lines up correctly with the orbit polylines. The arrow length is purely visual (scaled to the Moon sphere's radius); it does *not* represent astronomical distance.

For propagation purposes the engine uses the full-precision DE440 ephemeris — the analytic Sun direction is used only for the viewport arrow, never for any force computation.

11. Diff and validation workflow

The diff plots (chapter 9, section *Diff (pick 2 files)*) compare two trajectories sample by sample. This chapter explains how to use them in the two most common scenarios: regression between SpOdy runs, and validation against an external reference such as a SPICE ephemeris. It also documents the time-grid alignment rules and how to read the resulting numbers.

11.1 Two scenarios, same dispatcher

The same six diff plots apply to two distinct workflows:

1. **SpOdy vs SpOdy regression.** You ran a scenario twice with slightly different settings (different harmonics degree, with and without SRP, on different machines, or before and after a code change). The diff tells you how much the two runs disagree.
2. **SpOdy vs reference validation.** You have a reference trajectory in `SPDYOUT_` format produced by another tool — SPICE, a previously-validated engine, or any other source — and you want to know how close SpOdy gets.

The interaction is identical in both cases: load one file by clicking it (call this A), Ctrl-click the second (call this B) in the file tree to multi-select, then click a diff leaf in the plot tree.

11.2 A is the loaded file, B is the other

The two-file selection has a small but consequential rule: the **currently-loaded file is treated as A**, and the other selected file is **B**. Concretely:

- The title and info label show `A = <basename>`, `B = <basename>` so you can always read off which is which.
- The RIC frame is built from A's state. Out-of-plane errors are measured against A's instantaneous orbital plane, not B's.
- Time-grid alignment (next section) is done **B onto A's grid**, so the rendered samples are at A's times.

Swap which is which by clicking the other file in the tree (making it the loaded one); the previous loaded file remains in the selection so the second Ctrl-click is not needed.

11.3 Time-grid alignment

The diff dispatcher checks the two arrays' time columns before subtracting. Two paths:

11.3.1 Fast path: aligned grids

If A and B share the same time grid sample-by-sample (every pair of times matches within 1 ms), both arrays pass through unchanged and the diff is computed directly. This is the case when both runs were produced by SpOdy at the same `output.mode = fixed` with the same `output.interval_s`, or are otherwise known to be on the same fixed grid.

The plot title shows `A = … B = …` without further annotation.

11.3.2 Slow path: cubic-Hermite interpolation

If the grids do not match, B is interpolated onto A's grid restricted to the **overlapping time window**:

- **Position** uses cubic Hermite interpolation, with B's velocity vector as the analytical derivative at each sample. This gives C¹ continuity at sample points and is exact for any polynomial position trajectory up to cubic order.
- **Velocity** uses per-component linear interpolation.

The plot title appends `(B interpolated)` and the info label records the parameters in the form `B interpolated onto A's grid (46053 -> 8640 samples, cubic Hermite on r, linear on v)`.

When the two time windows have no overlap at all (one run started after the other ended), the dispatcher reports `no overlap` cleanly and refuses to render anything.

11.3.3 Why cubic Hermite

For trajectory data sampled at coarse intervals relative to the orbital period, linear interpolation introduces a curvature error of order `(v &Delta;t)^2 / R`. For LRO at 11 s sampling that is about 170 m of interpolation error per sample — much bigger than the m-level diff signal we want to measure.

Cubic Hermite interpolation with `v` as the analytical derivative catches the position curvature exactly to cubic order, reducing the interpolation error to negligible levels for any practical sampling cadence. We use linear interpolation for velocity because velocity is smoother than position (one fewer derivative of the underlying acceleration field), and linear interpolation of v is adequate at the precision the diff plot needs.

11.4 Reading the magnitudes

Concrete numbers for two of the validation scenarios shipped with SpOdy: a Moon-centred Apollo-era propagation and an Earth-centred GNSS broadcast comparison.

11.4.1 LRO 6-day, N=80 harmonics, vs SPICE LRO reference

Quantity	Value
Duration	6 days
Sample count	~8000 (spody) vs ~46000 (SPICE)
`|Δr|` at t=0	0 m exact (shared IC)
`|Δr|` at t=6d	~110 m
`|Δr|` max	~1.2 km at mid-window
`|Δr|` mean	~250 m
`|Δv|` max	~0.8 mm/s

The growth pattern of `|Δr|` is **not monotone**: it accumulates, peaks somewhere in the middle of the propagation, and recovers toward the end. This is characteristic of the gravity-harmonics mismatch between SpOdy's degree-80 expansion and SPICE's reconstructed ephemeris, which uses the full GRGM1200B set.

11.4.2 RIC interpretation for the same diff

On the same example, the RIC decomposition shows the in-track component growing roughly linearly to ~1 km mid-window, while the radial and cross-track components stay below 200 m. This is the canonical signature of a small **mean-motion drift** induced by truncating the harmonics expansion at degree 80 — the two propagations have slightly different effective orbital period because they capture slightly different fractions of the non-spherical potential.

Raising `harmonics_degree` to 150 reduces the in-track drift proportionally; raising it beyond 200 yields diminishing returns because the GRGM1200B coefficients become weakly observed at high degrees and adding terms can slightly increase the residual.

11.4.3 GLONASS R03 7-day, N=70 harmonics, vs IGS broadcast

The Earth-centred counterpart is the GLONASS slot 03 broadcast- nav comparison shipped in `examples/glonass_r03_validation/`. SpOdy propagates from the first 2024-01-21 broadcast TOC for 167.5 hours and is diffed against a reference binary built from 7 consecutive daily RINEX-NAV files (`spody convert glonass <day1.rnx> &hellip; <day7.rnx> &hellip;`; chapter 12). With `srp = false`, `harmonics_degree = 70`, and the Moon + Sun as third bodies:

Day	`|Δr|` RMS	`|Δr|` max
1	176 m	385 m
2	367 m	869 m
3	577 m	1.4 km
4	803 m	2.1 km
5	1026 m	2.7 km
6	1232 m	3.2 km
7	1425 m	3.7 km

The RMS grows roughly linearly at ~200 m / day. The signature is a near-secular in-track residual, the canonical fingerprint of the un-modelled in-track perturbation forces at GNSS altitude (box-wing SRP, antenna thrust, empirical ECOM-style accelerations). Adding a cannonball SRP with guessed Cr / area makes things worse on this particular spacecraft because the in-track sign happens to coincide with the residual; closing the budget to sub-km over a week needs either the proper IGS BOX-WING SRP model (Rodriguez-Solano 2014) or empirical ECOM5 accelerations, both of which sit outside SpOdy's current force-model menu.

Independent of the SRP question, the example is the canonical sanity check that the Earth-orientation pipeline (IAU 2006 + IERS EOP) is wired correctly: day-1 RMS regresses against the broadcast at the 177 m level set by the broadcast nav's own OD precision, which is the floor reachable with a pure force-model

propagation (no per-arc empirical fits).

11.4.4 GPS PRN 11 7-day, N=70 harmonics, vs IGS Final SP3

The GPS counterpart in `examples/gps_g11_validation/` is built on a **cleaner two-format reference scheme** — broadcast nav for the initial state, IGS Final SP3 for the per-sample truth. The broadcast bootstrap gives `(r, v)` at broadcast-OD precision (`~few m / few cm/s`) via the new `spody convert gps` Kepler-with- corrections propagator (IS-GPS-200 + Remondi 2004; chapter 12), replacing the previous 5-point Lagrange forward derivative on SP3 positions that gave the SP3 secant rather than the true Keplerian tangent (`|v0| ~3.57 km/s` vs the correct `~3.87 km/s`, a 7-8% artefact that swamped the residual at `t = 0`).

With `srp = false`, `harmonics_degree = 70`, Moon + Sun third bodies, and the cm-precision IGS Final SP3 as ground truth:

Day	`|Δr|` RMS	`|Δr|` max
1	46 m	91 m
2	128 m	316 m
3	212 m	552 m
4	300 m	800 m
5	390 m	1.1 km
6	484 m	1.3 km
7	581 m	1.6 km

`|Δr|` at `t = 0` is **2.3 m** — the broadcast-vs-SP3-OD floor, NOT a force-model error. The linear ~80 m / day growth is the same in-track signature as GLONASS, but with a 3-4× smaller day-1 baseline because both endpoints of the diff (the IC and the truth) are clean.

11.4.5 Multi-reference comparison: GPS vs GLONASS broadcast OD

The two GNSS examples allow a useful cross-check: diff the same propagation against **both** the broadcast nav reference AND a multi-GNSS SP3 reference (e.g. the CODE MGEX `COD0MGXFIN_*.SP3` files, which include G + R + E + C). Three numbers fall out:

- `prop vs broadcast` — the legacy reference, easy to build but only ~few m of truth precision;
- `prop vs SP3` — the cm-level truth, but introduces a multi-format alignment chore (see *Time-grid alignment* above);
- `broadcast vs SP3` — the truth-floor of the broadcast itself, exposed for free as a side effect of the comparison.

For 2024-01-21:

Constellation	prop vs brdc (d1)	prop vs SP3 (d1)	broadcast-OD floor
GPS G11	47 m	46 m	~2 m
GLONASS R03	176 m	317 m	~258 m

GPS broadcast is ~100× tighter than GLONASS broadcast, which is why a GPS example built on a broadcast-only reference already measures the propagator's force-model error cleanly. For GLONASS the SP3 reference is needed to push past the 258 m broadcast-OD floor — or the force-model residual must be recovered indirectly by composing `prop_vs_brdc &approx; sqrt(prop_vs_SP3^2 - brdc_vs_SP3^2)`.

11.5 Drag validation and ballistic calibration

Validating a drag-enabled propagation (chapter 6, `[spacecraft.drag]`) is structurally different from the gravity and GNSS comparisons above, and deserves its own method. This section documents the process, the calibration method, the numbers measured with it, and the problems you will meet on the way — so you can reproduce the analysis on your own spacecraft.

11.5.1 Why drag cannot be validated "as is"

The drag acceleration is

```
a_drag = -1/2 · rho(model) · v_rel^2 · (Cd·A/m) · v_rel_hat
```

and the density `rho` and the ballistic coefficient `Cd·A/m` enter **only as a product**. A +30% density bias with a correct ballistic coefficient is indistinguishable — at the trajectory level — from a correct density with a +30% ballistic error. Both factors are genuinely uncertain:

- **The density model.** NRLMSISE-00 is an empirical climatology fitted to pre-2000 data. Against modern orbit-derived density references it runs **20–40% hot at 400–500 km around solar maximum** (the thermosphere has cooled since the model's fit epoch). This is a property of the model, not an implementation defect.
- **The ballistic coefficient of record.** Free-molecular `Cd` is not a plate constant: it depends on attitude, surface temperature and gas composition. Operational centres re-estimate it continuously — the NASA/JSC ISS ephemerides changed their own published `DRAG_AREA × DRAG_COEFF` by 10% between two files six days apart.

Any single drag-on vs truth comparison therefore measures the *sum* of the two effects. The method below separates them.

11.5.2 Process: choosing reference data

Two public data families make clean drag benchmarks:

1. **NASA/JSC ISS ephemerides** (CCSDS OEM, EME2000, published ~weekly with an archive of dated snapshots). Each file carries mass, drag area and Cd in its `COMMENT` block plus a trajectory event summary. Pick a **pair of consecutive files** bracketing a 3–7 day gap with **no manoeuvre in the event summary**: the older file's first state is a tracking-fresh initial condition, the newer file's first day

is a tracking-fresh truth. Never validate against the *forecast* portion of a single file — that measures the publisher's space-weather forecast, not your propagator (see *Problems* below).

2. **ESA Swarm Level-2 precise orbits** (`SP3ACOM` reduced-dynamic POD: SP3 format, ITRF, GPS time, cm-level truth), with the companion `DNSAPOD` product giving the POD-derived thermospheric density along the same orbit. The density product enables a **density-space comparison that needs no mass or area at all**.

In both cases prefer a geomagnetically quiet window (daily Ap below ~15 in the CelesTrak table) so the drag signal is EUV-driven and the model's storm response stays out of the budget.

11.5.3 Method: the single-scale ballistic fit

Introduce one free factor `k` that scales the product: `a_drag(k) = k · a_drag(nominal)`. Because drag is a small perturbation, the in-track displacement it accumulates is linear in `k` to first order, so **two propagations are enough**: with `I_off(t)` and `I_on(t)` the in-track residuals of the drag-off and drag-on (k = 1) runs against the truth,

```
I(k, t) ≈ I_off(t) + k · [I_on(t) − I_off(t)]
```

and the least-squares scale over the scored epochs is

```
k* = Σ_t [ −I_off(t) · ΔI(t) ] / Σ_t [ ΔI(t)² ],   ΔI = I_on − I_off
```

Then **verify the linearity by actually re-running** the propagation with `Cd` multiplied by `k*`: if the chain is healthy the in-track residual collapses by two orders of magnitude. For an honest accuracy claim, fit `k*` on a **calibration arc** (the first 1–2 days) and quote the residual growth on the remaining **hold-out days**, which the fit has never seen.

The fitted `k*` is not a fudge: it is exactly the ballistic re-estimation every operational OD performs, and it cleanly splits the error budget into a *constant* part (density-model bias × ballistic uncertainty, absorbed by `k*`) and a *time-varying* part (day-to-day thermospheric variability the climatology cannot capture, visible in the hold-out drift).

11.5.4 Results (July 2024 window, Ap 2–10, F10.7a ≈ 205)

ISS, 5-day gap between the 2024-07-03 and 2024-07-09 JSC ephemerides, truth = first (pre-creation) day of the newer file:

Run	In-track @ 5 d
Drag off	−146 km
Drag on, BC of record (k = 1)	+53 km
Drag on, fitted k* = 0.725	973 m RMS, 1.8 km max
JSC's own 5-day forecast (yardstick)	1.3 km RMS, 2.1 km max

The fitted run lands in the same class as the publisher's own forecast — which is produced with internal tracking and attitude timelines. Radial and cross-track stay at tens of metres throughout.

Swarm-A, 5.5 days against the cm-level `SP3ACOM` POD, `k*` fitted on the first two days only (Cd 2.6 → 3.29 at the nominal area/mass), per-day in-track RMS:

Day	In-track RMS	Arc
0	51 m	calibration
1	37 m	calibration
2	120 m	hold-out
3	81 m	hold-out
4	453 m	hold-out
5	997 m	hold-out

The scale factor is **stable to 0.6%** between the 2-day calibration fit and a full-window fit, and the drag-off drift is −108 km over the same window: with one calibrated number the chain holds roughly **200 m/day in-track** against a −20 km/day signal. Radial ≤ 25 m and cross-track ≤ 17 m RMS everywhere.

The independent density-space comparison (NRLMSISE-00 evaluated along the Swarm orbit at the `DNSAPOD` epochs, same space-weather inputs the engine uses) gives the model bias directly: **median +38%** at 470–500 km over the window — and rising from +28% to +51% day by day, which is exactly the time-varying part that surfaces as the hold-out drift above. The three numbers close on each other: `k* × BC_nominal × 1.38` reproduces the physical `Cd·A` expected for the spacecraft geometry.

11.5.5 Problems and solutions

- **Forecast references flatter or damn you unpredictably.** A drag validation first attempted against the forecast portion of a single ISS ephemeris "failed" by +7.7 km in two days — entirely explained by an F10.7 spike and a geomagnetic storm that post-dated the file's creation. *Solution: anchor both ends of the arc on tracking-fresh states from an archive of dated files, in a historical window where all space weather is observed (`OBS` rows in `SW-All.csv`), never predicted.*
- **The density/BC degeneracy.** No trajectory comparison can separate them. *Solution: calibrate the product with `k*` and, when a POD-derived density product exists (Swarm), measure the density bias independently in density space; the ballistic part is whatever remains.*
- **The bias is not constant.** A single `k*` removes the mean bias but the residual climatology error (tens of percent over days) sets the hold-out floor — roughly 1 km at 5 days in a quiet window at ~480 km. *Solution: a time-varying node table. SpOdy applies the calibration natively through `force_model.density_scale` (constant `k*`) or `density_scale_file` (piecewise-linear `k(t)` nodes, chapter 6), so the fitted factor goes where it belongs instead of misdeclaring the physical `Cd`; and the whole fit described in this section — window partition, drag on/off arc pairs, in-track least squares — is automated by the `spody calibrate` subcommand (chapter 12), which emits the node file directly. The remaining ceiling — sub-daily density structure no multiplicative correction can capture — would need storm-time indices (JB2008-class models) or assimilative corrections.*
- **Manoeuvres poison the arc silently.** A reboost inside the gap turns the whole comparison into noise. *Solution: read the event summary in the OEM comments before choosing the pair; for spacecraft without event metadata, a kink in the fitted-run in-track residual is the diagnostic.*

The same calibration logic applies verbatim to solar radiation pressure: `Cr·A/m` is one more product of an uncertain optical coefficient and an effective area, and operational ODs estimate a `Cr` scale exactly like a ballistic factor. At drag altitudes (400–500 km) SRP is only ~5% of drag and its calibration error is second-order; above ~800 km the roles reverse and SRP becomes the term worth calibrating (see the GNSS sections above, where un-modelled SRP dominates the 7-day budget).

11.6 Combining diff plots through the tile dashboard

A useful pattern for the validation workflow is to tile the four diff plots into a 2×2 dashboard for a single overview shot:

1. Click A (your run) in the file tree to load it.
2. Ctrl-click B (your reference) so both are selected.
3. In the plot tree, Ctrl-click each of `|Δr| (log y)`, `|Δv| (log y)`, `Δx, Δy, Δz per component`, and `RIC frame`.
4. The `▦ Tile selected (4)` button at the bottom of the plot tree lights up. Click it.

The canvas splits into a 2×2 grid showing all four diffs at once. The shared subtitle reports A, B, and the `(B interpolated)` annotation if applicable.

11.7 Validating against an external reference

The recommended workflow when you have an external reference trajectory in `SPDYOUT_` format:

1. Place the reference file in the same folder as your TOML (or in any folder of your choice).
2. Open the TOML in the Run tab and run the propagation.
3. Switch to the Analysis tab. The working directory is already pointed at your TOML's parent.
4. Click your reference file in the file tree to load it.
5. Ctrl-click the SpOdy output file you just produced.
6. Click `RIC frame` in the plot tree.

If your reference file is not yet in `SPDYOUT_` format (e.g. you have it as a CSV of (t, r, v) rows, or as a SPICE BSP kernel), you need to convert it first — SpOdy's reader path accepts only the binary format. The format is documented in chapter 12, section *Output binary formats*; producing a converter for it is typically a 30-line numpy script.

11.8 Validating against another SpOdy run

The simplest regression scenario: run the same TOML twice and diff. The two runs will be byte-identical unless something has changed between them (the engine is deterministic for a given input). A non-zero diff therefore unambiguously points at the change between the two runs — a different harmonics degree, a recompiled engine, a different machine.

This is also the cheapest way to check that a code change you just made does not silently affect the physics: run before, run after, diff. The diff plot showing flat zero across all four panels is the convincing assertion that nothing changed.

12. Command-line reference

The engine `spody.exe` is a standalone command-line tool. The GUI drives it through subprocess calls, but you can drive it yourself — from a PowerShell prompt, from a Python script, from a CI pipeline. This chapter documents the subcommands and their flags.

The conventions throughout the chapter:

- All path arguments are resolved relative to the **current working directory** of the shell that launches `spody.exe`, *except* for the paths inside a TOML, which are resolved relative to the **TOML's directory**.
- The engine returns **exit code 0** on success and **non-zero** on any failure (parse error, validation error, runtime error). A human-readable error message is printed to stderr before exit.

12.1 Global structure

```
spody.exe <command> [arguments]
```

The six commands are:

Command	Purpose
`validate`	parse + sanity-check an input TOML, no run
`propagate`	run a single-case simulation
`batch`	run a multi-case sweep
`convert`	data-file conversions (ephemeris, harmonics, SP3, GLONASS, GPS, OEM)
`calibrate`	fit the drag density-scale `k(t)` nodes against a reference
`info`	print version + build info

Pass `--help` or no command to print a short usage summary.

12.2 `spody validate`

```
spody.exe validate <input.toml>
```

Loads the TOML, runs the engine's parser and the validator, and prints either:

- a multi-line summary of the parsed configuration starting with `OK`, on success;
- a single-line `error: <file>:<reason>` message on failure.

The validator runs the same checks the **Validate** button in the GUI triggers, so the verdicts agree by construction. Useful in scripts to gate downstream work on a clean input.

Exit codes.

- `0` — the TOML is well-formed and internally consistent.
- `1` — parse error, validation error, or unreadable file.

12.3 `spody propagate`

```
spody.exe propagate <input.toml> [--out <dir>]
```

Reads a TOML describing a single scenario, integrates the trajectory, and writes the output files specified in the `[output]` block.

Flags:

- `--out <dir>` (optional) — override the `output.*_file` paths' parent directory. Useful when you want to dispatch the same TOML against many output folders without editing the file.

Stdout/stderr. The engine prints a header line and per-step diagnostics to stdout, with errors on stderr. The GUI streams this into the terminal pane; on the command line you see it in your shell.

CR3BP parameter block. Under `dynamics_model = "cr3bp"` the opening block gains the three lines below (`spody batch` and `spody validate` print them too). The input TOML names the primary pair and nothing else — separation and GM values come from the engine's tables — so this is what pins down the model a given log belongs to:

```
cr3bp      : Earth + Moon  (L = 384400 km)
cr3bp GM   : mu1 = 398600.4415, mu2 = 4902.8005821478 km^3/s^2
cr3bp mu   : 0.0121505853505625  (omega = 2.66531440062302e-06 rad/s)
```

`mu = mu2 / (mu1 + mu2)` and `omega = sqrt((mu1 + mu2) / L^3)` are derived from the three resolved values, printed at 15 significant digits. Enable `output.log_file` to keep them next to the results; the form additionally writes the same values as a comment under `[cr3bp]` in the TOML it saves (ch. 6).

Cost report. The run ends with two lines: the time it took, and what the integrator actually did.

```
done in 6.394 s (final state at t=603900 s)
integrator: 11232 accepted steps, 0 rejected, 78624 RHS evaluations
```

The timing brackets the integration only — parsing, ephemeris loading and harmonics loading all happen before the clock starts, so a 250 MB gravity file does not inflate the figure.

The three counters measure the work rather than the machine, which is what makes them comparable between runs on different hardware, and between a build with and without optimisations. `RHS evaluations` counts every evaluation of the dynamics, including those spent on trial steps that were later rejected: that work was really done. `rejected` climbing into the same order of magnitude as `accepted` is the signature of a step-size controller fighting the problem — usually too tight a `rel_tol`, an `h_max_s` that lets the step grow past what the dynamics allow, or a close approach.

Reading the counters against the accuracy you get is the cheapest way to find out whether a tighter tolerance is buying anything. Past the point where the force model's own error dominates, more evaluations buy nothing at all.

Exit codes.

- `0` — the propagation reached `duration_s` without an unrecoverable error.
- `1` — parse error, validation error, integrator failure (step size driven below `h_min_s`), or I/O error.

12.4 `spody batch`

```
spody.exe batch <input.toml> [--out <dir>]
```

Reads a TOML with a `[batch]` block and runs the parameter sweep described by `batch.cases_file`. The TOML must contain a `[batch]` section; running `propagate` against a batch TOML is an error, and vice versa.

Flags:

- `--out <dir>` (optional) — override `batch.output_dir`.

Threading. Reads `batch.thread_number` and dispatches that many concurrent cases through OpenMP. Use `1` for sequential execution; the engine handles any value up to the host's logical- CPU count.

Exit codes.

- `0` — every case completed successfully.
- `1` — one or more cases failed; the engine prints a summary line `batch stopped at case N/M after T s` and exits.

12.5 `spody convert`

The umbrella command for data-file conversions. Seven sub-commands are implemented today: planetary ephemeris (`ephemeris`), ICGEM spherical-harmonic gravity coefficients (`harmonics_icgem`), IGS SP3 precise orbits (`sp3`), RINEX-NAV GLONASS broadcast (`glonass`), RINEX-NAV GPS broadcast (`gps`), CCSDS OEM text ephemerides (`oem`), and general-perturbation mean elements (`gp`). The first two are the conversions the setup wizard triggers automatically; the four after them produce reference binaries used in the diff-validation workflow (chapter 11); `gp` is the odd one out and writes no file at all — it prints the initial state a propagation starts from.

12.5.1 `spody convert ephemeris`

```
spody.exe convert ephemeris <folder> <de_family> <date1> [date2 ...]
```

Converts the JPL ASCII DE-family chunks present in `<folder>` into the internal `.spody` binary format SpOdy uses for ephemeris queries.

Arguments:

- `<folder>` — directory containing the `header.<de_family>` file and the `ascpXXXXX.<de_family>` chunks. The output `de<de_family>.spody` is written into the **same folder**.
- `<de_family>` — the DE-family identifier (`440` for DE440, the only supported value today).
- `<date1> [date2 ...]` — one or more chunk identifiers (without the `ascp` prefix and the `.<de_family>` suffix). Order does not matter; the converter sorts them internally.

This is the same conversion the setup wizard runs automatically on first launch (chapter 3, section 3.5). You typically invoke it manually only when you have downloaded additional chunks outside the wizard or want to regenerate `de440.spody` for any other reason.

Example.

```
spody.exe convert ephemeris .\data\DE440 440 01950 02050
```

Reads `data\DE440\header.440`, `data\DE440\ascp01950.440`, and `data\DE440\ascp02050.440`. Writes `data\DE440\de440.spody`.

Exit codes.

- `0` — conversion succeeded.
- `1` — missing file, unreadable folder, or library failure.

12.5.2 `spody convert harmonics_icgem`

```
spody.exe convert harmonics_icgem <input.gfc> <output.tab> [--max-degree N]
```

Converts an ICGEM-format spherical-harmonic gravity coefficients file (the `.gfc` format published by GFZ Potsdam for EIGEN-6C4, EGM2008, and most modern Earth gravity models) into the GRGM-style `.tab` format the engine reads at run time.

Arguments:

- `<input.gfc>` — the ICGEM source file.
- `<output.tab>` — destination for the GRGM-style table.
- `--max-degree N` (optional) — truncate the output at degree N. Default is the full degree declared in the source file. Useful when you want a slimmer file for a known altitude regime (e.g. EIGEN-6C4 truncated to 250 is ~3 MB vs the full 252 MB).

This is the second conversion the setup wizard runs automatically when the Earth gravity-model raw file is downloaded (chapter 3). Manually invoking it is useful when you want a custom-truncated table or you have downloaded a different ICGEM model.

Example.

```
spody.exe convert harmonics_icgem .\data\EIGEN-6C4\EIGEN-6C4.gfc `
                                  .\data\EIGEN-6C4\eigen-6c4.tab
```

Exit codes.

- `0` — conversion succeeded.
- `1` — missing file, parse error, or write failure.

12.5.3 `spody convert sp3`

```
spody.exe convert sp3 <input.sp3> [input.sp3 ...] <output.bin> <sat_id> --eop <file> --iau2006-dir
```

Converts one satellite's track from one or more IGS SP3 precise- orbit files into a SpOdy `SPDYOUT_` reference binary in the central-body inertial frame. SP3 records carry ITRS position only (no velocity); the converter applies `R_ITRS→ICRF(t)` per record using the EOP + IAU 2006 tables, writes position with zero velocity, and emits one record per SP3 epoch.

Multi-file mode: IGS final products ship one SP3 file per UTC day, so a week-long cm-precision reference is 7 daily files passed in chronological order. The converter concatenates them into one binary with a continuous 0-anchored time axis; single-file calls behave bit-for-bit identically to before.

Arguments:

- `<input.sp3> [input.sp3 ...]` — one or more IGS SP3-c / SP3-d files in chronological order. For the GPS-only IGS Final products (`IGS0OPSFIN_*_ORB.SP3` on BKG) only `G<NN>` ids are resolvable; for multi-GNSS reference (G + R + E + C + J) use a multi-GNSS analysis-centre product such as CODE's `COD0MGXFIN_*_ORB.SP3` (mirrored at `ftp.aiub.unibe.ch/CODE_MGEX/`).
- `<output.bin>` — destination `SPDYOUT_` binary (always the *second-to-last* positional argument).
- `<sat_id>` — 3-character SP3 satellite id (`G11`, `R03`, `E14`, ...); always the *last* positional argument.
- `--eop <file>` — path to `finals2000A.all`.
- `--iau2006-dir <dir>` — path to the directory containing `tab5.2{a,b,d}.txt`.

The time column of the emitted binary is **0-anchored** at the first record across all inputs (`t_record - t_first_overall`), matching the propagator's convention so a diff against a propagation lines up sample-by-sample at `t = 0`.

Example. Used by the bundled `gps_g11_validation` example to build the 7-day SP3 reference:

```
spody.exe convert sp3 .\examples\gps_g11_validation\IGS0OPSFIN_20240210000_01D_15M_ORB.SP3 `
                      .\examples\gps_g11_validation\IGS0OPSFIN_20240220000_01D_15M_ORB.SP3 `
                      ... `
                      .\examples\gps_g11_validation\IGS0OPSFIN_20240270000_01D_15M_ORB.SP3 `
                      .\examples\gps_g11_validation\gps_g11_2024_01_21_7d_sp3_ref.bin `
                      G11 `
                      --eop .\data\eop\finals2000A.all `
                      --iau2006-dir .\data\iau2006
```

12.5.4 `spody convert glonass`

```
spody.exe convert glonass <input.rnx> [input.rnx ...] <output.bin> <sat_id> --eop <file> --iau2006
```

Converts one GLONASS satellite's broadcast nav track from one or more RINEX-NAV files into a SpOdy `SPDYOUT_` reference binary in the Earth-centred ICRF frame. Unlike SP3, GLONASS broadcast nav carries position **and** velocity in PZ-90 every ~30 min, so the resulting binary has true `(r, v)` per record (no finite-difference velocity needed).

The converter accepts **multiple input files** as positional arguments — the workhorse pattern for week-long-or-more validations, since IGS-BKG ships GLONASS broadcast as one RINEX-NAV file per UTC day. Pass the files in chronological order; the converter walks them in sequence, scans each for the requested slot, and appends `SPDYOUT_` records to a single binary with a continuous 0-anchored time axis (no gap at day boundaries). Calling with a single input reproduces the single- file behaviour bit-for-bit.

Arguments:

- `<input.rnx> [input.rnx ...]` — one or more RINEX 3.x GLONASS-or-mixed nav files in chronological order.
- `<output.bin>` — destination `SPDYOUT_` binary (always the *second-to-last* positional argument).
- `<sat_id>` — the 3-character GLONASS slot id, e.g. `R03` (always the *last* positional argument).
- `--eop <file>` — path to `finals2000A.all`.
- `--iau2006-dir <dir>` — path to the directory containing `tab5.2{a,b,d}.txt`.

Example. Used by the bundled `glonass_r03_validation` example to build the 7-day reference binary from 7 daily RINEX files:

```
spody.exe convert glonass `
    .\examples\glonass_r03_validation\BRDC00WRD_R_20240210000_01D_RN.rnx `
    .\examples\glonass_r03_validation\BRDC00WRD_R_20240220000_01D_RN.rnx `
    .\examples\glonass_r03_validation\BRDC00WRD_R_20240230000_01D_RN.rnx `
    .\examples\glonass_r03_validation\BRDC00WRD_R_20240240000_01D_RN.rnx `
    .\examples\glonass_r03_validation\BRDC00WRD_R_20240250000_01D_RN.rnx `
    .\examples\glonass_r03_validation\BRDC00WRD_R_20240260000_01D_RN.rnx `
    .\examples\glonass_r03_validation\BRDC00WRD_R_20240270000_01D_RN.rnx `
    .\examples\glonass_r03_validation\glonass_r03_2024_01_21_7d_ref.bin `
    R03 `
    --eop .\data\eop\finals2000A.all `
    --iau2006-dir .\data\iau2006
```

Each file contributes its own per-file summary line on stderr (`-> 48 records (sat=R03, &hellip;)`), followed by an aggregate line covering the whole 7-day track.

12.5.5 `spody convert gps`

```
spody.exe convert gps <input.rnx> [input.rnx ...] <output.bin> <sat_id> --eop <file> --iau2006-dir
```

Converts one GPS satellite's broadcast nav track from one or more RINEX-NAV files into a SpOdy `SPDYOUT_` reference binary in the Earth-centred ICRF frame. Unlike GLONASS broadcast (which carries `(r, v, a_lunisolar)` directly in PZ-90), GPS broadcast carries a *Kepler-with-corrections* element set per record (`sqrt_A, e, i0, Omega0, omega, M0` + `Delta_n, OmegaDot, iDot` + harmonic corrections `Cuc/Cus/Crc/Crs/Cic/Cis`). The converter propagates each record to its own TOC per IS-GPS-200 sect. 20.3.3.4.3 (positions) + Remondi 2004 (analytic velocity derivatives), giving `(r, v)` at broadcast-OD precision (`~few m / few cm/s`).

This is the recommended **initial-state bootstrap** for any SP3-based GPS validation: replaces the 4th-order Lagrange forward derivative of 5 SP3 positions, which gave the SP3 secant rather than the true Keplerian tangent (`|v0|` came out 7-8% off, swamping the propagation residual at `t = 0`). With broadcast IC the day-1 RMS measures force-model error, not bootstrap artefact.

Like `convert glonass` and `convert sp3`, multi-file inputs are concatenated into one binary with a continuous 0-anchored time axis.

Arguments:

- `<input.rnx> [input.rnx ...]` — one or more RINEX 3.x GPS-or-mixed nav files in chronological order (BKG hosts a GPS-only `_GN.rnx` variant; the multi-GNSS `_MN.rnx` file also works).
- `<output.bin>` — destination `SPDYOUT_` binary (always the *second-to-last* positional argument).

- `<sat_id>` — the 3-character GPS PRN, e.g. `G11` (always the *last* positional argument).
- `--eop <file>` — path to `finals2000A.all`.
- `--iau2006-dir <dir>` — path to the directory containing `tab5.2{a,b,d}.txt`.

Example. Used by the bundled `gps_g11_validation` example to build the 7-day broadcast IC file:

```
spody.exe convert gps `
    .\examples\gps_g11_validation\BRDC00WRD_R_20240210000_01D_GN.rnx `
    ... `
    .\examples\gps_g11_validation\BRDC00WRD_R_20240270000_01D_GN.rnx `
    .\examples\gps_g11_validation\gps_g11_2024_01_21_7d_brdc.bin `
    G11 `
    --eop .\data\eop\finals2000A.all `
    --iau2006-dir .\data\iau2006
```

Each file contributes its own per-file summary line on stderr, followed by an aggregate line covering the whole track.

12.5.6 `spody convert oem`

```
spody.exe convert oem <input.oem> [input.oem ...] <output.bin>
```

Converts one or more CCSDS OEM (Orbit Ephemeris Message) text files — the format used by the NASA/JSC public ISS trajectory ephemerides, among many others — into a SpOdy `SPDYOUT_` reference binary with full `(r, v)` states. This is the reference of choice for `spody calibrate` on LEO spacecraft: operator OEMs carry tracking-fresh states plus, usually, mass / drag-area / event-summary comments.

Supported subset (anything else is rejected with a message naming the offending line):

- `REF_FRAME` — `ICRF`, `EME2000` or `J2000`. Per the SPICE convention J2000 ≡ ICRF, so no rotation (and no EOP data) is involved.
- `TIME_SYSTEM` — `UTC` (full leap-second chain + TDB periodic term) or `TDB`.
- Epochs in calendar ISO form `YYYY-MM-DDThh:mm:ss[.sss]` (the day-of-year OEM variant is not supported).
- Ephemeris rows `epoch x y z vx vy vz` in km / km/s; optional trailing acceleration columns are ignored, `COMMENT` lines and covariance blocks are skipped, multi-segment files (several `META_START` blocks) are fine.

Multi-file inputs are concatenated into one binary with a continuous 0-anchored time axis; pass them in **chronological order**. Consecutive operator releases overlap in span, so any record that does not advance past the last written epoch is dropped (**first file wins**) and counted in the summary line. To calibrate against the freshest prediction of each day, convert the files one at a time instead.

Example. The ISS reference used in chapter 11's bench:

```
spody.exe convert oem .\ISS_OEM_2024-07-03.txt .\iss_ref.bin
```

The stderr summary reports the record count, the ET span in hours and the number of skipped overlapping records.

Exit codes (sp3, glonass, gps, oem).

- `0` — conversion succeeded (or wrote zero records with a WARNING when the requested `<sat_id>` was absent across all inputs).
- `1` — missing file, parse error, frame-rotation setup failure, or write failure.

12.5.7 `spody convert gp`

```
spody.exe convert gp --epoch-mjd <utc> --n <rev/day> --ecc <-> --incl <deg>
                     --raan <deg> --argp <deg> --ma <deg> --bstar <1/ER>
                     --eop <file> --iau2006-dir <dir>
```

Takes one general-perturbation element set — the eight mean elements a TLE or an OMM carries — and prints the ICRF state at its epoch, in the shape of the `[initial_state]` block it is about to become.

This is the only `convert` sub-command that writes no file. It behaves like `help`: everything goes to stdout, so it can be redirected, piped, or read and pasted.

Why it is needed at all. The elements in a TLE are *mean* elements of the SGP4 theory, not a state. Handing them to a numerical integrator is a physical error rather than an approximation: the theory has to be run first, and it answers in TEME, which nothing else in SpOdy speaks. This command runs it and rotates the answer.

What the answer is worth. The rotation is exact to centimetres and the propagator matches the published reference vectors bit for bit, and neither of those is the accuracy of the state. A GP element set is a fit, worth kilometres, and no amount of arithmetic downstream makes it better. The seventeen digits printed are what it costs to move a number through a text file unchanged, not a claim about how well the satellite is known. The output says so in a comment header, on purpose.

Why every element is a named flag. Eight bare numbers in a row is a transposition waiting to happen, and two of them — argument of perigee and mean anomaly — carry the same units and the same order of magnitude. Swapping those two raises no error anywhere: it produces a healthy-looking orbit that is not the one asked for.

Example.

```
spody.exe convert gp --epoch-mjd 53841.745032470 --n 15.72125391 `
                     --ecc 0.0006703 --incl 51.6416 --raan 247.4627 `
                     --argp 130.5360 --ma 325.0288 --bstar 1.0e-4 `
                     --eop .\data\eop\finals2000A.all `
                     --iau2006-dir .\data\iau2006
```

```
# SGP4 at the element-set epoch, rotated TEME -> ICRF.
# What this state is worth is what a general-perturbation
# fit is worth -- kilometres -- and not what the digits
# suggest. They are here so the value survives being
# pasted, which is a different thing from being accurate.
et_start_s   = 198482035.99104285

[initial_state]
frame        = "icrf"
position_km  = [4085.6826275667108, -999.13905635608523, 5241.1698975079889]
velocity_kms = [2.5226906524348545, 7.2563208512806634, -0.58563608337435047]
```

Exit codes.

- `0` — conversion succeeded.
- `1` — an element or a table path is missing, an unknown flag was given, the element set was rejected by SGP4, or the EOP / IAU 2006 tables could not be read.

12.6 `spody calibrate`

```
spody.exe calibrate <input.toml> <reference.bin> [--window <hours>]
```

Fits the time-varying density-scale table `k(t)` of chapter 11's *Drag validation and ballistic calibration* section — entirely inside the engine — and writes it in exactly the format `[force_model].density_scale_file` consumes (chapter 6). One command closes the calibration loop: convert a reference, run `calibrate`, point the TOML at the emitted `k_nodes.csv`. The GUI form wraps the same command behind the **Calibrate…** button next to `density_scale_file` (chapter 5), streaming this report into the Run-tab console and auto-filling the path on success. The bundled `examples/iss_drag_calibration/` scenario ships the NASA/JSC ISS OEM plus its converted reference, ready for the whole workflow.

Requirements, checked up front:

- the TOML is a valid single-scenario input with `force_model.drag = true` (the whole drag stack — `[spacecraft.drag]`, `space_weather_file` — must be configured; a density scale is unobservable otherwise);
- `[simulation].et_start_s` equals the epoch of the reference's first record — the same 0-anchored-time contract as the Analysis-tab diff workflow;
- the reference is a `SPDYOUT_` binary with **full states** (`convert gps`, `convert glonass` or `convert oem`). SP3-derived binaries carry zero velocities and are rejected: each window re-anchors the propagation on a reference state.

Any `density_scale` / `density_scale_file` already in the TOML is ignored (with a warning): the fit must price the raw, uncalibrated model. The TOML's `[initial_state]` and `duration_s` are ignored too — the calibration span is the reference span, and every window anchors on the reference itself.

Method. The reference span is cut into windows of `--window` hours (default 24; at least 6 reference samples per window, a runt tail is folded into the last window). Per window the engine:

1. anchors the initial state on the reference record at the window start;
2. propagates the window twice — drag **off** and drag **on** at `k = 1`;
3. resamples both arcs onto the reference epochs (cubic Hermite between accepted steps) and projects the position residuals on the **in-track** axis of the reference's RIC triad;
4. solves the closed-form least squares `k* = -sum(I_off * dI) / sum(dI^2)` with `dI = I_on - I_off` (in-track drift is linear in the density scale — chapter 11 derives why);
5. emits one node `(mjd_utc_of_window_centre, k*)`.

Windows with no usable fit are **skipped with a warning** rather than failing the run: a drag signal below 1 mm rms in-track (widen `--window`), or a non-positive fitted `k` (typically a manoeuvre inside the window). The piecewise-linear evaluator bridges the gap between the surviving nodes; if *no* window survives, the command fails.

Outputs. A fresh timestamped run folder under the TOML's `output_dir` (self-contained, like every run): the snapshot TOML, the per-window `cal_wNNN_off.bin` / `cal_wNNN_on.bin` arcs (kept for inspection — they are ordinary trajectory binaries the Analysis tab can open), and `<ts>_k_nodes.csv`. The report goes to stdout, one line per window:

```
  [ 1/ 3] t=    0.00..   24.00 h  n=  360  k=0.7931 +/- 0.0006  in-track rms 2685.1 -> 36.7 m
```

`n` is the number of fitted reference samples, `+/-` the 1-sigma of the fit, and the two rms values are the raw (`k = 1`) and post-fit in-track residuals over the window. The footer reports the node file path and the **pooled k** (the constant-scale equivalent over the whole span, i.e. what you would put in `force_model.density_scale` if a single number is enough).

To use the result, copy (or reference) the node file from the scenario and set:

```toml
[force_model]
density_scale_file = "k_nodes.csv"
```

Mind the node span: nodes sit at window *centres*, so a propagation covering the full reference span extends half a window past the first/last node on each side — the engine holds the end values there and prints the chapter-6 clamp warning. That is by design and harmless.

Cost. Two short propagations per window — a 3-day ISS reference with 24 h windows (6 day-arcs) fits in ~35 s.

Exit codes.

- `0` — at least one window produced a node and the file was written.
- `1` — parse/validation failure, reference format error, a propagation failure inside a window, or every window skipped.

12.7 `spody info`

```
spody.exe info
```

Prints the SpOdy application version, the engine library version, the engine library's git commit hash, and the build timestamp.

No flags; no error path.

12.8 Examples

Quickly validating every example TOML at once on PowerShell:

```powershell
Get-ChildItem .\examples -Recurse -Filter input.toml | ForEach-Object {
    .\spody.exe validate $_.FullName
}
```

Re-running the LRO 6-day example in a fresh output folder:

```
.\spody.exe propagate .\examples\lro_6day\input.toml --out C:\Temp\lro_run
```

Dispatching a batch across all logical CPUs from a shell:

```
# Set thread_number = N inside the TOML, then:
.\spody.exe batch .\examples\batch_demo\input.toml
```

12.9 Output binary formats

For completeness, the three binary formats SpOdy writes are documented here so external tools (Python notebooks, CI pipelines) can read them without going through the GUI.

Every binary starts with the same **24-byte header**:

Offset	Bytes	Content
0	8	ASCII magic (no NUL terminator)
8	4	uint32 little-endian version
12	4	uint32 little-endian payload metadata
16	8	reserved (two zeroed uint32)

The three magics are `SPDYOUT_` for trajectories, `SPDYACC_` for the per-force accelerations breakdown, and `SPDYEVT_` for events. The payload metadata is interpreted per format:

12.9.1 `SPDYOUT_` — trajectory

- Payload: `state_dim` (always `6` today, meaning `r, v`).
- Record: 7 little-endian doubles in order `t, x, y, z, vx, vy, vz`.
- Record size: 56 bytes.

12.9.2 `SPDYACC_` — per-force accelerations

- Payload: record size in bytes (varies with the number of active forces).
- Record: a header double `t`, followed by per-force triples `ax, ay, az` for each active force in the order documented in the engine's source. The total `a_total` triple and the `eclipse_fraction` scalar are included.

12.9.3 `SPDYEVT_` — events

- Payload: record size in bytes (80).
- Record: an `EVENT_KIND` uint32, a flags uint32, the event time as a double, and a 64-byte body whose layout depends on the kind (IMPACT, ECLIPSE_ENTER, ECLIPSE_EXIT).

A NumPy-based Python reader for these formats is the standard companion to the GUI and lives in the `spody_io` package distributed alongside the GUI bundle. Importing it from a script gives you `read_trajectory`, `read_accelerations`, and `read_events` functions that return structured `ndarray`s.

13. Troubleshooting

This chapter is a catalogue of the issues you are most likely to hit, grouped by the part of the workflow they belong to, with the recommended fix for each. If you encounter something not listed here please file a report with the steps that reproduce it; the catalogue grows with experience.

13.1 Setup wizard

13.1.1 "Download failed" on a DE440 chunk

The JPL FTP-over-HTTPS server occasionally returns transient errors during high-traffic periods. The standard fix is to:

1. Wait a minute.
2. Click **Download** on the affected card again.

If the failure is persistent, check the URL printed in the error dialog against the canonical pattern `https://ssd.jpl.nasa.gov/ftp/eph/planets/ascii/de440/ascpXXXXX.440` and adjust it in the card's URL field if the server has moved.

13.1.2 "Download failed" on a GRGM1200B file

The PDS Geosciences Node at Washington University hosts the GRGM1200B coefficient files. If the canonical URL (`https://pds-geosciences.wustl.edu/grail/grail-l-lgrs-5-rdr-v1/grail_1001/shadr/`) returns a 404, the file has been relocated. Open the PDS Geosciences GRAIL landing page in a browser, navigate to the `shadr/` subfolder, find the current location of `gggrx_1200b_sha.tab` and `gggrx_1200b_sha.lbl`, and paste the new URL into the card.

13.1.3 "Conversion failed (exit 1)"

The automatic conversion runs `spody.exe convert ephemeris` under the hood. The two common failures:

- **Missing files**: at least one ASCII chunk was not actually downloaded successfully. Hit **Refresh** at the bottom of the wizard and verify every required card is green. Re-download any that are not.
- **`spody.exe` is not configured**: the wizard delegates the conversion to the engine, and if the engine path is empty or invalid the conversion cannot run. Open **Settings › Paths**, set the path to `spody.exe` (it lives next to `spody-gui.exe` in the bundle), and reopen the wizard.

13.1.4 Wizard reopens at every launch

This means the **hard run-guard** decides one or more required files are not in place. Common causes:

- The data folder is on a removable drive that is not currently mounted.
- A virus scanner quarantined the downloaded files (some enterprise scanners are aggressive about large binary downloads from FTP-derived URLs).

- The bundle was moved to a different folder after the wizard was completed, and the registered data folder no longer points at a writable location.

Open the wizard and inspect the status of every card to identify the missing file; restore the file or change the data folder via the **Change…** button.

13.2 TOML form

13.2.1 "Form has invalid values" placeholder in the preview

A field's value cannot be coerced to its declared type — typically a non-numeric character in a float or integer field. Look for fields with the red border and fix the offending value. The TOML preview restores automatically once the form is again in a valid state.

13.2.2 **Validate** badge shows `(spody binary not set)`

The engine path is not configured. Open **Settings › Paths** and point at `spody.exe`.

13.2.3 **Validate** badge shows `(data not ready)`

The hard run-guard intercepted the Validate call because one or more required data files are missing from the data folder. The button next to the badge invites you to open the wizard; do so and complete the downloads.

13.2.4 **Validate** badge shows `(form has invalid values)`

The form contains a value that cannot be serialised. This is distinct from "out of range": the type conversion itself failed (e.g. typing letters in a numeric field). Fix the indicated field; the badge clears on the next edit.

13.2.5 **Validate** badge stays red after a fix

The previous red badge persists until you press **Validate** again. There is no auto-revalidation on edit; only a successful explicit Validate turns the badge green.

13.2.6 Floats render in scientific notation after a load

This is intentional. The form reads the float, then re-emits it through Python's `repr()` to ensure a round-trippable representation; the result may look like `1e-5` even if you typed `0.00001`. The two are equal. No information is lost.

> *Earlier versions of the form attached a `QDoubleValidator` that aggressively normalised user input (`1e-5` ⇒ `1e-05`, `0.00001` ⇒ `1e-05`). The current version intentionally does not validate floats with a Qt validator and keeps the typed text verbatim until the next Load.*

13.3 Run mode

13.3.1 "spody binary not set"

The engine path is empty. Open **Settings › Paths**.

13.3.2 "Cannot run: data not ready"

Same as the validate counterpart: the run-guard intercepted the launch because data files are missing. The dialog offers to open the wizard; accept the offer and complete the downloads.

13.3.3 Run starts but stops within seconds

Open the terminal pane on the right; the engine printed a one-line diagnostic before exit. Common cases:

- `error: harmonics_file: ENOENT` — the `[force_model].harmonics_file` path inside the TOML does not resolve. Remember the path is relative to the TOML's directory.
- `error: ephemeris.file: ENOENT` — same for the ephemeris path.
- `error: integrator step h dropped below h_min_s` — the adaptive controller could not maintain `rel_tol` and gave up. Try a smaller `rel_tol` (so the controller is happier with larger steps) or a smaller `h_min_s`.

13.3.4 Run is much slower than expected

Two common amplifiers:

- A **harmonics degree** above 200 in a long propagation; cost scales as $O(N^2)$, so going from N=80 to N=200 makes the engine 6× slower without proportional accuracy gain.
- A **tight** `rel_tol` (smaller than 1e-13) below the engine's resolution; the controller spends all its time rejecting steps.

Tune both first; engine-side performance is rarely the actual bottleneck.

13.3.5 Terminal pane stops updating mid-run

A long-running step (heavy harmonics evaluation, especially with event detection enabled) can keep the engine inside one numeric routine without producing console output for several seconds. The status bar still increments the elapsed-time counter, which is your hint that the process is alive even when the terminal is quiet. Worry only if the elapsed-time counter freezes too.

13.4 Analysis tab

13.4.1 File tree shows no files in the working directory

The recursive `.bin` scan goes three folders deep. A binary buried deeper does not appear; either move the file up or use the **+ Add external file…** button to register it explicitly.

13.4.2 "Unknown file" on a `.bin` you know is good

The kind detection reads the first 8 bytes of the file and matches them against the four supported magics (`SPDYOUT_`, `SPDYACC_`, `SPDYEVT_`, `SPDYEVTB`). A `.bin` produced by an external tool that does not write a SpOdy magic will be flagged; convert the file into one of the supported formats first.

13.4.3 Plot button is missing

There is no plot button. Click a leaf in the plot tree to render it; the dispatch is on the click itself.

13.4.4 Sun-arrow row is missing

The Sun-arrow row appears only when the **active plot is 3D**. The currently-selected leaf in the plot tree is 2D (which covers every plot except `3D orbit + Moon` and `Impact 3D on Moon`); pick a 3D plot and the row reappears.

13.4.5 Impact-view says "No input.toml found" / "Could not locate ephemeris"

The four impact lat/lon views (equirect, Mollweide, density heatmap, 3D) read the run-folder `input.toml` snapshot for `et_start_s` and `[ephemeris].file` — both are required to project ICRF impacts onto the Moon's body-fixed PA frame. The snapshot is normally written by the engine at every invocation (chapter 7). It can be missing when:

- the events file you loaded came from a *pre-run-folder* run (an older spody build, before the layout refactor);
- the events file was hand-copied out of its original run folder to another location.

In both cases, copy the matching `input.toml` next to the `<batch>_events.bin` and reload the file. The ephemeris path inside that TOML must still resolve from somewhere reachable; the resolver tries the snapshot directory, then two ancestor levels up (where the original TOML usually lived), then the current working directory.

13.4.6 Moon texture not showing in the impact map / 3D scene

The Moon texture is an **optional** wizard asset and is not downloaded by default (chapter 3). Open the Setup wizard and press **Download** on the *Moon texture* card. The impact map falls back to a plain background and the 3D scene to a flat-grey sphere when the texture is absent; no error is raised.

13.4.7 "Diff needs exactly 2 files (currently selected: 1)"

You clicked a diff leaf with only the loaded file in the selection. **Ctrl-click** another file in the file tree to add it to the selection, then re-click the diff leaf.

13.4.8 "Diff requires overlapping time windows"

The two selected files were propagated over disjoint time spans. This is fundamental: there is no way to compare two trajectories that do not coexist in time. Re-run one of them so the time windows overlap.

13.4.9 "Tile cannot mix single-file and diff plots"

The tile dispatcher requires the selection to be uniform: either all single-file specs (rendered against the loaded file) or all diff specs (rendered against the two-file selection). Clear the selection (Ctrl-click to unselect) and rebuild it.

13.4.10 3D viewer is unresponsive

The first VTK render after a fresh launch can take a few seconds while OpenGL is initialised. Subsequent renders are fast. If the viewer remains unresponsive after ten seconds, your graphics adapter may have an unusual OpenGL driver; switch to a 2D plot to confirm the rest of the application is healthy.

13.4.11 Moon sphere is flat grey instead of textured

The 3D scene falls back to the flat-grey sphere when no Moon texture is configured. Open **Settings › Paths › Moon texture (3D view)** and point at an equirectangular Moon image (JPEG or PNG). The texture is reloaded on the next plot dispatch; no application restart is needed.

13.5 Settings

13.5.1 Persistent paths are not remembered across launches

Settings are stored in the Windows registry under `HKEY_CURRENT_USER\Software\SpOdy\SpOdy`. If you are running as a guest or restricted user, registry writes may be blocked by policy. Run the application as your normal interactive user.

13.5.2 Resetting the application to a clean state

Two steps:

1. Delete the `data/` folder next to `spody-gui.exe`.
2. Delete the registry key `HKEY_CURRENT_USER\Software\SpOdy\SpOdy` via `regedit`.

The next launch behaves exactly as a fresh install: the wizard pops, no recent files are remembered, no paths are configured.

14. Glossary

A short list of the technical terms used through the manual, defined where SpOdy uses them. Cross-references to the chapter that introduces each term in detail are given in square brackets.

Accelerations binary. — A `SPDYACC_`-format output file listing, at every record, the contribution of each active force to the total acceleration. Used by the **Per-force breakdown** plot. [Chapter 9.]

Accelerations file. — The `output.accelerations_file` TOML field, naming the on-disk accelerations binary. [Chapter 6.]

Adaptive step. — The mode of the RKDP integrator in which the step size `h` is chosen at every step to maintain `rel_tol`. Opposed to fixed-step integration. [Chapter 6.]

A/m. — Area-to-mass ratio, in m²/kg. The single parameter SpOdy needs to compute SRP acceleration when used in debris mode; in spacecraft mode it is derived from `area_m2 / mass_kg`. [Chapter 6.]

Argument of periapsis (`ω`). — The classical orbital element measuring the angle from the ascending node to the periapsis, in the orbital plane. [Chapter 9.]

Batch. — A multi-case parameter sweep, driven by a CSV file and a column-to-target mapping. [Chapter 7.]

Cases file. — The CSV file describing one case per row, referenced from `batch.cases_file`. [Chapter 7.]

Central body. — The body the propagation is centred on (today only the Moon is supported). Defines the inertial frame the state vector is expressed in. [Chapter 6, chapter 10.]

Central-inertial frame. — The reference frame the engine propagates in: origin at the central body's centre, axes aligned with the ICRF (J2000), right-handed. [Chapter 10.]

Coverage profile. — The user's choice of ephemeris time window in the setup wizard: *Modern era* (1950 – 2050) or *Full pack* (1550 – 2650). [Chapter 3.]

`Cr`. — Coefficient of reflectivity. `1.0` = pure absorbing, `2.0` = pure mirror, intermediate values for partial diffuse reflection. SpOdy uses the cannonball SRP model with a single Cr. [Chapter 6.]

Cross-track. — The third RIC axis: perpendicular to the orbit plane, positive along the angular-momentum direction. [Chapter 10.]

Cubic Hermite. — Interpolation scheme used by the diff dispatcher when the two trajectories are on different time grids. Uses position and velocity at each sample to produce a piecewise cubic that matches both at sample points. [Chapter 11.]

Data dir. — The folder the wizard populates with the external data files. By default a `data/` subfolder next to `spody-gui.exe`, overridable via the wizard. [Chapter 3.]

DE440. — The JPL planetary ephemeris model SpOdy uses for third-body positions. Distributed as ASCII chunks; the wizard converts these into the internal `de440.spody` binary. [Chapter 3, chapter 6.]

Delta mode. — The batch column mapping mode in which the CSV cell value is *added* to the TOML's nominal value, as opposed to *overriding* it. Useful for Monte-Carlo perturbations. [Chapter 7.]

Diff plot. — A plot in the `Diff (pick 2 files)` folder of the plot tree. Subtracts trajectory B from trajectory A sample-by-sample (with cubic Hermite interpolation when the grids do not match). [Chapter 9, chapter 11.]

Eccentricity (`e`). — The classical orbital element measuring how non-circular the orbit is. `e = 0` for circular, `0 < e < 1` for elliptical, `e = 1` for parabolic, `e > 1` for hyperbolic. [Chapter 9.]

Eclipse threshold. — The sunlight-fraction value at which an eclipse event fires. Configured in `[events]`. [Chapter 6.]

Engine. — The C executable `spody.exe` that runs the actual numerical integration. Distinct from the GUI (`spody-gui.exe`), which is a Python application that drives the engine. [Chapter 1.]

Ephemeris. — Time-tabulated planetary positions and velocities used to compute third-body gravity. SpOdy uses JPL's DE440 in an internal binary format. [Chapter 3, chapter 6.]

Event. — A discrete occurrence detected by the engine during a propagation: an IMPACT against a celestial body surface, or (when enabled) an ECLIPSE entry/exit. [Chapter 6.]

`et_start_s`. — The start epoch of the simulation, in TDB seconds past J2000. [Chapter 6.]

Form. — The Run tab's left pane: one widget per TOML field, with range checks and live preview. [Chapter 5.]

`GRGM1200B`. — The recommended lunar spherical-harmonic gravity coefficient set, derived from the NASA GRAIL mission's observations. SpOdy reads the PDS-distributed `.tab` file at runtime. [Chapter 3.]

Hard run-guard. — The runtime check that refuses to launch the engine if any required data file is missing. It also blocks the **Validate** button for the same reason. [Chapter 3, chapter 13.]

Harmonics degree. — The truncation order `N` of the spherical-harmonic gravity expansion. Higher = more accurate but O(N²) more expensive. [Chapter 6.]

ICRF. — International Celestial Reference Frame, the modern realisation of the J2000 inertial axes. SpOdy's central-inertial frame is ICRF-aligned. [Chapter 10.]

In-track. — The second RIC axis: perpendicular to the radial direction, in the orbital plane, in the half-plane containing the velocity. For circular orbits this aligns with the velocity direction. [Chapter 10.]

Inclination (`i`). — The classical orbital element measuring the orbit plane's tilt with respect to the reference plane (the ICRF XY plane in SpOdy). [Chapter 9.]

Inline table. — The TOML syntax `{ key = value, &hellip; }`, used in `[batch.columns]` for delta-mode column descriptors. [Chapter 7.]

J2000. — The standard epoch 2000-01-01 12:00:00 TT, used as the zero of TDB-seconds time scale. [Chapter 6.]

`mu`. — Gravitational parameter, in km³/s². For the Moon: `4902.800066`. [Chapter 9.]

Override mode. — The batch column mapping mode in which the CSV cell value *replaces* the TOML's nominal value. The default. [Chapter 7.]

PA (Moon Principal Axes). — The Moon's body-fixed reference frame in which the GRGM1200B harmonics are expressed, and the same frame the impact lat/lon views project IMPACT events into. The rotation from ICRF to PA is `Rz(psi) · Rx(theta) · Rz(phi)` where `(phi, theta, psi)` are the lunar mantle Euler angles from DE440. [Chapter 10.]

Overlay-safe. — Property of a plot that draws exactly one line per file, so an N-file overlay produces N lines (legible) rather than 3N or more (illegible). [Chapter 9.]

Plot tree. — The lower-half tree in the Analysis tab's left column, listing the plots applicable to the currently-loaded file's kind. [Chapter 8.]

RAAN (`Ω`). — Right Ascension of the Ascending Node, the classical orbital element measuring the angle from the reference X axis to the orbit's ascending node. [Chapter 9.]

Radial. — The first RIC axis: from the central body to the spacecraft, positive outward. [Chapter 10.]

`rel_tol`. — The relative error tolerance the integrator maintains per accepted step. Smaller = more accurate, slower. [Chapter 6.]

RIC. — Radial / In-track / Cross-track. A local orbit frame defined sample-by-sample from the state vector. Used by the RIC diff plot. Equivalent to the RSW frame in Vallado's nomenclature and the Hill frame in Clohessy-Wiltshire studies. [Chapter 10.]

RKDP / RKDP45. — Runge-Kutta Dormand-Prince 5(4), the adaptive integrator SpOdy uses. The 5/4 numbers refer to the order of the embedded error estimate. [Chapter 6.]

Run-guard. — See *Hard run-guard*.

SPDYOUT_ / SPDYACC_ / SPDYEVT_ / SPDYEVTB. — The 8-byte magics of SpOdy's four binary output formats: trajectory state vectors, per-force accelerations, per-run events log, and the batch-aggregated events log respectively. [Chapter 7, chapter 12.]

spopy. — Pure-Python (numpy-only) re-implementation of spody-core's read-side helpers (DE440 ephemeris reader, lunar libration, ICRF ⇔ PA rotations). Bundled under `python/spopy/` and used by the Analysis tab's impact views to project IMPACT events onto the lunar surface at interactive speed without spawning a subprocess. Bit-identical to the C implementation (validated at landing). [Chapter 10.]

SPICE. — NASA NAIF's toolkit and associated kernels. SpOdy's planetary ephemeris is derived from SPICE-format DE440 data; the recommended validation reference for any SpOdy run is a SPICE-derived trajectory of the same epochs. [Chapter 11.]

SRP. — Solar radiation pressure. SpOdy uses the cannonball model: a single A/m and Cr, with eclipse cuts when enabled. [Chapter 6.]

Step mode. — The `output.mode = "step"` value: one output record per accepted RKDP step, irregular sampling. Opposed to fixed-step output. [Chapter 6.]

Target. — A dotted path inside the TOML schema that a batch column can override. E.g. `spacecraft.mass_kg`, `debris.am_srp`, `initial_state.position_km[0]`. [Chapter 7.]

TDB. — Barycentric Dynamical Time, the time scale the DE440 ephemeris is expressed in. SpOdy's `et_start_s` is in TDB seconds past J2000. [Chapter 6.]

Third body. — A celestial body whose gravity perturbs the central-body two-body solution but which is not itself propagated. Configured in `force_model.third_bodies`. [Chapter 6.]

Tile dashboard. — The multi-subplot rendering mode triggered by the ▦ **Tile selected (N)** button: N plots from the multi-selection are drawn as a grid in a single matplotlib figure. [Chapter 8.]

TOML. — The input file format SpOdy reads. A human-readable, strongly-typed configuration syntax; see https://toml.io/ for the specification. [Chapter 6.]

True anomaly (`ν`). — The classical orbital element measuring the angle from the periapsis to the current position, in the orbital plane, measured in the direction of motion. [Chapter 9.]

Two-body. — The Kepler-problem central acceleration `-mu * r / |r|^3`. Always present; the rest of the force model is added on top. [Chapter 6.]

Validate. — The action of running `spody.exe validate` against a TOML to check it parses and passes consistency rules, without actually propagating. [Chapter 5, chapter 12.]

Vis-viva. — The relation `|v|^2 = mu * (2/|r| - 1/a)` that ties together speed, distance, and semi-major axis on a Keplerian orbit. Used by the orbital-elements solver to derive `a` from the state vector. [Chapter 9.]

Working directory. — The folder the Analysis tab scans for `.bin` files. Set explicitly via **Change…** or auto- filled from the loaded TOML's parent. [Chapter 8.]